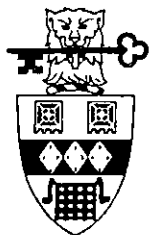


VECTOR

Conference Bumper Issue!

- 16-page Educational Supplement 17
- APL91: Advance Information 33
- Dyalog for MS DOS explored 49
- Iverson's new Notation 67
- De Kerf on $\square NC$ 127



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.7 No.1 July 1990

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (ideally with a photograph of the author and a brief biographical note). Unless otherwise specified, each item will be considered for publication as a personal statement by the author.

Please supply as much material as possible in machine-readable form, ideally on an IBM PC compatible diskette (any format). APL code should be kept separate from the text, and can be accepted as camera-ready copy, or in workspaces from APL*PLUS/PC, IBM APL2/PC or Dyalog APL.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1990-91

Category	Fee	Vectors	Passes
UK Private	£10	1	1
Overseas Private	£18	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£85	10+	5
Overseas Corporate	£140	10+	
Sustaining	£360	50+	5
Non-voting Student	£5	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £125 for half-page or less (There is a £50 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to, Alison Chatterton, whose address is given on the inside back cover.

CONTENTS

Page

EDITORIAL

Reliability of APL Systems	Jonathan Barman	3
----------------------------	-----------------	---

APL NEWS

Quick Reference Diary		5
APL Course Diary		7
General Correspondence		9
British APL Association News	Graham Parkhouse	9
News from Sustaining Members	Alison Chatterton	12
The Education VECTOR	Alan Sykes	17
APL91: Conference Announcement	Curtis Jones	33

REVIEWS

APL Product Guide	Alison Chatterton	38
Dyalog APL for MS DOS	Adrian Smith	49
Powertools/CUA	Adrian Smith	54

RECENT MEETINGS

Vendor Forum: 20th April 1990	Adrian Smith	60
-------------------------------	--------------	----

SPECIAL FEATURE

J: An Informal Introduction	Kenneth E. Iverson	67
-----------------------------	--------------------	----

GENERAL ARTICLES

The APL Statistics Library Project	The ASL Team	80
Some Questions of Programming Style for the APL Statistical Library	David Appleton	90
APL-2741	Adrian Smith	96

TECHNICAL SECTION

Hackers Corner: Lots More VGA Colours	Adrian Smith	110
Technical Correspondence		111
APL Standards Update	David Ziemann	124
A Survey of $\square NC$	Joseph L.F. de Kerf	127
Index to Advertisers		143
Submitting Material to Vector		143

APL.68000 Level II

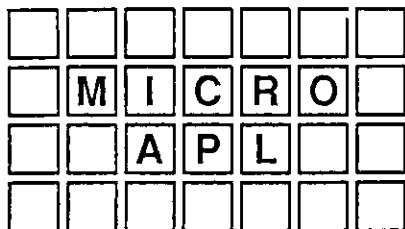
A fully configured APL2

MicroAPL is proud to announce the immediate availability of APL.68000 Level II. APL.68000 Level II combines the features of APL2 with the user-friendly extensions of our current APL.68000 product. APL.68000 Level II is available on all the computers that run APL.68000 - the Apple Mac, Commodore Amiga, Atari ST, NCR Tower and many other multi-user Unix computers.

Main features:

- Close conformance with APL2
- Most complete non-IBM implementation of APL2
- Nested Arrays and Mixed Arrays
- $\leftarrow$ $\rightarrow$ $\cdot$ $\square$ $\equiv$ $\leq$ ϵ $\uparrow$
- System Functions include $\square EA$ $\square ES$ $\square ET$ $\square EM$
- Workspace interchange via $)IN$ $)OUT$
- Full upwards compatibility with APL.68000
- No conversion to existing APL.68000 code

To check pricing and to order your copy, contact MicroAPL or your local dealer.



MicroAPL Ltd.
South Bank Technopark
90 London Road
LONDON, SE1 6LN
Telephone: 071 922 8866

Editorial: Reliability of APL Systems

by Jonathan Barman

Are APL systems less reliable than systems written in other languages? There was some dismay at the last Vector working group meeting when I indicated that this was the topic of my first editorial, because it looked as if I would be knocking APL. Whilst I think that many APL systems are unreliable, I do not think that this is necessarily caused by APL. A programmer's ability to produce reliable code seems to be much more important than the programming language being used.

APL is a superb language for quick, off the cuff, solutions which are to be used by the writer and no one else. This is the major factor which contributes to the impression that APL systems can be unreliable. However, the impression can be equally true of any language that is superb for quick, off the cuff, solutions - but which language falls into that category? Do you expect quality systems from Lotus 1-2-3? You can get both brilliant and rotten systems written in any language you care to name.

APL is also excellent for writing large quality systems. However, moving from a small quick solution to a quality system is much more time consuming than many people realise. Unless one has had the bitter experience of bugs coming back to haunt one, it is quite difficult to appreciate the care, time and trouble that goes into getting a system right. Time means money, and a quality system is going to be expensive.

Big companies producing major systems keep statistics of the number of errors encountered at the various stages of a product's life cycle: during development, during formal testing, and when the product is in the field. These statistics are compared with some measure of the size and complexity of the code, so that trends and performance can be assessed. I am not aware of any such statistics having been published in the APL world, so I have no hard facts to compare APL with other languages. Perhaps someone will write and tell us what is available?

In the absence of hard facts one is left with perceptions; it is easy to see how APL can get a bad name. For example, supposing there is a key business problem which requires a quick solution: someone creates a solution in APL, and it turns out to be really useful. The system is gradually extended until it is quite large - the users of the system are in the same department as the author, and when something goes wrong it can be fixed immediately. Eventually the author leaves

the department and the users are left in the lurch when the system crashes, the DP department is then expected to get them out of the fix.

Exactly the same scenario happens with Lotus, Basic etc., however the problems are not normally quite as bad. The sheer power of APL allows really significant systems to be developed which would be quite difficult to achieve in other languages.

Another charge made against APL is that it is a hackers language. To some extent this is true, but I am not sure that it does or should matter. APL makes it easy to try out ideas. For some reason the trial and error approach is seen as a bad thing, unless you call it prototyping. There are many aspects to this problem; producing code without proper thought leads to stupid errors, time spent at the design stage is usually very worth while but can be overdone. How does one achieve a balance?

Searching for a solution to a problem is a very personal and idiosyncratic thing. Some like to mull over the problem in the bath, others like to bash away at the keyboard to try and generate ideas. Nobody can say which is best! APL allows, and even encourages, the latter approach; the risk is that it is possible to find an APL solution which works with the examples given, but which fails in the general case. Hackers find an apparent solution and barge in and change the system without a second thought. Suffice to say that they tend to produce poor quality systems, but hackers often produce the beginnings of useful systems which would never see the light of day in the traditional DP environment.

Prototyping is very helpful in crystalising user's ideas. It is quite extraordinarily difficult for a user to specify a large system in the complete detail needed by the programmer. Knocking up a sample system which can actually be used is where APL scores. However, there is usually an extreme reluctance to go back and rewrite any code that is working, so a prototype very often ends up as the final system, with only relatively minor changes. Sometimes the APL prototype system becomes to be viewed as the final system and when it creaks at the seams APL gets a bad name. In some ways the special prototyping languages score over APL in that they force a rewrite. At that stage all the proper things can be done, like writing the user manual first, and getting the initial design right.

In conclusion, I think that the reliability of APL systems is no worse than those written in other programming languages, but that sometimes APL gives the impression of being worse because of its power, speed and flexibility.

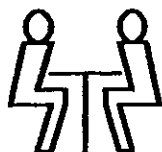
Quick Reference Diary

Date	Venue	Event
13th - 17th August 1990	Denmark	APL90: APL for the Future
28 September	London	probable BAA Meeting; please await confirmation of date and venue.
19 October	York	BAA Field Trip to Rowntrees
23 November	London	probable BAA Meeting

British APL Association meetings begin at 2.00pm and close at 5.30pm. Please note that we are urgently seeking a new venue for the 1990/91 program, and you should confirm dates and venues before travelling.

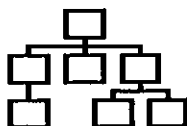
The field trip to Rowntrees is **definitely on**; please contact Adrian Smith on 0904-653071 extn 3382 to book your place, as space will be limited.

A single source for APL



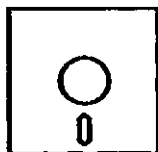
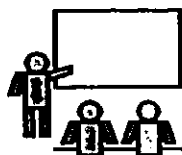
Advice

Design



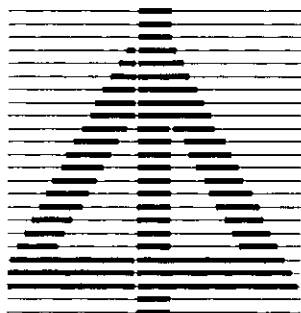
**Application
Development**

Education



APL software

Support



COCKING & DRURY LTD.
THE APL PROFESSIONALS

180 Tottenham Court Road
London W1P 9LE
Tel: 01 - 436 9481

(From May 1990, our STD code will be 071)

APL Training Courses for 1990

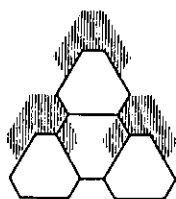
compiled by Adrian Smith

Dates shown cover the period from July to November 1990. For confirmation of dates and/or further details please contact the vendor directly.

Level	Company	Days	Dates
Beginners	Cocking & Drury	3	11 Sept
	MicroAPL	1	14 June, 2 Aug, 4 Oct
APL*PLUS/PC	MicroAPL	1	19 July, 13 Sept, 1 Nov
Intermediate	MicroAPL	1	5 July, 9 Aug, 11 Oct
APL*PLUS II conversion	MicroAPL	1	7 Jun,
			23 Aug, 8 Nov
Advanced (Nested Arrays)	MicroAPL	1	12 July, 6 Sept, 18 Oct
	Cocking & Drury	2	25 Sept
System Design	Cocking & Drury	4	17 Sept
Statgraphics	Cocking & Drury	2	2 Oct

Dates for Future Issues of VECTOR

	Vol.7 No.2	Vol.7 No.3	Vol.7 No.4
Copy date	7th Sept 90	1st Dec 90	1st March 91
Ad booking	21st Sept 90	8th Dec 90	12th March 91
Ad Copy	30th Sept 90	23rd Dec 90	21st March 91
Distribution	October 90	January 91	April 91



APL PEOPLE

JOBS!

JOBS!

JOBS!

As THE PREMIER APL employment agency, more APL vacancies are filled through us than any other agency.

Established over eight years ago, our understanding of the APL market-place is unrivalled. We always have a good selection of vacancies available throughout the UK.

To find out what we can do for you, contact

Jill Moss on 0225 462602 during office hours
or 0225 333618 eves & w/ends

or write to her at

The Old Malthouse, Clarence St., Bath BA1 5NS

British APL Association News

from Graham Parkhouse

Minutes of the AGM of the British APL Association Friday 8 June 1990 at Over-Seas House, London

The meeting commenced at 14.05 with about 25 people present. John Sullivan, the Treasurer, was in the chair standing in for Peter Donnelly who was suffering from a damaged achilles tendon.

1. Approval of the minutes of last year's AGM.

The minutes are recorded in VECTOR 5.4. Anthony Camacho proposed they be approved and Ray Cannon seconded the proposal. By a show of hands the meeting confirmed its approval, nem. con.

2. Chairman's report.

John Sullivan read the report that Peter Donnelly had prepared. Please turn to page 11 for the full text.

3. Treasurer's Report.

A provisional unaudited statement of income and expenditure for the year ended 30 April 1990 was circulated to those present. The Treasurer pointed out the surplus of income over expenditure of £8,500, greater than last year's £1,800, which was largely due to recovery of last year's debts. He had had great difficulty understanding last year's accounts, particularly concerning subscription income. Audited accounts would be published in VECTOR as soon as they were available.

The Treasurer asked for questions. Mel Chapman asked what the committee were doing about increasing the membership, which the Treasurer had reported were declining in numbers. Jill Moss, the Recruitment Officer, explained what had been done over the last year to increase membership numbers, including adverts in VECTOR, a competition with a prize of a free place at APL90 for someone drawn from those who had introduced a new member to the Association, an advert in BCS's newsletter, a mailshot to lapsed members and soon Alison Chatterton is going to send a mailshot to those on APL People's mailing list who are not members of the Association asking them to consider the benefits of joining. Graham Parkhouse said that the figures he had to hand

showed that membership numbers for 89/90 were very similar to those for 88/89. David Ziemann suggested it would be a good idea to have membership information on an overhead at the AGMs, and that membership invitations should be included in the VECTORS to be distributed at APL90. Alan Sykes said that six people had responded to the suggestion that they should join the BAPLA as a result of the questionnaire in Education Vector.

Herbert Walton asked whether the Association kept in touch with lapsed members by continuing to send them details of meetings etc. John Sullivan and Adrian Smith both confirmed that this did happen.

Adrian Smith asked why we need members any more in view of the small proportion of revenue coming from subscriptions. John replied that the membership was the Association's reason for existence and therefore increasing the membership was important. Mel Chapman, speaking from his knowledge of the BCS, said they expected their specialist groups to balance expenditure with income, and that large reserves should not be an excuse to lose interest in subscription income. John Searle suggested that it would be entirely compatible with the aims of the Association to spend some of its reserves on securing new members.

Herbert Walton remarked that he could not see in the accounts any contribution to the BCS. John Sullivan replied that the only contribution is from the BCS to the Association in the form of £500 worth of mailing expenses. There is no affiliation fee.

Jonathan Martin noted the increase in committee expenditure which the Treasurer explained included payment for Alison Chatterton's hours, which were up on last year's. Graham Parkhouse took the opportunity to record his appreciation for her hard work: he was impressed with the amount of work she got through in the hours she booked.

4. Any other business.

There were no further questions and the AGM closed at 14.25.

Graham Parkhouse
9 June 90

Chairman's Report

Peter Donnelly

In my report last year I admitted that despite considerable individual efforts the administration of the British APL Association left much to be desired. I am happy to say that due to the continuing efforts of Alison Chatterton and to the excellent teamwork of this year's committee the administrative problems have largely been resolved.

On behalf of the Association I would like to thank Graham Parkhouse for his hard work and boundless enthusiasm. Graham has been a really excellent secretary and has served the association well.

Adrian Smith is also retiring this year as editor of Vector, but will continue to be involved in its production. During his period as editor, VECTOR has always been interesting, topical, and a 'good read'. It is highly regarded by APLers throughout the world and we can justifiably be proud of it. Adrian and the Vector working group deserve our praise and thanks. Adrian's successor as editor is Jonathan Barman, a highly experienced and respected figure in the APL community. Thank you Jonathan for taking on such a demanding role; I wish you luck.

Of great significance last year was the instigation of the APL Statistics Library project. This project is partly funded by the British APL Association, but is otherwise dependent on the unpaid efforts of individuals. The project is well managed and is by all accounts going very well. Thanks to all involved.

The meetings last year benefited from an increased use of live presentations using colour projection equipment. The committee intends to encourage this trend still further and is actively seeking an alternative location where such equipment is provided at a more reasonable price.

To conclude this rather short report, I would like to thank the members of the committee for their hard work and support last year. Few committees in recent years have been so free of politics and willing to get on with the job. It has been a pleasure to serve on it.

News from Sustaining Members

Compiled by Alison Chatterton

APL People Limited

Enquiries have started to come in about the interface between APL and IBM's AS (Application System) product. For those of you who are not aware, APL People are collaborating with IBM in the development and marketing of this interface to Information Centres. We offer assistance to those wishing to be able to use the APL on their mainframe in conjunction with AS, and have the full backing of IBM in this.

On the recruitment front, at the time of writing, we still have a number of prestigious positions in the City waiting to be filled. Anyone who wishes to be considered should have excellent APL development skills, be keen to work in a fast-moving dynamic environment, and be prepared to put in 110% effort in return for a generous remuneration package! A knowledge of 'C' and a financial or banking background would be a distinct advantage as well.

We invariably have a range of vacancies for people with APL skills and are always pleased to hear from companies seeking to recruit, and individuals contemplating a move, whether they be in the UK or overseas.

Expansion plans are afoot too on the Applied Production Logic front (APL People's sister company). We have recently had a major order for our Production Engineering software (now numbering some 27 modules in total) from a company in Scotland. This has prompted us to investigate the possibility of setting up another office in the North of England to service this part of the country better (and save on travel costs for our salesmen!)

Dyadic Systems Ltd

Dyadic is pleased to announce Dyalog APL Version 6.0 for Unix systems. This is functionally identical to the DOS/386 implementation and features the same integrated development environment consisting of a window-based Session, Editor, and Tracer. Apart from external system interfaces, Dyalog APL applications are now 100% compatible between DOS/386 and Unix platforms.

In addition to functional improvements the Unix implementation also benefits from the performance optimisation techniques which were introduced in Release 2 of the DOS/386 version. These result in a significant improvement in speed for basic arithmetic primitive functions working on large arrays. Typically this means a speed-up by a factor of 2 in the arithmetic benchmarks published by STSC.

There are in fact two flavours of Version 6.0 for Unix. The first is a multi-user version designed to support APL/ASCII terminals and/or PCs running terminal emulators. In this version, the APL development environment works in a surprisingly effective manner despite the inherent limitations of ASCII terminals.

The second flavour is Dyalog APL/X which is designed to take full advantage of X-windows, the widely accepted basic window manager for Unix workstations. The product is also designed to be consistent with OSF/Motif, which sits on top of X-Windows and provides a standard user interface compatible with Presentation Manager.

In Dyalog APL/X the Session, Editor and Tracer operate in separate X.11 windows which may be moved and resized using the mouse. The User Screen managed by `USM` also becomes a separate moveable and resizable window. New trappable event codes have been added to allow such user actions to be handled under program control. For example, if the user resizes the `USM` window your APL function can trap a resize event, query the new window dimensions, and reformat the display accordingly. Controls are provided to allow the APL programmer to specify what a user may or may not do.

Dyalog APL/X also includes an Auxiliary Processor which provides a direct interface to X.11 functions. This will initially provide graphics facilities, but is intended to be developed to include access to the Motif "Widget" set, a toolkit of high-level use-interface functions such as dialog boxes, menus, buttons, and so forth. The Auxiliary Processor will allow an APL application to pop-up a new window and display graphical information within it.

Dyalog APL/X will initially be available on IBM Risc System/6000, Hewlett-Packard and Sun workstations. In due course the product will be ported to other popular X.11/Motif platforms. Full details and dates of availability will be announced at APL90.

In response to customers requests Dyadic has developed a software interface that allows Dyalog APL to be called from a C-program. The interface is very simple and requires only a few additional lines of C-code in the calling program. It allows the C-program to invoke Dyalog APL, `)LOAD` a workspace, and then run any function and capture its result. Data is transferred efficiently between the C and APL tasks in binary form. The interface can be developed to support a network environment.

Dyadic is also proud to announce a change of ownership following a **Management Buyout** on March 31st 1990. The new directors and shareholders of the company are Pauline Brand, Peter Donnelly and John Scholes who have been with Dyadic for between ten and thirteen years. The company is delighted by its newly won independence, will continue to develop innovative APL products, and looks forward to providing an even more effective service to its current and future customers. By the time this edition of Vector is printed, Dyadic will have moved to new and larger premises. If you miss the mailshot, the new address can be obtained by calling the company's old number 0420-87024.

Cocking and Drury Ltd

We have to begin our news with an apology this time. The release date of June that we announced for the APL*PLUS II System Version 3 was the US release date - international release will be during August.

Cocking & Drury will be previewing the new features of Version 3 at a presentation in our London office on July 12 - anyone who would like to attend should call Beverley Satterley on 071 436 9481 to reserve a space.

Our consultants are spending an increasing proportion of their time working with clients migrating applications from mainframes to 80386 machines running the II System.

We have also noticed an increasing level of interest in the APL*PLUS UNIX product to run on Sun Sparc and other RISC stations.

At the more elementary level the APL Fundamentals courses run here in London continue to attract a large number of learner APLers. The first Nested Array course in May proved popular and we are now taking bookings for that course

on July 17/18 and September 25/26. The final System Design course before it is broken into separate units will take place on September 4-7.

Paul Bolwell - our current sandwich year student - will be leaving us at the end of July for a holiday before returning to college. His replacement, Steve Hill, joins in mid-July. And Steve Weller, the sandwich year student from 1988/89, will be joining the Company as a permanent employee in July too.

MicroAPL Ltd

MicroAPL is delighted to be able to use the pages of Vector to announce a major new software product - APL68000 Level II. For the last year or so we have to all intents and purposes given the appearance of being preoccupied with the user interface of our APL68000 product - notably on the Mac and Amiga versions. This emphasis does indeed reflect our view that the user interface is the most neglected area of APL but we have not been completely ignoring the development of our APL interpreter.

There is a consensus that a 'second generation' definition for APL exists and that this includes, amongst other things, concepts such as nested arrays and heterogeneous objects. The definition of a second generation APL doesn't go much further than that because of the differences in the specifications of those second generation APLs that are on the market.

MicroAPL has made a major contribution to the portability of second generation APL code. APL68000 Level II is an implementation of APL2 for processors which belong to the 68000 family. As with APL68000 we have attempted to use as our model the version of APL that we regard as the de-facto standard. That standard is, in our estimation, IBM's APL2. In the same way that the original APL68000 represents a useful superset of APLSV, APL68000 Level II offers a faithful implementation of IBM's APL2 (see below for what we think that is!) together with a number of vital enhancements.

There are actually two main versions of APL2 issued by IBM - APL2/370 and APL2/PC. APL68000 Level II is midway between these two versions. A full list of the features that we have included is beyond the scope of this article but includes:

Nested Arrays; Mixed Arrays; Selective Specification; Axis Extensions

```
c > e (enlist)  e + (first)  = ~ (without)  [] ""
[]AF []AT []CS []EM []EC []ES []EA []ET []FC []PR []TC []TF
)IN )NMS )OUT )RESET )TIME
```

At the same time APL.68000 Level II incorporates all the features of APL.68000. In fact APL.68000 Level II is upward compatible with APL.68000 and existing workspaces can be)*LOAD*ed into the new APL. For those of you who are not familiar with APL.68000's features, this means that we have added □*EDIT*, □*HC* (hard copy), component files, □*SS*, □*BOX* and many other useful features to the APL2 specification.

The inclusion of the)*IN* command means that APL code can be transferred to APL.68000 Level II from other APL2 environments with the minimum of fuss. This last point is critical as APL.68000 Level II adds a number of key machines to the family of those computers that run APL2. Users of mainframe or PC-based APL2 systems can now move applications painlessly to the Apple Mac, or any one of a number of Unix computers or even their Atari ST to carry on working in APL2 at home! APL.68000 Level II has been designed by Richard Nabavi and is available for all machines that support APL.68000.

APL.68000 Level II will be officially launched at
APL90 in Copenhagen.

For more details, contact David Eastwood at MicroAPL

THE EDUCATION VECTOR

April 1990

Editor Alan Sykes

This Education Vector has been reprinted from VECTOR Vol.7 No.1. VECTOR is the Quarterly Journal of the British APL Association; for more information about the British APL Association, contact the Secretary: Anthony Camacho, 2 Blenheim Rd, St Albans, Herts AL1 4NR.

Contents

Editorial	Alan Sykes	18
Communications		19
Function Sampling With IAPL	Walter G Spunde	20
Some More String Manipulations	Alan Sykes	27
APL Booklist	Ed Shaw	32

Editor:
Dr Alan Sykes,
c/o European Business Management School
Swansea University
Singleton Park, Swansea SA2 8PP
Wales, UK

Editorial

Welcome to another edition of Education Vector. And thank you to all readers who took the trouble to reply to my questionnaire. To date, I have had 41 replies. Of these 41, 16 wished to discontinue receiving 'EV', 4 were members of the British APL Association who did not want to receive EV separately, and 19 wished to continue.

Of the 19 who wished to continue, 6 wished to become full members of the British APL Association, and 13 were prepared to pay £2 towards the cost of producing EV.

25 replies assessed the quality of EV as follows: 11 'good', 13 'helpful' and 1 'unhelpful'. Here are some specific comments that were made.

1. Please keep producing EV as it fills a gap - when you have teenagers interested in the language, it is not always easy to find material to maintain their interest.
2. Fine from my sideline interest - I don't use APL as a rule but the journal does allow me to view the progress of APL in Education - fine.
3. It gives some clever ideas, but more straightforward applications (rather than clever ones) would be appreciated.
4. Haven't got the time. (See the correspondence section for an interesting discussion of this theme.)
5. Written in a nice friendly style.
6. Wide variety of subjects required e.g. graphics, data entry forms and validation, discussion on coding/naming standards, how to integrate APL, structured programming, access to non-APL systems (whew! you need VECTOR as well!)
7. It's nice to see APL written in a way that is understandable by everyone (especially novices) rather than having code that takes 3 days to decipher!
8. Present form is good and helpful. I would like to suggest a full application related to teaching. Suppose you wanted to teach a subject (e.g. analysis of variance, laws of optics, genetics ...) what are the successive steps to follow from the idea to the package that the students would use in the classroom?

It is too early to say yet what the British APL Association will decide to do with EV in the light of the results of the questionnaire. (Incidentally, if you feel that the results above are unrepresentative and you haven't sent your reply, it is not

too late to do so.) However I am confident that it will do all it can to support EV in its endeavours.

Clearly you have given me much food for thought, and I hope that in my last year as editor of Education Vector, some of your many valuable suggestions may be implemented. Please help me to achieve this by sending your contributions for circulation to the many people, at home and abroad who do receive, read, and apparently appreciate reading Education Vector. In this context, please note a slight change in the editorial address.

Communications

Looking through my file of correspondence, I realise that there are many letters and notes from people in different countries that I have not acknowledged. So here is some attempt to remedy this.

From Nottingham, England

E.J.Sargeant replied to the questionnaire in the last issue, and commented on the problem of retrieving ASCII files into APL using IAPL. I must confess to not having time to investigate this, so may I ask readers who have used IAPL to read an ASCII file please to respond with a note to Education Vector discussing relevant details?

From Hereford, England

Graham Bennett also wrote a reply to the questionnaire. Graham is a Maths and Computing teacher in a comprehensive school. He writes .. 'I have been getting Education Vector for about two years but still have not bought an interpreter and am not using APL in the classroom. It may help you to know why.

In our school, and in others around, Computer Studies has recently been dropped both at A-level and GCSE because it is not perceived to be a very useful course for most students and it is felt better to devote time and resources on IT for all. This means that no one now studies programming in our school where previously we taught BASIC and Pascal. There is an emphasis on applications instead.

In Maths we are so busy struggling with the National Curriculum that no one has the time to look at how APL might be of use to us. I do not know what you can do to make APL more known and used. The pressure all teachers are

under makes it very hard for any but the most enthusiastic and dedicated to devote precious time to an unknown quantity like APL however good it may be.'

From La Habra, California

Zdenek Jizba sent a letter to Anthony Camacho describing in part the way IAPL is used in the fifth grade (eleven years of age) to stimulate discovery and exploration. I would like to extend an invitation to Zdenek (and anybody else) to present a typical (or atypical!) exploration in Education Vector:- volunteers please.

From Dunoon, Scotland

Mr Bill McLean may be just such a person to send me an account of using APL in investigative working? Bill tells me of his work in Physics, optical illusions, graphs, and playing Noughts and Crosses. I particularly liked his notes on how to produce Morse Code on a BBC 128 machine using IAPL. The first step is to produce a sound table for a dot, dash, space and longer space.

TABLE			
1	12	101	1
1	12	101	3
1	0	0	1
1	0	0	16

As a result *SOUND TABLE*[1 3 2 3 2 3;] would produce . - - in sound.

Morse code corresponding to input from the keyboard can be achieved using *SOUND TABLE*['.- '␣] if the user presses dot, dash or space.

To translate actual letters, the alphabet is required:

```
ALPH←' ABCDEFGHIJKLMNOPQRSTUVWXYZ'
```

together with a 27 by 9 character matrix called *CODE* whose first row for example contains ' . - '␣.

Then the function *MORSE* (written here in direct definition) will produce Morse Code corresponding to letters entered at the keyboard:

```
MORSE: SOUND TABLE['.- '␣CODE[ALPH␣;];]
```

From Sendai, Japan

Makato Kikkawa wrote to Anthony Camacho to report the successful porting of IAPL to NEC machines by Mr Tohru Kawahata. Makato also reports that the translation of 'An Encyclopedia of APL' and 'APL in Social Studies' is well under way.

From Kensington, New South Wales

Jim Douglas wrote to me concerning graph plotting using character graphics after reading an article I wrote in VECTOR on this subject. Jim is in the Department of Statistics at the University of New South Wales so we have lots of interests in common.

From Amherst, Massachusetts

Howard Peelle has sent two communications to me from the University of Massachusetts at Amherst where he lectures in the School of Education. First he points out, quite correctly, that the answer to the puzzle on numbers and bases (Education Vector July 1989) given in Education Vector January 1990 was incorrect. There, I stated that the only bases from 10 to 100 that gave rise to two digit numbers being reversed when converted to base 16 were 10 13 16 19 22 25 28 37 46 55 64 73 and 82. These are in fact the bases which give rise to reversals when numbers are compared to base 10 and not base 16. So the correct answer, given by Howard is 10 11 13 16 21 26 31 46 61 76 and 91. Here is his solution:

```

      ▽ B+L IF C
[1]  B+C/L
      ▽

```

```

      ▽ N+SEEK B
[1]  N+B IF ∨/∧f((B,[.5]B)τN)=0((2,pB)ρ16)τN+ι99
      ▽

```

```

      SEEK 9+ι91
10 11 13 16 21 26 31 46 61 76 91

```

(N.B. this one-line solution gave me *WS FULL* error when I tried it with $B+9+ι91$ - so you may have to use it repeatedly to get Howard's result in toto.)

Howard also supplied a solution to the card collecting problem which he claims to be faster than Claude Henrion's. It is:

```

      ▽ R←N CARDS C
[1]  R←1++/▽/∧\ (1C) • .#? (N, 11×C) ρ C
      ▽

```

Readers may also be interested in Howard's 'awfully inefficient' recursive function which generates a probability tree in order to approximate the expected number of cards required:

```

      ▽ R←A CC W
[1]  R←W[1]
[2]  +0 IF (= /A) ▽ = /W
[3]  R←((÷A) × A CC W+1 0) + (1-÷/A) × (A+1 0) CC W +1 0
      ▽

```

but of course, the correct expected value for N cards is $N \times + / \div 1 N$.

From New York

Paul Mansour also wrote to me concerning the card-collecting problem. However, his solution makes use of the extra facilities that second-generation APL interpreters have to offer. If X is a vector and M is a matrix, then the inner product using $\lceil$ and $\lceil$, i.e. $M \lceil . \lceil X$ returns a vector of the first occurrence of an element of X in each row of M . Hence it is possible to simulate the card-collecting problem as follows:

```

      ▽ R←N CARDS C;M
[1]  M←? (N, 11×C) ρ C ◊ R←M[. 1 1 C
      ▽

```

As you can see, this has been quite a round-the world trip. Please continue to send comments and correspondence particularly including code that you feel other readers might appreciate.

Function Sampling With IAPL

by Walter G Spunde

(Darling Downs Institute of Advanced Education, Queensland, Australia)

In engineering analysis, the shape of a function representing a continuous signal is deduced from values of the function recorded at a finite number of points. Fourier analysis and other advanced techniques may be involved to infer the general shape from a recorded sample. When students plot points in order to draw a graph, they are deducing, from *continuity* alone, the general shape of the functional relationship from a finite sample. This very simple exercise, apart from focussing on the concept of a function, can provide a contrast with the insight into a function's behaviour obtained more quickly by the techniques of calculus, and it can also be a quite interesting exploration.

Usually the tedium of extensive calculation, however, exhausts any interest a student may have had in an arbitrarily given function long before the job of experimentation is complete. With the computing power now available to most high school students, this should no longer be the case. What is required is not (only) a graphing routine, where the emphasis is on producing a finished product but rather, a sampling routine, where the emphasis is on developing an adequate sample for the finished product. The ease with which this can be done by computer, may change our perceptions of the role of calculus in curve plotting.

The workspace *TABLES*, written in IAPL, was developed to meet the need for function sampling experience for engineering students at UCSQ. Students are introduced to IAPL in the first semester of their first year, and while they require very little training in APL to use it successfully in the linear algebra half of their course, it was thought desirable that they should have preprepared functions to handle sampling and plotting. The workspace *GRAPHS* draws full-screen graphs of the tabulated functions on a labelled grid. While it produces the final product, learning by observation takes place in executing *TABLES*.

A necessary pre-requisite to using *TABLES* is adequate exercise with function definition, and the evaluation of functions at a selection of points, so that students may fully understand what the computer is calculating. This program makes no allowances for singularities of functions and the graphing routine will join points on either side of a singularity. If points close enough to a singularity are chosen, then a vertical line will result at such a point. The possibility of *DOMAIN ERROR* cannot be avoided in IAPL, and actually it is not always

desirable to shield students from encountering such an error - they may prove to be quite instructive. Students who have been frustrated in plotting points for $\log(x \tan x)$ - where they will have no trouble finding points on the domain $(-1,1)$ or even $(-1.5,1.5)$ but domain errors on $(-2,2)$, while nevertheless being able to calculate points in the domain $(3.5,4.5)$, etc. will never again forget about the modulus sign when calculating logarithms - a thing which they mostly prefer to forget!

This workspace prepares tables of values for functions defined explicitly, in APL notation. A sample table for $y = \cos x$ is shown.

Each table must be given a name, which should be unique.

Several tables may be compiled at the one sitting. A list, containing the names of the tables, the definitions of the functions and the domain of the tabulations will be created and must also be given a name.

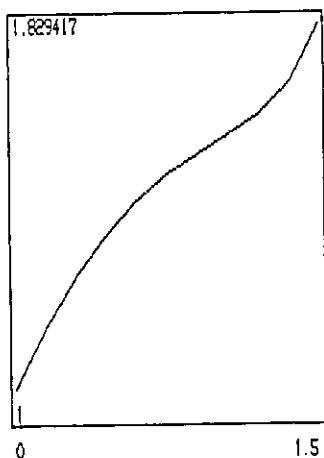
-3.20	-1.00
-2.40	-0.74
-1.60	-0.03
-0.80	0.70
0.00	1.00
0.80	0.70
1.60	-0.03
2.40	-0.74
3.20	-1.00

COSINE	20X	-3.2 X X 3.2
SINE	10X	0 X X 6
CUBIC	X POLY 1 1 2 3	-1 X X 1

TABLES compiles a table of sampled values and gives a plot of these points, together with maximum and minimum values attained, but without showing the position of the axes. Students may deduce the general shape and identify those regions in which more detailed sampling might be desirable, before copying the final tables over to *GRAPHS* for a 'nice' plot.

To illustrate the use of *TABLES*, two functions have been chosen. The first is the common $y = \tan x$. The steepness of the function is not commonly noticed by students. Although it is well known that $\tan x$ becomes infinite at $\pi/2$, its value at 1.5 is only 14.1 and the run into very large numbers (over 1000) only occurs between 1.57 and $\pi/2 = 1.57079..$ Relative to the large increases in these small intervals around the singularities, the variation of the function between -1.5 and 1.5 is negligible.

The second function is drawn from a standard example used to illustrate the *variation in parameters* technique for solving differential equations.



A particular solution of the equation

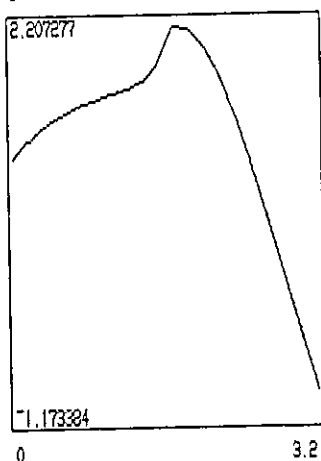
$$y'' + y = \tan x$$

is obtained, using this method, as

$$y_p = \cos x \log |\sec x + \tan x|$$

Many students have no doubt found such a solution, but few, it seems, have any idea of the graph of this function. Most would believe that because both $\sec x$ and $\tan x$ become infinite at regular intervals, the logarithm of such a function (even multiplied by $\cos x$) will be badly behaved. If they bother to apply L'Hospital's Rule, they could disprove this, but they would probably not be stimulated to do so. A plot of the function running across the suspected singularities reveals that in fact no irregular behaviour occurs.

While the function is not even defined at the points $\pi/2$ etc., it is nevertheless quite smooth and only a slight distortion of $y = \cos x$.



For a slightly more interesting curve, the particular solution of

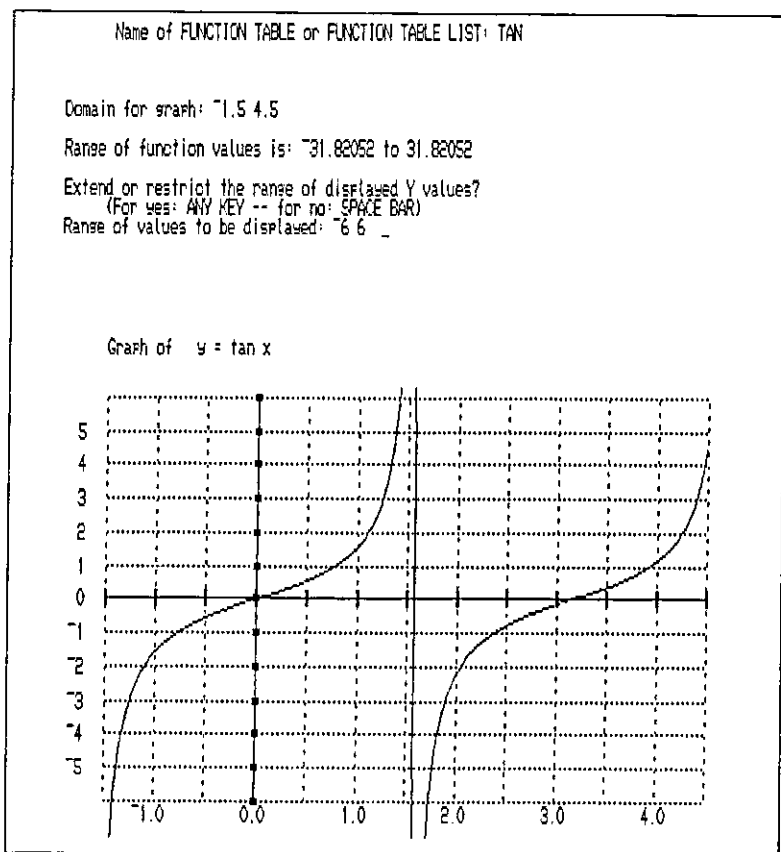
$$\begin{aligned} y'' + y &= \tan x \\ \text{with } y(0) &= y'(0) = 1 \\ \text{namely,} \\ y &= 2 \sin x + \cos x (1 - \log |\sec x + \tan x|) \end{aligned}$$

has been tabulated and plotted.

Some Comments on IAPL Development

Whilst the functions in these workspaces check some of the inputs, too much checking slows the execution on an XT to an unacceptable speed. As it is, the speed of the response is low enough to risk losing a student's interest in the proceedings. For the same reason, more intricate menu programs had to be

discarded. Space constraints dictated the tasks of tabulation and graphing be separated into two workspaces and that the functions remain uncommented.



File handling functions were developed to store and retrieve functions and tables in native files, but took up too much space and slowed execution too much to make it worth while keeping all operations in one workspace.

It was most annoying, incidentally, to have to reserve 2000 bytes, or 10% of the space we were allowing ourselves for functions and (text) variables, just to clear the screen. (Editorial comment - how about using `AV[25014]` or even `25 10 ' ' ?`) The two-workspace structure means that students have to `)SAVE` and `)COPY` table lists which is unfortunately cumbersome.

Some More String Manipulations

by Alan Sykes

Introduction

Replies to my questionnaire seemed to suggest that readers of Education Vector have found its contents interesting and helpful. One of the nice things about being its editor, is that I benefit from readers' attempts to broaden my experience of APL. This short note comes as a response to one of Bill McLean's notes to me, where he draws attention to a program published in 'APL IS EASY' by Jerry R Turner. It produces a calendar for a specified month and year and is used to illustrate the typical compactness of APL programs. I thought other readers might be interested in this program, particularly as Bill had problems converting it to run in IAPL because of a sophisticated 'format' statement used.

Recently I have made great use of a program I wrote to annotate columns of a matrix. So I thought that I would continue my theme of string manipulations for a second issue. First then the calendar.

A Calendar Program

Here is a typical calendar for August 1990 as you might find in the front of a diary, or on the current calendar in your kitchen.

AUG 1990						
SUN	MON	TUES	WED	THUR	FRI	SAT
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

The APL approach to producing such objects is generally to regard them not as consecutive lines of printing but as a matrix of characters - this one happens to be 9 by 42. What are the steps involved in producing such a calendar?

First we have to calculate two numbers - one telling us that the 1st of August 1990 occurs on a Wednesday, and the other that there are 31 days in the month required. So we require two functions, call them *DAYINWEEK* and

DAYSINMONTH. The second one is the easier one to do since February and leap years is the only complication we have to cope with. Here is the function, with right argument being the year and left argument the month (1 to 12):

```

V R+M DAYSINMONTH Y
[1] R+31-(M€ 4 6 9 11)+(M=2)*3--/0= 4 100 400 4000 |Y
V

```

DAYSINWEEK is more complicated, and quite honestly I confess to not even trying to understand the code as presented by Jerry Turner:

```

V Q+M DAYOFWEEK Y;T
[1] M+M,1,Y
[2] Q+100|(M[1 3]+ 1 0 +ϕ 0 12 τ-3+ 0 12 1M[3 1])[2]
[3] Q+7|M[2]+Q+(⌊-0.2+2.6×M[1])+(⌊Q÷4)-⌈1.75×M[3]+100
V

```

Note by the way, the value returned is the day of the week starting from Monday, so 8 *DAYOFWEEK* 1990 gives (what I shall call my *OFFSET*) 3, telling me that I must allow for 3 blanks at the beginning of the first week in August.

Having successfully pirated the nasty bits (thanks Jerry) now let's concentrate on the formatting. The format operator ∇ converts numbers into characters. It can be used with a two digit prefix that specifies the total size (in characters) of the field associated with each number and the number of decimal places. Hence for example:

```

6 3∇1 2 3
1.000 2.000 3.000

```

and

```

6 0∇1 2 3
-----1-----2-----3    (where the dashes indicate blank chars)

```

Since we can have up to six different weeks represented in a month, and assuming that day titles have block length 6, we need to create a 6 by 42 array of characters consisting of blanks and then the numbers 1 to 31, plus more blanks.

This can be done using the statement:

```
CAL←6 42 p252+((6×OFFSET)ρ' '),6 0 ▽131
```

The rest is now relatively easy - we can append a string of day titles by using the expression

```
TITLES,[1]CAL
```

as long as *TITLES* is also of width 42. Then we have to append the appropriate month and year onto the top of this. The resulting program will be something like the following, where I have used last issue's function *BOX* to provide a matrix of (abbreviated here) month names.

```
▽ CAL←MONTH CALENDAR YEAR;DAYS;MONTHS;NDAYS;OFFSET
[1] DAYS←' SUN MON TUES WED THUR FRI SAT'
[2] MONTHS←BOX' JAN/FEB/MAR/APR/MAY/JUN/JUL/AUG/SEP/OCT/NOV/DEC/'
[3] OFFSET←MONTH DAYOFWEEK YEAR
[4] NDAYS←MONTH DAYSINMONTH YEAR
[5] CAL← 6 42 p252+((6×OFFSET)ρ' '), 6 0 ▽1NDAYS
[6] CAL←' ',[1]DAYS,[1]CAL
[7] CAL←(42+(15ρ' '),MONTHS[MONTH;],' ',▽YEAR),[1]CAL
▽
```

Labelling a Matrix

Producing tables with rows and columns nicely labelled is not easy. It was one of the first things I tried to do in a demonstration in my early days with APL and failed abysmally. The function *HEADER* is quite useful in this respect. It builds on some of the code in *BOX* to produce a matrix of specified names (underlined) of desired width with the extra facility of being able to left- or right-justify the titles. (As you probably noticed, the format operator right-justifies numbers whereas columns of text such as names will be left-justified, so it is useful to be able to do both.)

Here is the function.

```

      V R+N HEADER ST;I;J;K;L;U;LL;LNEG;T;TT
[1]  A Header with left or right just'n according to sign of N
[2]  L+(I+ST=-1+ST)/K+1pST A pos of separators
[3]  J+1+L-0,-1+L A size of strings
[4]  ST[I/K]+' ' o U+ST o U[(~I)/K]+'-'
[5]  LNEG+(N<0)/1pN o LL+L o LL[LNEG]+1+(0,L)[LNEG]
[6]  L+(N<0)/L o LL+(N<0)/LL A L is pos to move to, LL to move
[7]  T+(pST)p1 o T[LL]+0
[8]  TT+v\~T o K[T/K]+K[T/K]-T/TT o K[LL]+L
[9]  AK now puts blanks at start or end according to r-j or l-j
[10] ST+ST[K] o U+U[K] o I+1+I[K]
[11] I[(I=2)/1pI]+(N)-J
[12] R+(I/ST),[0.5]I/U
      V

```

Let's see what it does ...

```

      15 20 10 HEADER 'BANANAS/APPLES/PEARS/'

BANANAS      APPLES      PEARS
-----

      15 ~20 10 HEADER 'BANANAS/APPLES/PEARS/'

BANANAS      APPLES      PEARS
-----

```

Now let's use it. Suppose we have a 3 by 3 matrix of marks M , corresponding in rows to three pupils, and in columns to three subjects say MATHS, ENGLISH and PHYSICS.

```

      M
14 76 46
54 22 5
68 68 94

```

If we wish to label the columns, all we need do is decide on the width (which should be big enough to cope with the label size - say 10), format the columns of M , create the header, and glue together.


```
(-10 -10 -10 HEADER 'MATHS/ENGLISH/PHYSICS/'),10 0M
```

MATHS	ENGLISH	PHYSICS
-----	-----	-----
14	76	46
54	22	5
68	68	94

(If you require different widths for the columns, then your format statement must have a pair of numbers for each column, e.g. 10 3 15 2 20 0M.)

So far so good. What about pupils names. These can be created using *BOX*:

```
NAMES+BOX 'JONES A M/BLOGGS J/KEATON B/'
```

and to accommodate the names we can have a new title:

```
TITLE+20 -10 -10 -10 HEADER 'NAMES/MATHS/ENGLISH/PHYSICS/'
```

where we have allowed 20 spaces for the names. But *NAMES* is only of width 9. So to glue *NAMES* to the marks to make a matrix of width 50 we must expand *NAMES*, which we can do using 3 20+*NAMES*.

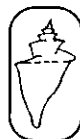
Hence finally, we get the full table thus:

```
TITLE,[1](3 20+NAMES) 10 0M
```

PUPILS	MATHS	ENGLISH	PHYSICS
-----	-----	-----	-----
JONES A M	14	76	46
BLOGGS J	54	22	5
KEATON B	68	68	94



Renaissance Data Systems
Enlightenment Thru Information Processing
ALL APL BOOKS IN PRINT
Catalog Excerpts June 1, 1989



If we do not carry it (in English, at least), we dare you to find it elsewhere!
Specializing in books on:
APL AS A TOOL OF THOUGHT!

ACCOUNTING STRUCTURED IN APL , Iliri, 1984, 491p.	\$10.00
The basic principles of accounting with numerous APL expressions to model them.	
ABSTRACT ALGEBRA - A COMPUTATIONAL APPROACH , Simms, 1984, 491p.	\$55.95
An introductory course in the basic concepts of algebra and fundamental algebraic algorithms. A methodology for investigating computational questions. Includes lemmas, theorems, and corollaries.	
COMPUTING IN STATISTICAL SCIENCE THRU APL , Anscombe, 1981, 426p.	\$39.00
Lots of statistics and APL. His workspaces are available from Yale University. A classic.	
MATHEMATICAL EXPERIMENTS ON THE COMPUTER , Grenander, 1982, 525p.	\$59.50
Case studies using APL in a number of fields, including statistics, linear algebra, geometry, asymptotics, neural networks, invariant curves. Detailed description and analysis of APL functions in all topics discussed.	
PROBABILITY IN APL , Alvord, 1984, 142p.	\$19.50
A delightful, yet serious development of 31 functions for fun and learning with roll, deal, binomial distributions, combinations, permutations, geometric distributions, and the World Series. Friendly examples. No previous APL required.	
APL - THE LANGUAGE AND ITS ACTUARIAL APPLICATIONS , de Kerf, Goovaerts, Stiers, 1987, 223p.	\$89.00
Introduction to APL. Loss reserves, credibility, probability, numerical analysis, forecasting, with APL functions.	
COMPUTER GRAPHICS FOR DESIGNERS AND ARTISTS , Kerlow and Rosebush, 1986, 298p.	\$44.95
A well organized, beautifully done reference book. Step-by-step explanations of terms and techniques from bits and bytes to 3-dimensional imaging. Many explanatory illustrations, most in color. No APL, but, says Rosebush, APL thinking and data structures are implicit throughout.	
APL2 AT A GLANCE , Brown, Pakin, Polivka, 1988, 444 p.	\$29.00
Solid, unimpeachable, introduction to APL2. Clear illustrations, modern exercises in each chapter. Gets you started.	
THE APL IDIOM LIST , Perlis and Rugaber, 1977, 50p.	\$ 6.50
Funded by Mobil. Written at Yale. Very terse, but includes nuggets of gold.	
APL - ADVANCED TECHNIQUES AND UTILITIES , Berquist, 1986, 450p.	\$44.95
Good discussion of alternative approaches to a wide range of programming tasks - efficiency, searching, dates, workspace documentation, file design, boolean techniques, financial utilities. Many idioms.	
Diskette for above	\$15.00
Contains all functions described in text. Formatted for APL*PLUS/PC, uploadable to Sharp APL VS/APL APL2.	
APL AS A TOOL OF THOUGHT, PROFESSIONAL DEVELOPMENT SEMINARS, NY/SIGAPL	
They cover a wide range of topics in business and education. 1987 - logic, insurance, statistics, A. I., accounting, fractals, linked lists. Other years: Teachers toolbox, computer science, biology, graphics, engineering, data bases, and more.	
1984, 1985, 1986, 1987, 120 to 150 pages each (1983, 47p. \$10.00)	each \$15.00
All five as a package.	\$55.00

Name: _____	Date: _____	Please make check or money order payable in \$U. S. and send with this form to: Renaissance Data Systems P. O. Box 20023 Park West Finance Station New York, New York 10025-1510 U.S.A. (212)864-3078	
Street: _____			
City: _____	Prov./State: _____ Postal Code: _____		
Country: _____	Telephone Day: _____ Evening: _____		
Title _____	Price _____	Quantity _____	Total _____
Title _____	Price _____	Quantity _____	Total _____
Title _____	Price _____	Quantity _____	Total _____
Subtotal: _____			
Postage and handling: _____			
- U.S. and Canada (\$2.50 + \$2.50 each item over \$25.00): _____			
- International (surface mail add 15%, airmail add 50%): _____			
NYC Residents please add 8 1/4% sales tax, NY State, 4%: _____			
Please send complete CATALOG of 64 Titles: ☐ _____			
Total: _____			_____

APL91: Conference Announcement

We are glad to bring readers an early and highly provisional draft of the plans for APL 91. Vector will circulate news of the plans as they develop, and will later include the call for papers and details of how you get on to the mailing list.

APL 91

THE NEXT 25 YEARS

[Provisional Theme]

The International Conference on APL 4 - 8 August 1991
Stanford University, California, USA

Call for Participation

Sponsored by the Associating for Computing Machinery, Special Interest Group on APL ACM SIGAPL, in cooperation with Stanford University Computer Science Department. Organized by The APL Bay Area Users' Group

Conference Overview

The APL91 conference will take place 4 - 8 August 1991, on the campus of Stanford University, in Palo Alto, California. University accommodation will be available as primary housing with local hotels for those who prefer them. University dining facilities will be available as well, so that subsistence expenses will be quite low. We hope that the low cost, location, and time of year will make it attractive and induce strong international attendance.

The main conference will run from 5 - 8 August, with paper presentations, general sessions, poster sessions, exhibitors, vendor forums, and social events for informal exchange with other attendees. Two tutorial tracks are planned for Sunday, 4 August.

Tutorials

The first tutorial will be aimed at APL experts, addressing the use of APL in one or more of these areas:

Artificial Intelligence/Expert Systems:

- APL has been used in a variety of research and production applications such as medical diagnosis and factory automation.

Object Oriented Programming:

- Recent papers have shown various ways that APL can be used for object oriented programming style.

Case:

- APL can be considered a tool for Computer Aided Software Engineering with its proven capabilities and uses in generating code for ADA, assembler, FORTRAN, and C.

The second tutorial is being planned to introduce APL to a new generation of students, through informal sessions. Attendees at this tutorial would receive a package of free APLs to take away and continue learning and using. In addition to the regular conference registrants, students and teachers from local schools and colleges will be invited to participate.

Papers

50 papers will be presented in two or three parallel tracks depending on topic, 45 minutes each.

Posters

Posters will have a special session set aside for them and will address short topics. We expect to include work in progress and informal presentations which may later be prepared for publication.

General Sessions

Opening and closing sessions will feature prominent speakers in fields of interest to APL users, but not necessarily in APL.

Vendor Forums

Vendors will be invited to present products and/or participate on panels.

Exhibitors

Exhibitors will display products and services in an exhibit area open during the entire conference and featured during an exhibit reception one evening.

Discussions

Rooms will be available for ad hoc scheduling on birds-of-a-feather topics.

Events (Provisional):

- | | |
|-------------------------------|---|
| Welcome Reception: | Sunday evening. |
| Exhibit Reception: | Monday evening. |
| Exploratorium Evening: | Tuesday evening. Bus to San Francisco for a buffet dinner at a science museum where the exhibits are intended to be used. |
| Closing Picnic: | Thursday lunch. To be held outdoors on campus. |

Program Statement

Important early developments in implementing Kenneth Iverson's mathematical notation as an executable computer language occurred at Stanford in the 60's: the first full working version of APL resulted from collaboration of Stanford graduate students with IBM. APL has now come into wide use in academic, scientific, and commercial endeavours around the world. 1991 will be the 25th anniversary of the availability of an APL system, marked by the date IBM's 1 CLEANSAP was saved. 1991 will also be the 10th anniversary of the annual international APL conference last held in the San Francisco Bay Area. Now that free APLs make widespread growth of its educational use it seems appropriate that we meet at Stanford to consider the next generation of APL and APL users.

APL development has diverged and converged through many paths over the last 25 years, achieving an ISO standard and enhancement to second generation APLs, and now working on an ISO standard for extended APL. APL has pioneered in many areas, including interactive timesharing; functions acting on whole arrays; dynamic data arrays; array theory; use of vector, array, and multi-processor hardware; cooperative processing; spreadsheet like applications; independence of national languages; free distribution of interpreters; use for AI and expert systems; use by non-programmers and use in education from grade

school to university. Computing technology is currently facing several large issues, such as proprietary versus open systems, hardware interchangeability versus new hardware architectures, programming versus CASE, and PCs versus workstations versus mainframes. The recent availability of free APLs for PCs and school computers has made APL freshly available to a wide audience in education. Thus, it seems an appropriate time to consider the role or directions for APL in "The Next 25 Years," the theme for this conference.

APL is subject to an interesting variety of paradoxes:

- APL is often considered an isolated technology. Yet software developments continue to credit ideas from APL, rightly for example, work on the new standard for vectorized FORTRAN or Mathematica.
- It has been said that APL cannot be taught; it can only be learned. It is not often taught as such in University. Yet first courses in computer science taught to undergraduates at Yale University with APL had students writing simulators and assemblers by the end of a term. And APL is being used in grade school in parts of Europe to teach mathematics.
- APL's symbolic basis is widely criticized for requiring special handling. Yet this makes it largely independent of national languages (APL needs no translation). And graphical user interfaces based on icons for mnemonic association are becoming the norm on workstations.
- Some people think the symbols inhibit portability. Yet the strict syntax of APL easily reveals implementation-specific features, and APL is the only language that can run on PCs, mainframes, and vector-hardware supercomputers without program changes.
- APL is a powerful tool for non-programmer computer users, such as engineers, statisticians, and actuaries. Yet it still bothers professional programmers to subordinate detail in the expression of algorithms. It has been called too general a tool for production commercial applications. Yet computer science students now graduate thinking that C is appropriate for business applications of all kinds.
- APL has sometimes been thought to use too many computing resources in return for its conceptual advantages. Yet in other advanced computing technologies such as artificial intelligence, this is expected. And the result of the conceptual advantages is often better algorithms for much larger performance gains.
- APL permits unstructured programming. Yet its natural programming style is functional and it can utilize structure inherent in the data.

- APL has proven productivity advantages over other languages, yet it is not uncommon for DP managers to discourage its use. Do we know why?

We hope to receive papers addressing these paradoxes and issues, concentrating on real experiences or work in progress toward new uses. These experiences, together with conference discussions among attendees, general sessions of speakers and panels, and participation of exhibitors, will focus on where and how APL should best be used in the next twenty five years. Is APL a notation, a language, an environment, a system, or a paradigm? Where should vendors and users take it among other technologies? In which areas is APL successful, and in which areas is it not? Now that it can be accessible to schools with limited computing resources, what should the next generation of students be encouraged to think about it?

Conference Committee:

Co-chairman, Administration/Finance: Charles Schulz
Co-chairman, Program: Tibor Horvath
Secretary, Communications: Curtis Jones
Exhibits, Events: Nancy Wheeler
Press, Publicity: George Mendonsa

Program committee: to be formed

APL Product Guide

Compiled by Alison Chatterton

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry - see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Timesharing Services
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * APL Training Courses
- * Other services
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to Alison Chatterton, at the address on the inside back cover.

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	IBM PCs & compatibles	poa	includes PC, mono/colour monitor, APL interpreter, operating system software, plus optional printers, graphics boards, additional memory etc.
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
MicroAPL	Aurora	20,000+	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
	Spectrum	7,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
M.T.I.C.			Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
	386sx AT	2600+	Entry level 32-bit compact AT with DIALOG APL. 2MB ram, VGA, 40MB hard disk, 1.44MB floppy disk, 102-key keyboard, monitor, mouse, MS-DOS 4.01, WORKS and WINDOWS-386. Many expansion options available.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES(£)	DETAILS
I.P. Sharp	SHARP APL	poa	International Network application systems and public databases.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	APL*Plus/PC Release 9	535	STSC's APL for IBM PC, PC/AT and PS2. Upgrades from earlier Releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1600	Incorporates mainframe features & performance in a version of APL for the PC
	Run-time	poa	
	Dyalog APL	1000-10,000	2nd generation APL for Unix systems
	APL2/PC	376	IBM's APL 2 for the PC.

Cocking/Drury	APL*PLUS/PC Rel 9	535	STSC's full featured APL for IBM 's and compatibles - Version 9 offers the potential for larger workspaces using extended/expanded memory; full screen editing of numeric arrays; user command processor; point & edit etc. Upgrades to version 9 are available from Version 8 and earlier releases.
	APL*PLUS PC Run-Time	poa	Closed version of the interpreter for developers, prevents user exposure to APL.
	APL*PLUS II System	1600	High powered APL interpreter for the 80386 chip. Price includes one years maintenance and free upgrades - volume discounts VERSION 3 DUE JUNE 1990.
	APL*PLUS II Developer System	poa	All the features of the APL*PLUS II System plus unlimited free run-times to enable developers to distribute their applications.
	APL*PLUS UNX	poa	STSC's 2nd generation APL for all major Unix computers and workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers running under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dyadic	Dyalog APL	for DOS/386	995 Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and bSM Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL	for Unix Systems	995-12,000 Second generation APL for Unix systems. Available for Aitos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
I-APL Ltd	I-APL/PC or RML Nimbus	4.50	ISO conforming interpreter. Supplied only with manual. (see 'Other Products' for accompanying books)
	I-APL/BBC	4.50	As above
	I-APL/Archimedes	4.50	As above
IBM UK	IBM PC APL2	348	APL2 for the IBM PC. Program 5799-PGG/1, PRPQ number RJ-0411. From all IBM dealers.
MicroAPL	APL68000	2,000	Full implementation with component files, error trapping etc. for SPECTRUM, HP300, SUN, NCR, NIXDORF, BULLEtc.
	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features.
	APL68000 Apple Macintosh	87	
	APL68000 Apple Mac II	280	
	APL68000 Amiga	87	Full APL interpreters with support for windows, mouse, graphics etc.
	APL68000 for Atari ST	87	
	APL*PLUS/PC - REL 9	450	
	APL*PLUS II V2.0	1,395	
	APL*PLUS/PC	480	STSC APL
I.P. Sharp	Upgrade V7 to V8	100	
	Upgrade any to V8	200	
	Sharp APL/PCX V1	1,650	For IBM XT/AT 370
	Sharp APL/PC V2	40	shareware for IBM PC/XT
	Sharp APL	poa	for IBM mainframes

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL People	IBM & compatibles	poa	IBM and 'budget' APL VDUs - monochrome/colour/graphics.
Dyadic	IBM 3151	599	Monochrome APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
	IBM 6154	1,228	Colour APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
General Software	Meilordata	poa	Second-hand Elite 3035A
Shandell	HDS3200/10 APL	965	15" screen, 8 page memory, windows. 80/132 columns, full overstrike. Multi-host multi-session support. ANSI X3.64, DEC VT100, VT220, Tektronix 4010/4014 1024 x 390 resolution.
	HDS3200/25 APL	1065	As above plus switchable 25 or 50 line screen. 75 Hz refresh. Resolution 1024 x 780 in graphics mode.
	HDS3200/35 APL	1265	As HDS3200/25 plus local pan & zoom.
	HDS3200/50 APL	1395	14" colour monitor. 75 Hz refresh. 8 or 16 colours from palette of 256. Display memory 96 lines of 80 or 132 columns. APL processing & keyboard with full overstrike. Windows, multi-host, multi-session support. ANSI X3.64, DEC VT220, VT100, VT52 emulation.

APL PRINTERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL People	Epson series	200	Inexpensive dot-matrix and NLQ printers
	Quen-data & Qume etc	500	Daisy-wheel printers
Dyadic	Various	poa	Range of APL printers available.
MicroAPL	Datasouth DS180+	1,195	See Datatrade entry
	Philips GP480	2,137	Matrix printer with letter & draft quality and APL (480 cps).
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL-385	APL-385	50(PC),125(mf)	Including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
APL IMPETUS	Impetus	poa	Hierarchical Planning System
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad- functions & nested arrays for VSAPL under VM/CMS & MVS/TSO

	COMPILER	poa	The First APL compiler
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Converts non-APL files to APL
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Information Centre spreadsheet incorporating data exchange between APL, FOCUS, IFPS, SAS, APL/DI, ADRII, Lotus123, Visicalc, Multiplan & DIF
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(for PC's)	APL*PLUS PC Tools	325	Utilities including: RAM disk, full screen data entry, menu input, report generation, exception handling and games.
	IRMA Module	120	327x IRMA support.
	Fin & Stat. Library	350	Financial & Statistical routines
	SPREADSHEET MGR	195	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII & Lotus
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	PC Utilities including: APLMAC (windows); Unlock (unlocks functions in .AWS); DTEX (text and spreadsheet imp/exp). Mostly available in English or French.
Interprocess	IEDIT	1900-3200	Full screen APL2 editor with immediate APL execution, and a full-screen debugger
(mainframe)	AFM	8200-9800	High performance component and keyed file system (VS APL and APL2)
	Format	1650	A QuadFMT data formatter for VS APL and APL2
	FSM124	1650	AP124 programming for APL applications without GDDM (APL2)
	PowerCode	1300	External functions for APL2
	CALL/AP	3000	for calling non-APL programs (VS APL and APL2)
	UCF	1800	Inter-user data transfer for VM users via IUVC
(PC)	AFM	115	Single user component and keyed files for APL2/PC
Mercia	STATGRAPHICS 4	684	Integrated statistics/graphics system for the PC. Now with macros. Bulk and educational discounts available.
	Upgrade 3 to 4	215	
	Upgrade pre 3 to 4	375	
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC and 385. Sales Forecasting, Inventory Control, Master Scheduling, Distribution Requirements, Planning etc
	TWIGS		A modular library of tools to teach and explore state-of-the-art materials management concepts.
	T	299	Time series forecasting
	W	99	Warehouse replenishment
	I	199	Inventory Management

	G	99	Grouping requirements into EOQ's
	S	99	Scheduling production/purchasing
		599	All 5 modules above
		4999	All 5 modules site licence
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self instruction for APL*PLUS/PC
	STATGRAPHICS Rel 4	590	
I.P. Sharp	GLOBAL LIMITS	poa	Exposure management for banks
	IPSA/CONNECT	poa	Mainframe to micro link
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	65	Full screen front end to IPSA mailbox
	upgrade to V.2	15	
	NEWSFLASH	poa	Real time message exchange
	VIEWPOINT	poa	4GL - Info centre product
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Consultancy	poa	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL6900, C/Unix, TSO/MVS, VM/CMS, graphics, Operational Research etc. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry - designing, programming and implementing applications.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-275 275-350 300-450 400-600 450-750	Junior consultant Consultant Senior consultant Principal Consultant Managing consultant
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Delphi	Consultancy	poa	APL system development on mainframes and micros.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
E & S	Consultancy	poa	System prototyping; all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
General Software	Consultancy	from 120	
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Ian A. Clark	Consultancy	poa	Computer-based Information Systems implementation where acceptance is critical. APL on PC and Macintosh. Human Factors of HCI; novice ease of use; online assistance; training courses; distance-learning materials.
Intelligent Programs Ltd	Consultancy	175-350	Systems development, enhancements, support.
	Documentation	160-250	Preparation of new manuals, rewriting of existing materials.
	Training	150-250	Training for APL experts through to non-technical system users.
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.C.	Consultancy	240-500	Business analysis and APL consultancy
Parallax Systems Inc	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	250	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.
Rex Swain	Consultancy	poa	Independent consultant, 15 years experience. Custom software development & training, PL and/or mainframe.
Wickiffe			

Computer Ltd	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems
--------------	-------------	-----	---

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	An APL Tutorial	2.50	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2nd Edn)	5.50	228pp by Helzer
	APL in Social Studies	2.50	36pp by Traberman
	I-APL Instruction Manual (2nd Edn)	2.50	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	14.50	185pp by Thomson
** Please add one pound packing charge per order **			
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.
ISI	Tangible Math	\$19	An APL Approach to Math - Shareware, includes base Sharp APL
	J	\$24	Dictionary APL simplified and enhanced - Shareware (Mac, PC)
	SharpAPL/PC	\$74	Registered Shareware and Reference Manual
	ISI APL	\$99	Improved APL/PC - enhancements, performance, large workspaces.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APL Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
APL Software	David Alis Jill Moss	The Old Malthouse, Clarence ST, BATH, BA1 5NS. Tel: 0225-462602
Buckland Management Systems	John Buckland	Westwood, 19 Grange Road, Camberley, Surrey, GU15 2DH Tel: 0276 684327
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 60130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN Tel: 081-767 4254
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 071436 9481 Fax: 01-436 0524
Datatrade Ltd.	Tony Checkseld	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289

Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Park House, The High Street, Alton, Hampshire, GU34 1EN. Tel: 0420-87024
E & S Associates	Frank Evans	19 Hornesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
General Software Ltd	M.E. Martin	22 Russell Road, Northolt, Middx, UB5 4QS. Tel: 081-864 9537
H.M.W. Computing Ltd.	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA Tel: 071-353 4212 Telex: 926604 HAMHSEG Fax: 01-353 3325
HRH Systems	Dick Holt	Box 4496, Silver Spring, Maryland 20904
Ian A. Clark		9 Hill End, Frosterley, Bp. Auckland, Co. Durham DL13 2SX Tel: 038852-7190
I-APL Ltd	Anthony Camacho	2 Blenheim Road, St. Albans, AL1 4NR. Phone 0727-60130 for queries, order forms, bulk orders
	J C Business Services	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU N.B. PAID ORDERS ONLY
IBM UK Ltd	Nat.Enq. Centre	414 Chiswick High Rd, London W4 5TF Tel: 081-747 0747
Interprocess Systems	Stella Chamberlain	9040 Roswell Road, Suite 690, Atlanta, Georgia 30350-1131 Tel: (404) 992-8400
Intelligent Programs Ltd	Alastair Charatan	9 Gun Wharf, 130 Wapping High St., London, E1 9NH
ISI	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: (416) 925-6096
	Orders	3512 Cameron Mills Road, Alexandria, Virginia, USA 22305-1103 Tel: (703) 548-1799
Mercia Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5096
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 071-922 8866
M.T.I.C.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: Sunbury(09327) 80648
Peter Cyriax Systems	Peter Cyriax	213 Goldhurst Terrace, London NW6 3ER Tel: 071-624 7013 (Answerphone) 0860-377963 (Mobile)
Parallax Systems Inc.	Kevin Weaver	Avery Road, Box 319, Garrison, NY 10524, U.S.A. Tel: 914-424-4265
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB. Tel: 0276-20789
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A.. Tel: (212)864-3078
The Rochester Group	Robert Pullman	50 S Union St., Rochester NY 14607, U.S.A. Tel:716-454-4360 or 716-454-4541
Shandell Systems Ltd.	Maurice Shanahan	Chilham House, High Street, Chalfont St. Giles, Bucks., HP8 4QH. Tel: 02407-2027. Fax:02407-3118
I.P.S.A. Ltd.	Mike Butler	7th Floor B Block, Coventry Point, Market Way, Coventry CV1 1EA Tel: 0203 256562
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Rex Swain		8 South Street, Washington, CT 06793, U.S.A. Tel:203-868-0131 or 212-242-5816
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel:0946-692588
Warwick Univ.	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel:0203-523369

Reviews Section

introduction by Adrian Smith

Both reviews in this VECTOR are related to the user-interface. Dyalog APL has been greatly enhanced by the inclusion of a 'modern' approach to the APL session. This at long last brings APL into line with the vast majority of PC language products (from Quick-C to Turbo Prolog) and will I hope set the tone for all the major PC APL systems in the future.

Powertools has long been an option for those who need to wring the last ounce of power from their PC APL systems. You can now use their CUA routines to develop APL systems that look just like Windows, OS/2, and anything else that supports the CUA/Graphics standards.

Finally, we have a summary of a recent publication 'APL with a Mathematical Accent', and (continuing the user interface theme) we have received a review copy of 'The Computer Display Designers Handbook' by Eric Wagner. This will be reviewed in Vector 7.2.



for 386 PCs under MS-DOS

Dyalog APL for 386 PCs under MS-DOS is a complete implementation of Dyadic's popular APL for Unix systems, AND fully exploits the power of a DOS 386-PC.

JUST LOOK AT THE FEATURES . . .

- *An exciting new Screen Manager in which the contents of the screen are 'shared' with a nested system variable □SM (Screen Map).
Facilities include:*
 - Pop-up windows by localisation of □SM
 - Scrolling (vertical, horizontal, linked)
 - GENUINE numeric fields
 - Date fields
 - NESTED forms
- Fully integrated window-based Session Manager, Editor and Debugging Environment.
- Compatible with the Draft ISO standard for APL
- Huge Workspaces (16Mb and MORE!)
- Virtual Memory Support (using on-chip demand paging)
- Full 32-bit processing speed
- *APL2 features including:*
 - Nested Arrays
 - Defined Operators
 - Selective Specification
- *Commercial features including:*
 - FMT
 - Component Files
 - Error Handling
 - "AP124-like" Screen Manager
- External Variables
- Interface to GSS*CGI for fast device-independent graphics
- C Interface

. . . THE BENEFITS SPEAK FOR THEMSELVES

Requirements: 386/486-based PC or PS/2, 640K DOS Memory plus 2Mb RAM, EGA or VGA (colour STRONGLY recommended), 80387 coprocessor STRONGLY recommended, DOS 3.3 or later.

*For further information contact:
Dyadic Systems Limited*

Park House, The High Street, Alton, Hampshire, GU34 1EN
Telephone: (0420) 87024 Fax: (0420) 89886 Telex: 85881

The Dyalog APL Development Environment for MS DOS

explored by Adrian Smith (Rowntrees)

Introduction

This is definitely not a review! When you come across something as novel and exciting as Dyadic's new APL Session Manager, your first reaction is to play with it, then you start to wonder why no-one thought of it before, then you go back to APL*PLUS/PC and are overcome with sudden depression. I feel about as competent to review this product as I would be to review (say) Windows 3; I just don't have enough experience of working with windowed development environments, so what follows must be taken with a very substantial pinch of salt. If you want a full technical description of how the Dyalog environment works, please refer back to John Scholes' article in Vector 6.4, page 101.

Why has No-one Done it Before?

This is the first real attempt to market an APL system for the 1980's. Surely this is absurd; I bought my £50 copy of Turbo Prolog at least three years ago, and it came with windows for editing, tracing, program output etc. These could be tiled or overlaid, resized with the cursor keys, zoomed to full screen, set to your favourite colours and so on. Turbo Prolog, like APL, is interpretive, and benefits more than most languages from this kind of environment. Unfortunately, Prolog has the disadvantage that it is almost impossible to program anything useful in it, so I'm afraid it has joined the ranks of unwanted software on my bookshelf upstairs, and no longer even merits space on my hard disk.

Meanwhile, we soldier on with APL*PLUS/PC and APL2/PC. The former is OK (as glass teletypes go); the latter is a poor imitation of the mainframe AP120 and I am glad to hear it is due for a total rewrite in Release-2. APL*PLUS/PC is dogged with some dreadful limitations, the main one being the lack of full-screen edit on suspended functions. Coupled with the fact that `EDIT` doesn't window properly, and the hassle of moving code between functions, and from functions to the APL session and back ... I am beginning to get more than a little frustrated. Of course APL*PLUS II does actually do most of these things, but by such a horrific tangle of key combinations that only the most dedicated hacker will ever use it to its full potential.

In summary, Dyalog have moved APL a long way in the right direction. I hope the rest of the APL world is running as fast as it can to catch up, and hopefully do it even better. The rest of these notes should be read with this in mind!

Why a Development Environment Matters

I cost Rowntrees about £300 a day to run; between one third and one half of that time is spent developing and testing APL code; with the APL*PLUS/PC environment (the plain old kind) I reckon I go about 30% faster than on the mainframe (and we have sub-second TSO response). Watching carefully what I do most of, and bearing in mind the strengths and weaknesses of APL*PLUS/PC, I am confident that Dyalog would give me at least another 20%. At a conservative estimate, that looks like £20 per day, or £100 per week, or £4,800 per annum. That is clearly the kind of money that makes accountants look up and take note.

To look at it from the other end: we have two people working nearly full time rolling out a complete rewrite of our company-wide medium-term planning system; this is scheduled to hit the various UK sites between now and early 1991. Of course we are under all the usual pressures (the spec keeps changing, Oracle keeps going down at the wrong moments, ...); if we could cut 30% out of the final debugging and testing time, we would be more confident of meeting that schedule, and more likely to deliver a product that worked!

There is one more angle worth considering. If I come home after a day's mainframe work, I am tired and ratty, and likely to beat up the children for no particular reason. If I come home after a day working on APL*PLUS/PC, I may aim the occasional kick at the dog, but am usually able to sit down of an evening and hack out an article like this. A day at work with Dyalog APL, and I might actually become a nicer person, able to play Scrabble with the family and willing to take the dog for a walk in the rain!

Good APL developers are hard to come by, and just as the best secretaries can demand the best word-processors, I reckon that we poor bloody infantry have earned the right to some decent boots that keep the water out, and don't chafe too much round the ankles. If you are an OR manager, and you want to get rid of someone - give them a TSO system to maintain. If you want to keep them - put them on PC work. If you really can't afford to lose them - get them a copy of Dyalog APL.

To summarise: the development environment matters because it saves you money; because it gets better systems delivered faster; because it helps to keep

your staff. The next section looks in more detail at some specific areas where time and money can be saved.

Where the Dyalog Environment Scores

I spend a lot of time digging around in old systems looking for the bits that need to be changed. This usually means going in at the `□LX` and working down the tree of functions until I find the ones that do the real work. Often these are 6 or 7 levels deep, and I need to track up and down the calling tree making changes en route. The recursive 'point and edit' mechanism is absolutely ideal for this; coupled with the excellent search/replace mechanism it can be a huge time saver.

Our standard routine for rescuing crashed workspaces is to ask the user to run a *CRASH* function which kills the `□LX` and does a `)WSID CRASH` and `)SAVE`. To be able to load this up, hit the `<trace>` key and immediately see a stack of windows with the suspension at the bottom ... then to be able to back up the stack to the point where safe resumption is possible ... to execute lines of the suspended code in any order ... all this can not only be a time-saver, it could come close to being a life-saver!

We try to move responsibility for systems round the department, which means a lot of time is spent 'picking up' someone else's code. I tend to do this by stopping it at strategic points and making sure I understand what the data looks like, and what the functions are doing to it. The APL*PLUS stop and trace mechanism is so cumbersome (and dangerous if you forget to `⊖ □STOP 'FOO'`) that I usually just stuff a line of `□□□□□□`s into the function I want to stop. Dyalog's trace windows completely eliminate the need for `□STOP`, `□TRACE`, `SΔ`, `TΔ`, and are a superb and much safer way of exploring code.

I hope that gives you a flavour of the places where the Dyalog environment scores over any of our current APL systems. It beats mainframe APL out of sight on both functionality and ease of use; APL*PLUS II comes close in functionality, but is hopelessly outclassed on ease of use; APL2/PC remains an unknown quantity at present, but is sure to be vastly improved when release 2 is announced.

Nitpicker's Corner

Here, in no particular order, are some of the things that annoyed me with Dyalog's environment:

- The installation is wonderfully simple, but leaves off half way. It really ought to set up the UK keyboard driver, and either load the APL character set or create a suitable batch file for you. The average hacker will ignore the manual, and simply go into the dyalog directory and type APL. He will be faced with a plain blue screen, and will find that whatever he types, said screen remains obstinately blank! The more enterprising hacker will try) OFF, but will fail because the) is actually on shift-2 due to APL thinking that we are on a US keyboard!! I hereby nominate for the BAA Outstanding Achievement Award the first man to get out of this situation without a reboot.
- The APL character set is not nice. The ı looks as if it has had a bad night on the tiles, and the ρ is barely recognisable. When you are majoring on visual appeal and programmer productivity, the readability of the APL symbols matters. OK, you can fix it, but you shouldn't have to.
- The default colour scheme is awful. Even John Scholes was heard to apologise for it at the recent Vendor Forum. It is so easy to fix this ... e.g.

V249: 255 11	+ marked line	Cyan
V250: 255 2	+ session	Green
V251: 255 46	+ Status Line	Yellow on Green
V252: 255 63	+ Suspended Function	Yellow on Cyan
V253: 255 62	+ Pendent Function	White on Cyan
V254: 255 95	+ Edit Function	White on Magenta
V255: 255 126	+ Tagged block	White on grey

... so why continue to ship the product with something as bad as this. It is not just a matter of taste ... there are ground rules for display design and good use of colour, and Dyalog have broken most of them!

- I object in principle to packages which assume they can take over your PC. I reckon that a key labelled 'Page Up' should Page Up, not throw you into an editor. Of course there are plenty of worse offenders (Oracle Corp. to name but one ... they use Delete as 'delete record'), but I can't get comfortable with the way Dyadic have annexed the cursor pad. Nor can I face the thought of patching the keyboard tables to fix it!

Enough! What Dyalog need is the services of a professional moaner who is not so close to the product that he is quite unable to see its faults. In spite of the above paragraphs, I really do like this system; it is just frustrating to think how much better it could have been presented, and with so little effort.

Where it Could Go Next

Here are a few things on my personal wish list:

- an option to overwrite the session 'in place' rather than always echoing changes to the bottom. I like to work with a bunch of commands on the same bit of screen, and I don't give a damn if I lose the log in the process. I tend to use APL*PLUS II with 'dragdown off' and APL*PLUS/PC poked so that it doesn't clear the line before output. This is untidy, but saves scooting around with the cursor to find the line you want to re-execute.
- mouse support. How about click for execute, double click for trace, right button for edit? Click on the top corner of the window for exit. Sounds silly, but the mouse is a much faster pointing device than the cursor keys, and for exploring a function stack it would be ideal.
- some of that stuff John showed us for watching variables (see my notes on the Vendor Forum on page 64). The beauty of this kind of environment is that you can keep throwing things in without adding any complexity to the user-interface.
- please can the keyboard and screen customisation be a little easier?! Maybe we can have a sensible keyboard layout to start from? Surely we can leave the mainframe behind and put the [] on the [] keys ?? How about a little workspace to sling up a keyboard map on the screen, and allow 'point and grunt' editing of the keyboard tables?

Epilogue

As I said at the start, I am in no way competent to review this product; all I can do is note down some of my reactions to it, and my feelings about it. It is obviously APL's first real step into the 1980's, and as such has no serious rival at present. The economics look good, and the development environment alone gives this APL a significant lead over its two main competitors. It has some rough edges, but they are all easy to fix and I'm sure they will be.

The important thing is that the concept is dead right, and the basics are done consistently and well. I look forward to seeing where Dyadic take it from here, and how STSC and IBM respond.

Powertools/CUA from Systems Intelligence

reviewed by Adrian Smith

Introduction

This review is specifically of the CUA component of the Powertools product. This is part of their 'Forms Processor', and the handling of menus and mice are an extension of existing functionality. The review is mainly based on the CUA workspace which includes examples of all the common types of interaction.

Systems Intelligence is a new name for an old firm; they have a good pedigree as suppliers of add-ins to the APL*PLUS/PC system, dating back to a listing as 'APL Software Technology' in Vector 2.2 (October 1985).

Objectives of the CUA Package

One of the by-products of the published IBM SAA standards is a set of recommendations called Common User Access (CUA); these cover the whole gamut of devices, from the dumbest of dumb mainframe screens to the very latest in mice and windows. Both OS/2 Presentation Manager and MS Windows are based on these guidelines. Powertools/CUA is designed to enable the APL*PLUS/PC developer to build applications which comply with the most advanced guidelines: those which cover the Graphical User Interface (GUI).

If you already base your screen management on the Powertools Forms Processor, you should find that you have a simple upgrade path to full CUA compatibility.

Installation and System Requirements

Powertools must be copied into the APL working directory. It requires APL*PLUS/PC and a MicroSoft Mouse. The installation was fairly painless, with the following minor problems:

- don't be tempted to load the CUA demo workspace without having first started Powertools in the approved fashion. I note from the comments in the □LX that the workspace is supposed to give up with an error message if you do this; in my case it simply hung the system!

- your interpreter must be called APL.EXE. Mine is called APLn.EXE (where n is the version number); I feel that this should be a parameter when you start Powertools from DOS.
- if your mouse is running from COM1: you **must** start Powertools (i.e. APL) with e=256 to disable buffering of the COM1: port. If you don't, you kill off the mouse permanently, not just for the duration of the APL session. The lack of any notes on this is an unfortunate omission from the documentation.

Powertools is remarkably parsimonious in its use of workspace: you will lose a mere 35K of $\square$ WA as a result of loading the entire toolset. Smaller versions can be custom-assembled if required. Note that APL objects over 64K are not supported.

Features

The Systems Intelligence flyer which went out shortly after the last Vector is a good place to start. Essentially, the demo workspace allows you to run entirely in graphics mode, with a screen that looks very much like Windows-2. There are functions supplied to support pull-down menus, dialog boxes (complete with a variety of buttons), window scrolling and so on. The menus can be driven from the keyboard (using Alt-letter combinations), or with the mouse. They support 'greyed' options and option-grouping in the approved style. Obviously, you get the CUA recommended colours by default, but you can over-ride these as required.

You can set up the system for a variety of output devices, from text-mode to Super-VGA. Obviously the graphic modes are necessary if you really want that GUI look and feel; the text mode is useful if you have an old-style system, or need speed rather than beauty.

Performance

On my new 386SX, the GUI demo is almost acceptable; on anything less than a 16MHz 386 it would be like wading through treacle. This is not a criticism of the package as such, just a common consequence of doing all your screen-handling in graphics mode, where the application is working at the pixel level, and thus sending about 128 times as much information to the screen. I have seen just these comments about Word for Windows (as opposed to Word 5); I don't think that Powertools pays any more than the usual penalty for the advanced features.

If you run in text mode, the contrast in speed is dramatic; windows full of text whizz by; and the menus are far more responsive. On the other hand the mouse is largely unusable, as the mouse cursor jumps about in whole character chunks,

and I found it really hard to hit the right bits of screen. You also sacrifice some of the nice features of the windows (like a scroll bar at the side to show you which bit you are looking at). As always, you pay your money and takes your pick.

Documentation

This is by far the weakest element in the whole package: it really is bad, going on dreadful. It is in a small typewriter face on A4 sheets, with the APL bits shown using STSC's old keyword mode (**#is** for assign and so on). It is mostly just a catalogue of functions; fine as a reference manual but quite inappropriate as a user-guide. You can pick up much more by hacking through the CUA workspace then you will ever find in the manual!

I don't think I am just being fussy about this; the cost of using a package is the cost of buying it **plus** the time spent learning it. It is grossly unfair to penalise your purchasers with documentation as bad as this.

Summary

If your company insists on CUA compliant applications, and you want to write them in APL, then you should buy this package. If you just want to dabble a bit with mice and see how a GUI feels, the CUA workspace is a good introduction. Buy the package, set fire to the documentation, and hack the workspace!

Before you do either of the above, stop and ask yourself if this approach is really sensible in the long term. Why on earth are clever programmers wasting their time re-inventing tools that already exist in the Windows environments? Surely the only sensible way of doing this is to run under Windows, and use its toolset from within APL. APL*PLUS/PC has always had an unfortunate tendency to ignore its environment; tools like the Powertools/CUA package (however technically brilliant) can never be more than a short-term fix, a veneer of respectability rather than a true integration into the GUI world.

Availability

Powertools/PC release 6 is available from:

Systems Intelligence (UK) Ltd
Craven House
121 Kingsway
LONDON WC2B 6PA
Tel: 071-831 3978

Book Announcement: APL with a Mathematical Accent

notes by Norman Thomson

APL with a Mathematical Accent

Clifford A. Reiter and William R. Jones, Lafayette Collage

Publ. Brooks/Cole Publ Co., Pacific Grove, CA, USA

200 pp paperback

UK price approx £35 (Three-five ... no misprint!)

Provided that the reader is happy to stay within the bounds of APL1, this text lives up to all its claims. The format is paper covers with spiral binding, making for convenient use alongside a terminal. The page count of 200 belies the extent of the material contained: the publishers have managed to cram every page with information yet without any general feeling of clutter, and they use a heavy APL font which blends very pleasantly with the main text.

The audience is teachers who want to merge APL instruction with teaching maths, and this is the first text I have come across which addresses this objective. The maths reaches quite high levels, e.g. there are functions for eigenvector calculations, Rayleigh Quotients, Gram-Schmidt orthogonalization, Hamming Codes, and adaptive integration - to mention just a few of the many topics covered.

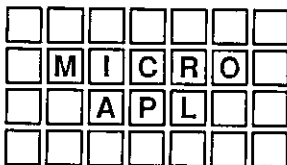
This need not however deter those of lesser mathematical accomplishments, to whom the bulk of the material is addressed. there are a generous number of exercises with answers, and the text is fully adequate for learning APL by self-study on the part of any mathematically motivated student. In short this is a book which deserves the notice of any maths teachers who seek to brighten the lives of their students with the gentle light of APL.

MicroAPL for ...

APL on the widest range of micros

STSC's APL*PLUS Release 9	£450
STSC's APL*PLUS II PC Version 2	£1395
APL.68000	
APL.68000 for the Mac, ST, Amiga	£86.91
APL.68000 - coprocessor version for the Mac II, Amiga	£260.82
APL.68000 runs on many multi-user computers, including:	
Sun, NCR, Wicat, HP300, Aurora, Bull, Nixdorf	£2000

All prices quoted exclude VAT and p&p



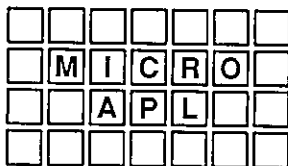
MicroAPL Ltd.
South Bank Technopark
90 London Road
LONDON, SE1 6LN
Telephone: 071 922 8866
Access and Barclaycard
Orders accepted

MicroAPL for ...

Consultancy and Training

MicroAPL has 10 years' experience with small-computer APL systems - stand alone, networked or multi-user - and using all the major versions of APL. We provide a full range of consultancy services including feasibility studies, project management, programming, documentation and support services. Our programming skills cover all areas of APL and systems programming. MicroAPL also offer a comprehensive range of APL training courses.

For more details, contact David Eastwood at:



MicroAPL Ltd.
South Bank Technopark
90 London Road
LONDON, SE1 6LN
Telephone: 071 922 8866

RECENT MEETINGS

This section of VECTOR is intended to document the seminars given at recent meetings of the association; it is of particular value to members who live away from London. It also covers other selected events which may be of general interest to the APL community.

If you would like to speak at one of the regular British APL Association seminars, please ring the Activities Officer (address on inside back cover) who will respond enthusiastically to your offer.

Vendor Forum: 20th April 1990

notes by Adrian Smith

Highlights

This was an interesting and worthwhile meeting, at least as much for what was not said, as for what was said. I.P. Sharp were conspicuous by their absence; I think we can reasonably conclude that Sharp APL is now a 'stabilised' product, and that they simply had nothing interesting to say about the future.

Slightly more surprising was the fact that neither STSC, nor MicroAPL had anything much to say either. This was in sharp contrast to previous meetings, where STSC were keen to stress the future convergence of their APL platforms (the obvious ones being APL*PLUS/1000 and APL*PLUS II) on a common standard. The impression I gained was that STSC are now taking a 'cash cow' approach to APL in general, and a 'dead dog' approach to their mainframe system. MicroAPL were also keen to stress stability and portability; in particular they majored on the benefits of free runtime, with some impressive examples of user-developed Mac packages.

The two outfits which did have something to say were IBM and Dyadic. Under the cover of a standard disclaimer, Dave Selby proceeded to tell us about his current research into APL2/PC; the 32-bit packager, and a totally new session manager are the key elements. Following in the wake of Pete 'n Pam (now coyly renamed as P&P, I regret to say), Dyadic have just been born again as Pete 'n John 'n Pauline (and the Royal Bank of Scotland). In short, they have just bought themselves out, and with one mighty bound have escaped from the clutches of both Lynwood and Hunting plc. Long may they prosper! John Scholes showed off a bit more of their new environment (see the review of this on page 49), with some fascinating additional (very experimental) bits for watching variables, both local and global. I think I am going to like this!!

Now to the talks proper: it was agreed that Dave Selby should bat at number-1, but following some haggling on the current value of $\square IO$, Romilly Cocking started proceedings at number-0 ...

Romilly Cocking, representing APL*PLUS

Romilly saw the current APL marketplace split up between:

- the core APL users, who rely extensively on APL for critical corporate systems. Even those who may decide now to get out of APL will need 5-15 years to achieve it. These users need stability and connectivity; they are interested in performance and support costs.
- power personal computing. They want more $\square$ WA and more speed. They are beginning to get interested in connectivity and workstations.
- vertical-market developers, producing closely targeted software in APL. They need developer productivity, communication with other languages, and a stable, long-term platform.
- Info Centre diehards. They want to carry on running ADRS and ADI - that means continued support for VS APL.

Next, Romilly turned to the APL*PLUS product family:

- mainframe APL. This is a mature market, where the revenue from maintenance agreements now exceeds the revenue from new licences. There is almost no drive for new features, except in the Sharefile arena, where enhancements to support networking and workstations are being worked on. These are likely to be 5 years away from the marketplace.
- workstation and mid-range APL. At long last the Unix market is growing from a significant installed base; in the UK it is being driven by UK subsidiaries of US companies. Many of the above 'core' users are looking hard at Unix opportunities, in particular STSC have come under unexpected pressure to port to the RS/6000 workstation ... maybe this will be at APL90?? We can look for graphics and X-windows interfaces in these APLs.
- APL*PLUS II. A pleasant surprise, as sales have been way over plan! The next release will be around in late summer / early autumn, and will feature a 'high' session manager (about 140K more space in the DOS box), more interfaces through $\square$ NA, and better comms and network support.
- APL*PLUS/PC. This seems not to have been hit by the APL*PLUS II launch, so it is obviously in a different segment of the market. The main pressures are towards enhancing the development environment, with windowing-type debuggers, similar to MicroSoft Codeview. (There was a fairly broad grin on John Scholes' face at this point ... I think he agreed with the idea!) Other things to look for will be better interfacing (sooner or later $\square$ NA will turn up), and good behaviour under OS/2.

Romilly was asked about APL*PLUS/PC pricing, and the implications of dongles on APL*PLUS II. He thought that an APL*PLUS/PC 'Developer's Kit' was unlikely, as STSC were milking a growing revenue stream from all those vertically integrated software products. However there was the possibility of a corporate runtime for existing APL*PLUS/1000 mainframe sites. Romilly removed his STSC hat for long enough to admit to a thorough detestation of dongles, but was pessimistic about the future. In answer to "When will the dangle go away?" he replied " ... when the Italians get honest!". Following recent experiences with Statgraphics, I would give him all the support I can in keeping APL*PLUS/PC undongled. Romilly also mentioned in passing the availability of 'partial' systems (basically software only with no documentation) at much reduced prices please can we have a product guide entry!

David Selby, representing APL2/PC

Being an IBMer, Dave had to prefix his talk with a disclaimer, pointing out that anything he said was purely hypothetical, and in no way represented an announcement, a statement of direction, or anything else! The reader is warned.

Dave covered 'work in progress' on both the interpreters themselves, and on the various auxiliary processors that go to make up the complete system:

- the 16-bit system gets a proper implementation of) *CONTINUE*,) *SINL* and) *LIB*. The SVP storage allocation is improved, and will hopefully be Camacho-proofed (see Vector 6.1 page 58). Parameters can be passed from the DOS command line, and there is a new *TIME* utility to monitor code performance and flag 'hotspots'.
- the 32-bit system has a number of performance improvements, and is extended to handle complex numbers. There is also the option to load most of the interpreter high, leaving a mere 30K in the DOS box. APL2PACK.EXE has an improved user-interface, and will package 32-bit code, to be run (with no license implications) by APL2RUN.EXE.
- AP120 - the session manager. Dave admitted to this having been "the bane of my life" in the current product. It is rewritten from the bottom up, although the screen layout has been preserved, and it can still be used in 'page-mode'. However, it now does sane things when you type off the edge of a line (you just push the screen across to the left), it will scroll with the arrow keys, and can run split-screen, with a graph in the other half of the screen. You can journal the session to file, or printer.
- AP124 - full-screen processing. Extended to handle character arrays better, and to give 'selectable' fields for menu operations.

- AP210 - much faster reading of simple DOS files, with a user-defined translation table. (EBCDIC to ASCII is provided as the default option.)
- Other odds and ends: Hercules+ support; Mouse support; Font editing and codepage creation; on VGA graphics "it was very stupid of us not to put this in the original product".

Dave was asked about selective specification, and $\square NA$ support. He replied that the theory of selective specification was still not fully understood, and that Jim Brown has asked him to hold back; he felt that a general $\square NA$ is hard to do, and it would not be in the product in the foreseeable future.

Anthony Camacho, representing I-APL

Anthony reported that the original objective of I-APL had been met, with the implementation of a working interpreter, and the sale of over 2,000 copies of I-APL and supporting documentation. Lots of ports were still in progress, but I-APL was now available on more than half the initial target list. He was at pains to point out that I-APL was in no way in competition with any commercial APL, and that he frequently recommended users to move towards an existing commercial APL when they outgrew the limitations (e.g. a 28K workspace on the IBM PC) of the free system.

Getting schools to use I-APL in the classroom remains a slow process, but it is beginning to get a following among both teachers and pupils, who form the bulk of the 1600+ mailing list. (Incidentally, it was good to see "APL in the Mathematics Classroom" recommended in the April Byte.)

The major work in progress is the development of a 32-bit version of I-APL to run on the Archimedes system; this has required a thorough rehash of the transcendental functions to give full IEEE precision.

David Eastwood, representing MicroAPL

David took us on a guided tour of MicroAPL's first 10 years, from hardware breakthroughs like the first double-sided floppy disk, to the Sinclair QL. MicroAPL still service 3 of their original CP/M systems, which is an interesting comment on the longevity of some computer applications. He emphasised the importance of their free runtime policy, and low price, in generating a stream of Mac applications which were impossible to tell from any other Mac system. Over 85% of these systems are overseas, mostly in the USA and Germany. David demonstrated three systems, all built by end users with no MicroAPL support: a superb starmap (complete with planets if you can find them); a program to assist

in downloading Reuters data; and a program to help with Cladistic analysis (whatever that is!).

MicroAPL have recently been appointed Value Added Retailers for the NCR 'Tower' range, so expect some interesting announcements at APL90.

Peter Donnelly & John Scholes, representing Dyalog APL

Peter started with the details of Dyadic's management buyout: as from the end of March the company is in the capable hands of Peter, John, Pauline (of Mexican Snacks fame ... see Vector 3.2 page 78), and the Royal Bank of Scotland. Peter was clearly delighted that they could now get on with the job of pushing forward their APL systems, without constantly having to justify this odd activity to their paymasters.

Peter was also rather chuffed to announce a complete range of RS/6000 systems, running Dyalog APL. He quoted some benchmarks for these, which I failed to copy down accurately, but which basically put this platform a good factor of 10 ahead of the fastest 386/PC on the market. The floating-point performance is phenomenal, with a π benchmark coming out around 40 times faster; this is getting within a factor of 10 of a 3090 mainframe, all for around £50,000 ... no wonder Dyadic are excited about it.

"You can do all sorts of crummy programming on these machines, and get away with it!"

John Scholes followed up with another live demo of his new toy, this time demonstrating the incredible feat of programming a recursive function:

```
IOTA 12
1 2 3 4 5 6 7 8 9 10 11 12
```

The interesting bit was to watch this executing line-by-line on screen, with the 'trace' windows automatically stacking and vanishing as the recursion unwound.

Several bursts of spontaneous applause marked this show (suggesting that Dyadic must be getting something right), but the best was yet to come. John saved his work, then gingerly loaded an experimental system, and showed us)WATCH. This lets you 'watch' the state of any variable, the object in question being shown in a pop-up window at the top of the screen. It happily handles numeric and nested data, and allows you to tab to the window and scroll it around if the variable is too big.

John tried watching the result variable of the recursive *IOTA*, (syntax *)WATCH IOTA; Z*) which gave a stack of windows of gradually increasing width (1, 1 2, 1 2 3, etc.) which peeled off the screen as the functions vanished from the stack. The whole show took about 1/5th of a second ... this is fast!

John was happy to take questions about the future of this product, but was as yet unsure as to its final form; in particular he would like to see it integrated with a variable editor, similar in form to the function edit/trace capability. When asked about *SI DAMAGE* when you edited suspended things:

"There is no *SI DAMAGE* in Dyalog APL; we didn't think it was a good thing to have."

Summary

From the evidence of the day's talks, I would draw the following conclusions:

- Sharp APL is almost at the end of the road, unless someone can raise the cash and commitment to buy it back from Reuters. This makes me rather sad, as Sharp have always been in the forefront of interpreter design, and their APL is still ahead of the competition in many important areas.
- STSC have a problem. APL*PLUS/PC has not moved significantly since release-5, and is existing increasingly on maintenance (disguised as upgrades) and run-time. APL*PLUS II has rather a short life ahead of it, unless it can drop its price to compete more effectively with APL2/PC, and free 32-bit runtime. In the increasingly vital workstation market, Dyalog APL has a substantial lead, both in the platforms supported, and in the quality of the development environment. The APL*PLUS/1000 mainframe product is moving rapidly from 'cash cow' to 'dead dog' status, as the mainframe becomes increasingly marginalised by distributed systems.
- IBM may just have got it right with the (hypothetical) release 1.2 of APL2/PC. Just as it took Lotus to version 1.01A to home in on an acceptable product, IBM have addressed the obvious deficiencies in the initial system, and look to me to have a winner in the making. I just hope they don't fluff it!
- MicroAPL have got themselves a really cosy niche as the only viable APL for a whole range of 68000-based machines; they deserve every credit for going for growth with cheap pricing and free runtime, not sitting around raking in easy money.

- Dyadic systems may have chafed under the yoke of a watchful paymaster, but I wonder if it did them some good by squashing some of the more exotic ideas at birth? I have always felt that they treated their APL as an interesting toy, and had a tendency to fill it with fascinating ideas (like *TOTAL++/*) just because it was interesting to do! I hope they employ a professional devil's advocate; if so I think they have a great future, and are likely to get most of the APL trade going in the lucrative workstation market.

Of course, I confidently expect to be proved wrong about most of the above conclusions; all I can say in justification is that they are an honest reflection of my feelings at the meeting, as rationalised on the train on the way home, and written down the day after!

Adrian Smith
Brook House, Gilling East, York
21st April 1990.

SPECIAL FEATURE:

J

BY KENNETH E.
IVERSON

J is a dialect of APL first presented at APL90 [1]. It is designed for easy porting, and is already available as freeware on a number of machines [2]. The present paper is an informal introduction to J, designed to illustrate its innovations, as well as its potential use in teaching. It begins with examples, and concludes with a discussion of its relationship with other dialects of APL. The reader is advised to study each group of examples, and perhaps to experiment with them and related expressions on a J system, before reading the accompanying comments.

J

by Kenneth E. Iverson

Acknowledgements

I am indebted to a number of colleagues: to Roger Hui for collaboration in designing and implementing J, and to E.E. McDonnell and Roger Hui for suggestions used in the examples. I am also grateful to David Ziemann for the invitation to contribute to the conference issue of VECTOR.

```

      i. 4
0 1 2 3
      a=. i. 4
      2*a
0 2 4 6
      +/a
6
      a+/a
0 1 2 3
1 2 3 4
2 3 4 5
3 4 5 6
      a!/a
1 1 1 1
0 1 2 3
0 0 1 3
0 0 0 1
      a-/a
0 _1 _2 _3
1 0 _1 _2
2 1 0 _1
3 2 1 0
      ;: 'abc=.i.4 5'
```

abc	=.	i.	4 5
-----	----	----	-----

SPELLING. The words used to denote primitives in J are spelled using ASCII characters, and are formed from one ASCII character or from two, the last of which is either a dot or a colon. Such words need not be surrounded by spaces.

In an expression such as `+/` the adverb `/` applies to the verb `+` to form a related verb whose monadic case is called *reduction* (or *insertion*) and whose dyadic case is called *outer product* (or *function table*).

The verb `;:` boxes the words in its argument according to the rhematic rules of J.

```

(<'abc'),(<'de'))


|     |    |
|-----|----|
| abc | de |
|-----|----|


'abc';'de'


|     |    |
|-----|----|
| abc | de |
|-----|----|


<\'abcd'


|   |    |     |      |
|---|----|-----|------|
| a | ab | abc | abcd |
|---|----|-----|------|


<\0 1 2 3


|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 | 2 | 0 | 1 | 2 | 3 |
|---|---|---|---|---|---|---|---|---|---|


+/\0 1 2 3
0 1 3 6
+/\. 0 1 2 3
6 6 5 3

```

```

a=.1 2 3 4 5
sum=. +/
sum a
15
prefix=. \
sum prefix a
1 3 6 10 15
div=. %
3 div 5
0.6
and=. *.
or =. +.
nor=. +:
not=. -.
min=. <.
floor=. <.
power=. ^
is=. =.
log is ^.
(a=2) or (a=4)
0 1 0 1 0

```

BOX and SCAN. The monad (monadic case of) < boxes its argument to form an atomic representation of it (of rank 0).

The prefix scan adverb \ applies its verb argument (in these examples the monads < and +/) to each prefix of the argument.

The suffix scan \. applies its verb argument to each suffix.

IS. The copula =. may be used to assign names to entities other than nouns.

In particular it can be used to assign any desired mnemonic to a primitive. Assign suitable names to primitives chosen from the Language Summary (Appendix A), using experimentation (or the names from the Summary) as guides to the behaviour of the primitives.

```

16^0.5
4
0.5^~16
4
^~1 2 3 4
1 4 27 256
into=.%~
6 into sum i. 6
2.5
randompermutation=. ?~
randompermutation 6
2 4 1 3 0 5
2^5
32
2&^5
32
poweroftwo=. 2&^
sqrt=. ^& 0.5
sqrt 0 1 2 3 4
0 1 1.4142 1.7321 2

t=. i. 2 3 4
t
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
16 17 18 19
20 21 22 23
,"2 t
0 1 2 3 4 5 6 7 8 9 10 11
12 13 14 15 16 17 18 19 20 21 22 23
+/"t
12 14 16 18
20 22 24 26
28 30 32 34
+/"2 t
12 15 18 21
48 51 54 57
+/"1 t
6 22 38
54 70 86

```

CROSS and COMPOSITION. The adverb *~* *commutes* or *crosses* connections to its argument verb. When *f~* is applied monadically (as in *?~y*), it uses *y* as both arguments to the dyad *f*. Try */:* 'cat' and */:~* 'cat'.

The *and* conjunction *&* attaches its noun argument to a verb to produce the corresponding monad. Experiment with cases such as *10&^*. (base-ten log), and *e.&'aeiou'* (vowel-test); apply it to an argument such as *'i sing of olaf'*

RANK. The rank conjunction *"* applies its left argument to cells of rank determined by its right argument.

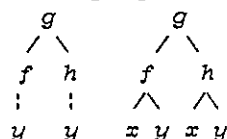
Insertion (denoted by */*) applies its left argument between the items (major cells) of the argument of the derived verb. The rank operator applied to *+/* produces summation over other axes.


```

      dts=. -**
      7 dts 4
33
      7 (-**+) 4
33
      3!5
10
      3(!*-)5
_20
      (!*-)5
_600
      (!,-)5
120 _5
      (!,-) 3 4 5
      6 _3
      24 _4
120 _5
      2 (> or =) i. 5
1 1 1 0 0
      (>&2 and <&5) i. 7
0 0 0 1 1 0 0
      t=. i. 3 4
      t
0 1 2 3
4 5 6 7
8 9 10 11
      # b=. 3 2 5 4 1
5
      #t
3
      3 %: 27 64 125
3 4 5
      am=. # %~ +/
      am b
3
      gm=. # %: */
      gm b
2.6052
      am t
4 5 6 7
      am"1 t
1.5 5.5 9.5

```

FORK. Three successive verbs (in isolation) comprise a *fork*. The monadic and dyadic cases of the fork (fgh) are defined by the following diagrams:



yields the number of items (major cells) of its argument, and *am* is therefore the arithmetic mean of the items of its argument.

Since $n\%:y$ denotes the n -th root of y , *gm* is the geometric mean.

The rank conjunction serves to apply *am* individually to lesser cells.

Experiment with expressions such as $(\wedge * ! - \%) 1 + i.$ 4 to illustrate that an expression of the form (*defgh*) is equivalent to the fork (*dez*), where $z = fgh$.

Experiment with the analogous forms (*p or q and r*) for propositions *p*, *q*, and *r*.

Examine the tautology $taut = <: = < or =$

Apply the geometric mean *gm* to the argument *i.* 2 3 4. Also try gm^2 , and gm^3 .

```

cos=. 2&o.
g=. %&180
rfd=. o.&g
rfd 0 30 45 60
0 0.5236 0.7854 1.0472
cosd=. cos& rfd
cosd 0 30 45 60
1 0.86603 0.70711 0.5
sind=. 1&o.&(o.&(%&180))
#&> 'abc'; 'de'; 'fghij'
3 2 5

3 (+%) 5
3.2
(+%) / 10 # 1
1.6182
(+%) / 7 # 1
1 2 1.5 1.6667 1.6 1.625 1.6154
(+%) / 1 2 2 2 2 2 2
1.4142
(+%) / 3 7 15 1
3 3.1429 3.1415 3.1416
integertest=. =<.
integertest 2%~i. 8
1 0 1 0 1 0 1 0

```

```

>: y=. 0 1 2 3
1 2 3 4
>: ^: 2 y
2 3 4 5
>: ^: _1 0 1 2 3 y
_1 0 1 2 3
0 1 2 3 4
1 2 3 4 5
2 3 4 5 6
0+^: 7 (1)
13
0+^: (i.9) 1
0 1 1 2 3 5 8 13 21

```

COMPOSITION of VERBS.

rfd gives radians from degrees, and *cos&rfd* is the composition of the functions *cos* and *rfd*. Hence *cosd* is the cosine function for degree arguments.

HOOK. Two verbs (in isolation) comprise a *hook*. The monadic and dyadic cases are defined by the following diagrams:

```

      g      g
     /\     /\
    y h     x h
      |      |
      y      y

```

The reductions by the hook +% provide continued fraction [3] approximations to the golden mean, the square-root of 2, and pi.

POWER and CHAINS.

The conjunction $\wedge$ produces a verb whose monadic case $v \wedge k y$ is defined as k applications of the verb v to the argument y , and whose dyadic case $x v \wedge k y$ is defined as the last of a chain of $k+1$ values beginning with $t_0 = .x$ and $t_1 = .y$ and $t_2 = .t_0 v t_1$ and $t_3 = .t_1 v t_2$, etc.

```

root =. 'y.^0.5' :: 'y.^%x.'
root a=. 0 1 64 729 4096
0 1 8 27 64
  3 root a
0 1 4 9 16
  r=. 'y.^0.5' :: ''
  3 r a
domain error
  (log=. '10^y.' :: ^.) 1e6
6
  100 log 1e6
3
  m1=. '$.=. 1,y.#2'
  m=. m1;'t=. 1';'t=. t*1+#$.'
  >m
$.= 1,y.#2
t=. 1
t=. t*1+#$.
  factorial=. m::''
  factorial"0 i. 6
1 1 2 6 24 120
  M1=. '$.=. 2-y.=0'
  fact=. (M1;'1';'y.*$: y.-1')::''
  fact"0 i. 6
1 1 2 6 24 120
  power=.2:: (m1;'t=. {';'t=.x.&t')
  ! power 0 " 0 i. 6
0 1 2 3 4 5
  ! power 1 " 0 i. 6
1 1 2 6 24 120
  ! power 2"0 i. 6
1 1 2 720 6.2045e23 6.6895e198
  each=. 1:: '<@x.&>'
  words=. 'i';'sing';'of';'olaf'
  !. each words

```

i	gnis	fo	falo
---	------	----	------

```
EACH=. 1:: 'x.'">'
```

GENERAL DEFINITION.

The conjunction `::` produces a verb whose monadic case is determined by the left argument, and whose dyadic case is determined by the right. If one argument is empty, the corresponding domain of the derived verb is empty.

Either of the arguments may themselves be verbs.

The sentences in a boxed list are executed in the order specified by `$`. (the sequence list), which is initially set to `i. ns`, where `ns` is the number of sentences in the list.

The verb *factorial* is defined iteratively and the verb *fact* recursively, using `$`: (self-reference) to refer to the verb being defined.

A conjunction is defined by `::` with a numeric left argument (2), and an adverb is defined by a left argument of 1.

EACH is a definition of *each* based upon the conjunction *UNDER* (which see).

```

a=. i. 3 2 4
1{a
8 9 10 11
12 13 14 15
0{1{a
8 9 10 11
1 {"2 a
4 5 6 7
12 13 14 15
20 21 22 23
{:p=. 4 4 $ 1 3 2 0
1 3 2 0
1 3 2 0
1 3 2 0
1 3 2 0
{/\p
1 3 2 0
3 0 2 1
0 1 2 3
1 3 2 0
{:b=. 4 5$'abcdefghijklmnopqrst'
abcde
fghij
klmno
pqrst
i=. 2 2 $ 3 17 2 14
i{,b
dr
co
W=. 2 2$'DRCO'
W i}b
abCDe
fghij
klmno
pqRst
{'AB';'abc'

```

Aa	Ab	Ac
Ba	Bb	Bc

FROM, MERGE, and CATALOGUE. The indexing function *from* (*{*) selects *items* from its right argument, and can be used with rank to select items from lesser cells.

The indices *i* in the merge expression *s i}{t* (which uses the merge adverb *}*) refer to positions in the ravel of *t*, that is to elements of *i.\$t*. The indicated positions are replaced by elements from *s*.

Used with a verb argument (as in *s v}{t*), the indices are determined by applying the verb *v* to the index array *i.\$t*.

Also try *m}{y*, the monadic case of merge. If *y* has *n* items, if the elements of *m* belong to *i.n*, and if *m* has the same shape as an item of *y*, then the result has the same shape as *m*, and contains elements selected from the various items of *y*.

```

a=. 100 } b=. 1000
log=. 10&^
log a,b
2 3
a+"log b
100000
a*b
100000
invlog=. log ^: _1
invlog log a,b
100 1000
invlog (log a) + (log b)
100000
{:c=. 0=?11#4
0 0 1 0 0 1 0 1 0 0 0
</^c
0 0 1 0 0 0 0 0 0 0 0
</^". c
0 0 0 0 0 0 0 0 1 0 0 0
a=. 'abcdefghijklmnopqrstuvwxyz'
text=. 'i sing of olaf'
{: j=. (nfa=. a&i.) text
9 0 19 9 14 7 0 15 6 0 15 12 1 6
nfa ^: _1 j
i sing of olaf
encode=. (?~# a)& i.
encode j
22 14 21 12 23 22 20 1 22 20 17 8 1
encode"nfa text
uvnulwvtavtqha
test=. {:"(encode & nfa)
test text
i sing of olaf
enc=. '({a):3+y.' :: ''
dec=. '({a):3~y.' :: ''
pair=. enc&dec
{:"(pair&nfa) text
i sing of olaf

```

UNDER. When used with two verbs f and g (as in $f" g$), the conjunction " produces a verb that applies f under g ; that is, the monad g is first applied to the argument(s), f is applied to the result(s), and the inverse of g is then applied to "undo the work of g ", or to "map the results back to the original domain".

The inverse of g is the function produced by $g^{\wedge} : _1$. Such an inverse exists for about a dozen primitive monads (such as $! . \wedge . o . < >$), for a larger number of derived functions (such as $2&o . n&! . n&* +&n + . . * &M$), and for compositions of invertible functions, as illustrated by *encode&nfa* (Number From Alphabetic).

The *obverse* conjunction $\&$ can also be used to specify a formal inverse, as was done for the verb *pair*.

RELATION TO OTHER APL DIALECTS

Spelling. Although orthography and spelling are superficial aspects of language, changes in them are jarring, and should be justified by significant advantages. Restriction to the ASCII alphabet simplifies keying (by avoiding a third case as well as avoiding backspacing to produce composite characters), and removes serious problems in display devices, printers, and publication.

Moreover, the spelling scheme adopted in J retains the major advantages of the special APL alphabet: it obviates reserved words and does not require spacing between words (as required in $a \bmod b$, contrasted with $a \div b$). Finally, it provides mnemonic clues at least as effective and international as those provided by the special APL alphabet.

Although mnemonic schemes should perhaps not be studied as such, their utilization will grow more quickly for a user who is at least cognizant of their existence and their general character. Such knowledge can be gleaned from a quick review of the Language Summary of Appendix A. For example, $+$, $.$ and $*$, for *or* and *and* not only use the analogies with *plus* and *times* exploited by Boole, but the *multiply* symbol $*$ also suggests the least common *multiple* also represented by $*$. Finally, $<$, and $>$, for *min* and *max* suggest analogies with the relations $<$ and $>$; the $\wedge$ used for power has some pedigree in mathematics (having been used by De Morgan), and leads to $\wedge.$ for the inverse (log) and to $\wedge:$ for the *conjunction* power; the $/$ in the symbol $\%$ suggests division and leads to $\%$, for matrix divide, and to $\%x:y$ for root (because $\%$ occurs in the equivalent expression $y \wedge \% x$).

Rank, Cells, and Items. The last k axes of an array y define a *cell* of rank k , and the rank conjunction (as in v^k) produces a derived verb that applies v to each k -cell of y . A major cell of y (of rank one less than the rank of y) is called an *item* of y ; verbs such as catenation ($,$) and insertion ($+/\$) apply between items. Consequently, the rank conjunction can provide their application to lesser cells, obviating variants such as *comma-bar* and *slash-bar*.

The utility of the rank conjunction is further enhanced by permissive frame-building, as illustrated by $\dot{z} . "0 \dot{z} . 5$ and $>'a'; 'ab'; 'def'$.

Scans. The utility of scan ($v \backslash$) is enhanced by applying the *monad* v to the segments rather than applying the derived monad $v /$ to them. As a consequence, the partial sums $+\backslash$ (previously denoted by $+\backslash$) is read not as *plus scan*, but more properly as *sum scan*, since the phrase $+\backslash$ denotes *sum*. Moreover, the *suffix scan* $\backslash.$ provides useful behaviour not provided by the prefix scan, even in conjunction with commutation ($\sim$) and reversal ($!.$).

The dyadic cases of $v\backslash$ and $v\backslash.$ apply v to infixes and outfixes, determined by a window or blind of width specified by the left argument. The infixes are useful for running sums, maxima, etc., and the outfixes are useful in cases such as $-1*\backslash.$ $x-r$ for the derivatives of a polynomial with respect to each of its roots.

Functional Programming. Although the operators in APL provided some capacity for functional programming from the outset, the use of a functional programming style was severely hampered in many systems by the inability to:

- 1) Apply operators to all functions, including derived and user-defined.
- 2) Use the copula to assign names to entities other than nouns, as in $sum = +/$ and $inv = 1::'x.\wedge: _1'$
- 3). Indicate self-reference in recursive definition without assigning a name.
- 4) Use a conjunction to define verbs, adverbs, and conjunctions within a sentence.

In J these limitations have been removed.

Further Reading. The foregoing examples omit many interesting facilities of J, such as the treatment of complex numbers and the *execute* (which has rank 1 and therefore executes each row of a table argument, and whose dyadic case provides control for error handling).

For further exploration, the Language summary may be used as a guide, and translation from earlier dialects may be practiced using so-called "idiom lists", and examples from References 4-5. Finally, the appendix of [1] provides examples of the use of J in a variety of topics, but should be approached with a warning that some definitions have been changed.

REFERENCES

1. Hui, et al, APL \2, APL90 Conference Proceedings, ACM APL Quote-Quad.
2. Versions made available at the APL90 Software Exchange included IBM PC and compatibles, and Apple Mackintosh. Inquiries concerning manuals and further ports should be addressed to ISI (listed in the Vector Product Guide).
3. C.D. Olds, Continued Fractions, The Mathematical Association of America New Mathematical Library, 1963.
4. Iverson, K.E., Notation as a Tool of Thought, CACM 1980.
5. Thompson, N., APL Programs for the Mathematics Classroom, Springer-Verlag 1989.

Appendix A: Language Summary

=	NubClassify ; Equal	Is (Local)	Is (Global)
<	Box ; LessThan	Floor ; Min	Decrement ; LeOrEq
>	Open ; GreaterThan	Ceiling ; Max	Increment ; GtOrEq
-	NegativeSign/Infinity	NOT USABLE	NOT USABLE
+	Conjugate ; Plus	; GCD (Or)	; Nor
*	Signum ; Times	; LCM (And)	; Nand
-	Negate ; Minus	Not(1-);Less	; Match
%	Reciprocal ; Divide	MatrixInv ; MDiv	SquareRoot ; Root
^	Exponential ; Power	NaturalLog ; Log	Power
\$	ShapeOf ; Shape	SequenceList	SelfReference
~	Both ; Cross	Nub;	Nubsieve ; NotEqual
	Magnitude ; Residue	Reverse ; Rotate	Transpose
.	NOT USABLE	Determ ; DotProd	Definition
:	NOT USABLE	Custom	Itemize ; Laminate
,	Ravel ; ChainItems	Raze;	Words
;	Table ; Link		
#	Tally ; Copy	Base2 ; Base	Antibase2 ; Antibase
@	Atop-DeferAxes	Dir-Cyc ; Permute	AtomPermute
/	Insert ; FunctionTable	Cut	GradeUp ; Sort
\	Prefix;Infix	Suffix;Outfix	GradeDown ; Sort
{	Catalog ; From	First ; Take	Right (Dex)
}	Merge	Rest; Drop	Left (Lev)
"	Const RankUnder	Execute ; Execute	Format ; Format
&	With	Then	Obverse
!	Factorial ; OutOf		
?	Roll ; Deal		
(	Open Parenthesis		
)	ClosePar-Label-Cmd		
α		Alphabet	
ϵ		; Epsilon (Member)	
i		Integers ; IndexOf	
π		PiTimes ; Circular	
x		First Argument	
y		Last Argument	
E		; MemberOfInterval	
X		External (Foreign)	

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers

APL Statistics Library Project

*by the ASL Team: Tony O'Hagan, Jake Ansell, Alan Sykes,
David Eastwood, John Searle*

Abstract

APL has been recognised as a powerful computational aid by a number of statisticians for some time. Unfortunately they have generally had to produce their own APL code for even the standard statistical computations. This has inhibited the growth of APL within statistics. The APL Statistics Library, ASL, intends to overcome this problem by bringing together the knowledge and material already developed within the APL/Statistics Community.

The paper describes the ASL project, sponsored by the British APL Association, at its inception outlining the need and market for ASL. The paper also details the unified framework of the library and the underlying philosophy of system design and structure.

Introduction

Over the last few years APL has gained substantial ground in statistics. An increasing number of university and polytechnic statistics departments are using APL as the medium for teaching and/or research in statistics. The list includes Birmingham, Swansea, Warwick, UCL and many others.

Whilst this progress is commendable it is realised that the battle to convert others from the standard packages to APL is likely to be more difficult. It is possible to suggest to the unconverted texts such as Anscombe [1981] or Grenander [1982], or pass them other papers/information which demonstrate the power of APL. This route, though, is not effective in marketing APL against standard statistical packages. What the user requires is the ability to compete with the packages on a level par from the start.

The gap between receiving an interpreter and being able to do useful statistical analysis has to be bridged to convince the non-APL statistical user. The APL Statistics Library, ASL, hopes to bridge that gap and hence gain the user for APL. The project is being carried out under the aegis of the APL Statistics User Group and with financial and personnel support from the British APL Association.

The goals of the project are two fold: to produce a library of statistical functions and promote the use of APL. Through the production of the library it is hoped to convince other statisticians at the research level of the power of APL, and hopefully this will overspill into teaching. The possible audience for APL through this route is large given the numbers of school and undergraduate students taking statistics courses. (Most institutions of higher education have at least one introductory course in statistics delivered to Science and Social Science Faculties.)

The paper describes the ASL project at its inception. The launch of the project took place at the ASL Workshop at Gregynog, Wales, September 1989. Most of the material presented in the paper was either finalised or arose in discussion at the workshop. Given that the ASL is still in its infancy most of the description will concentrate on the future of the project. However it is hoped that by the time of the APL 90 conference it will be possible to provide at least an interim report on the successes of the project.

Need for ASL

Currently, having obtained an APL interpreter the statistician, whether she or he wishes to use APL for lecturing or research, has to spend time amassing her or his own library of routines. Many of the routines will be either borrowed from others, copied from the literature or written by the individual. Bringing together this material takes both time and effort but also brings with it no guarantee of good or well designed code. The continual reinvention of the wheel seems a pointless exercise.

The alternative is to purchase an APL Statistics package with the code possibly closed to inspection. Again the buyer will not necessarily know how well the software has been put together and it is likely that the underlying procedures will not be described. Another reason why this approach is unsatisfactory is that it runs contrary to the concept of using APL as opposed to other languages. Most individuals turn to APL to gain flexibility to attempt non-standard procedures.

ASL will become the alternative. The APL Statistics Library is designed to retain the flexibility of APL with the provision of a wealth of tested code. Many other attempts to produce either idioms or libraries of code have failed to take a systematic approach to the development or certification of the code. (It would be invidious to name such software.) ASL is attempting to achieve both through careful consideration of appropriate design specification and through thorough testing of produced code.

It is hoped therefore to produce software satisfying at minimum the basic requirements/needs of a statistician with well tested, even approved, software. The software has to be designed so that it is easy to build more sophisticated routines and still remain flexible. It is hoped that ASL will set the standard for APL statistical computing.

Beyond the need of an individual statistician there is a need to ease the development of statistical computing. APL is one of the few languages where it is possible to enjoy the act of programming and achieve adequate code rapidly. The language does not generally impede statistical desires as many other packages or languages do. We are not claiming it is perfect, though; there will always be new squiggles to add or greater flexibility that can be created. The language is still developing.

The outcome of using APL is the relatively rapid transformation of statistical concepts into executable statements. This aids the development of statistical computing. It is not surprising that the APL Users Group gave birth to the Royal Statistical Society's Computational Section.

Beyond producing new algorithms the need for APL in statistical practice is almost obvious. Data analysis is necessarily exploratory. It is not a simple matter of applying a single standard analysis to the data, but trying different kinds of analysis, looking at the data in different ways, and drawing together a range of conclusions. The APL environment uniquely supports this activity.

The need is obvious. The market is large. All that is really needed is the organisation to bring it together. This has been created in the ASL project with the support of both the British APL Association and APL Statistics Users Group.

The Basic Framework

The APL Statistics Library represents a store of information about statistics written in APL. However, to be usable this information must be stored in some accessible manner. Currently the ASL Team has decided to retain the analogue with libraries by describing the structures as shelves which contain volumes. A volume will consist of a coherent set of functions which could be released by itself, see later for examples. A shelf may consist of one or more volumes.

The bottom shelf of the library will consist of the routines of general use. These will be general utilities which will be needed throughout the libraries, some basic statistical, numerical and graphical functions. The volumes thus would be 'utilities', 'numerics', 'statistics' and 'graphics'. There may be a need for other volumes on this bottom shelf. The main concepts arising out of the Workshop

about the bottom shelf are given in Appendix A. These arose following a paper given by Norman Thomson (IBM) on the Bottom Shelf. As indicated in the appendix it is already felt that the graphics volume will be split into two parts, WAGS (Warwick APL Graphics System), see O'Hagan [1990], the basic tools for graphics, and basic graphics for statistics.

On top of this bottom shelf would then be more specific shelves such as Regression and Time Series. The structure will allow the higher shelves to call on the lower shelves and they in turn may be used by other shelves. Given the development of a 'good' general framework it is hoped that the library will naturally grow to contain even thousands of functions. There will be no bar to any area of statistics being added to the library. Figure 1 indicates the structure of the library and how it might interact.

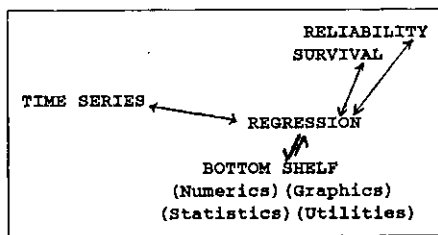


Figure 1 - An Overview of the library structure.

The connections are current illustrations of what may occur.

Implementing ASL

Having described the overview of the structure there is a need to describe in more detail the implementation of the system. Since the desire is to produce a coherent set of functions there is a need to produce a fairly rigorous control mechanism. For in addition to the

problem arising from the size of the code to be developed, those contributing to the project are working for a number of institutions and organisations, and it is possible that they will be working with a range of interpreters. There are three major parts to the implementation which can be grossly described as 'who', 'what' and 'how'.

The prime effort for the project is going to come from academic statisticians working in universities and polytechnics, with the assistance of individuals in the private and public sectors. Most of this effort is voluntarily given and represents a heavy investment by the individuals involved. To ensure that this effort is not wasted there is a need for some central body to organise the library.

At present this body consists of the ASL Team, established at ASL Workshop, see Appendix B. The ASL Team is composed of a chair (Tony O'Hagan), deputy chair (Alan Sykes), manager (Jake Ansell), British APL Association representatives (John Searle and David Eastwood) and there may in the future be other co-opted members. It has the role of attempting to organise the UK APL Statistics Community to assist with the project. Currently the community is supportive of the concept.

Since the Team cannot substitute itself for the community and there is a need for fairly rapid development/decisions, it has been accepted that working parties will play a significant role in the production of concepts, ideas and code. Figure 2 illustrates the relationships within the project.

The ASL Team role is to establish policy concerning the library. It has overall responsibility for the project, ensuring that the manager implements the policy. The team will be convened when necessary by the chair. The day to day management of the project is in the hands of the manager. The manager has the responsibility for production of the software, though it is not perceived that the manager will either write, produce specifications or designs for the code. A programmer paid for by British APL Association will write the code under the direction of the manager. The working parties will generally have responsibility for design and specification. The manager thus acts as a communication channel between the working parties, the programmer and the chair, and hence the Team.

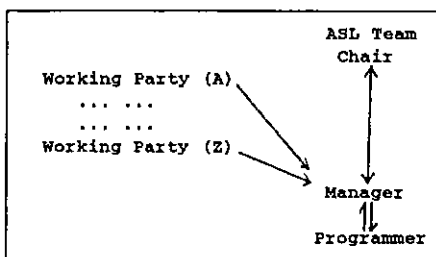


Figure 2 - The Project Personnel

The real success of the project therefore rests with the working parties. For it is their efforts which will define the success of the project. If the project rested solely on the Team it would take far too long to be practically useful. Fortunately most of the code for ASL already exists in some form, and will therefore only need to be organised according to the development standards.

The contents of the library have already been partially considered. Ultimately the desire is to build a statistical library covering the wide spectrum of statistics, and which can be continually updated. However, this is into the more distant future. At the present time with the current resources of the project, and given the so far unestablished track record of the project, the initial goals have to be more limited.

The contents of the library have already been partially considered. Ultimately the desire is to build a statistical library covering the wide spectrum of statistics, and which can be continually updated. However, this is into the more distant future. At the present time with the current resources of the project, and given the so far unestablished track record of the project, the initial goals have to be more limited.

It is therefore envisaged that an 'initial' bottom shelf will be built and fully tested within the first 6 months. More detailed comments arising out of the Gregynog Workshop about the 'bottom shelf' are given in Appendix A. The next stage will be the development of the regression shelf; Appendix C indicates the expected content of this shelf. These objectives are regarded as realistic within the year. More importantly the development of these will have set standards which will apply across the library. With the initial headaches resolved it is expected that other shelves will be added more rapidly.

Testing and quality will play a central role in the whole project. Hence it is expected that the mechanisms for testing will also be implemented at this early stage. A prerequisite to building any of the above will be the availability of a

structure. A framework for the library has already been outlined. The structure of the library will be first priority. It is expected that ASL will follow a structure similar to one of the larger APL implementations such as the British Airways Program Filing System (PFS).

The library will contain functions not in workspaces but in component form in a file system. This will enable rapid implementation of functions as they are brought into the workspace. A linking structure will also have to be developed. Specification of such a structure will be established by the working party on standards.

The Standards group is expected to play a major role in the early stages of the project. Senior APL developers have agreed to be involved with this work. Not only will they be recommending the structure but they will attempt the near impossible task of describing 'good' coding conventions in APL. Some of these have already developed from contributions from Adrian Smith (Rowntrees) and Maurice Jordan (British Airways) at the ASL Workshop. An outline for the working party on standards to work from was also produced at the ASL workshop and is presented in Appendix D.

With an established structure and a set of standards it is hoped that many problems will disappear. The subsequent working parties should be able to develop specifications within the project standards. How successful this is will be quickly established since the bottom shelf and graphics working parties will be working almost in parallel with the standards group.

The Product

Having given details of need, the framework and the implementation, this last section describes the product and some of the important related issues. Firstly it is hoped that we can never speak of the final product, for we expect that it will continue to develop almost organically on its well laid out framework. The product itself will consist of a library of statistical functions, which incorporates some basic facilities for display including graphics. It will not be a package, and there will be no attempt to develop sophisticated front ends. Commercial concerns may consider the benefit of producing such under licence to the ASL project.

The library will appear in a number of forms. It is hoped it will be stored primarily in a published form, either as one manual plus other texts or a set of volumes relating to developed shelves. Hopefully these texts will set the standard for statistical computation in APL. It is also desired that it should be maintained in machine-readable form on disks by the British APL Association and possibly by the University of Warwick on their IBM mainframe. The

software will be freely available with only a nominal charge to cover administration and materials.

Beyond the core texts it is hoped that for both non-APL research workers and teachers there will be tutorial texts. These texts should be written to explain how ASL might be used for common statistical analyses. If these books are well fashioned and written, the power and elegance of analyses should tempt the reader to learn APL. These should attract the desired wider audience for ASL.

Since there is a desire to be independent of commercial interests the choice of interpreter has been difficult. It was finally resolved that whilst ISO APL was the only generally agreed standard, there was at least the need in statistics to take advantage of nested arrays. Therefore it was accepted ASL should be written in the intersection of as many second generation APL interpreters as possible. The standards group would attempt to describe this object in the short term before a more general standard was available. The argument which was most substantial in favour of this move was that there was a need to look forward into the future market of the product. It was however felt that where ISO APL could be sensibly used this should be retained as the coding medium. It is also the intention where possible to provide support for as large a range as possible of interpreters.

APL 90

The first test of the success of this project will be the paper presented at the conference. If the paper is still speculation then we have failed. It is hoped that by the conference we will be delivering some part of the project in a tangible form. Look forward to our being proved correct!

References

- Anscombe, FJ, [1981], *'Computing in statistical science through APL'*, Springer, New York.
- Grenander, [1982], *'Mathematical experiments on the computer'*, Academic Press, New York.
- O'Hagan, A, [1990], 'WAGS', to appear.
- Thomson, N, [1989], *'The Bottom Shelf'*, paper presented at ASL Workshop, Gregynog, Newton, Wales.

Appendix A: The Bottom Shelf/Graphics

1. The bottom shelf would contain initially a set of Statistics and Graphics function with sufficient associated functions to ensure the shelf/library is usable.
2. The Statistics functions would be sufficient to cover the material in an introductory course to statistics at UK 'A' level and first year undergraduate standard. This will include functions described in Norman Thomson's 'The Bottom Shelf' and enhanced by Exploratory Data Analysis functions.
3. Other statistical functions which appear in more than one of the higher shelves will also be added to the bottom shelf subsequently.
4. The graphics functions will be split into two parts: WAGS (Warwick APL Graphics System) basic graphical routines, O'Hagan [1990]; and routines to produce standard statistical graphics for material in 2 above.
5. There will be a need for Numerical Analytic functions as well as Data Manipulation and general Utilities. Some of have already appeared in Thomson's 'The Bottom Shelf'.
6. By necessity alternative versions of some of the functions will be required. It will be the responsibility of those managing the project to resolve any problems arising in following the good practice evolved by the standards group.

Appendix B: Management Workshop

1. ASL should be independent of commercial interests, and the ASL functions should be made freely available with only a nominal charge to cover administration and materials.
2. Sponsorship of the initial development would be on the basis of neutrality. Only for the development of specialist packaged products would sponsorship be sought. For example sponsorship might be sought for APL2 developments of ASL.
3. The ASL is expected to be a most useful resource for statisticians already familiar with APL. 'Packaging' ASL with a user friendly interface would not be useful to such an audience, it may even be an impediment to realising the power and flexibility of APL. Reaching a wider audience, unfamiliar with APL, will require a tutorial text explaining how to use ASL for common statistical analyses. If the book is well written then the power and elegance of the analyses should tempt the reader to learn APL.

4. Beyond the audience of research workers and teachers there will be a need to produce more user-friendly front ends and eye-catching graphics. The production of such is not perceived as part of the ASL project. It is felt that commercial exploitation of the ASL code would be left to companies under licences from the British APL Association. Monies accruing from this activity would be used to continue the development of ASL.
5. To ensure neutrality it was recommended that ISO APL should be adopted as the standard language for the project.
6. Specification of code would be through working parties of 2 or 3 people with a single author of the specification.
7. There would be no geographic barrier to involvement in the project.

Appendix C: Regression Shelf

1. The first priority is a simple functioning system. This system may include Generalised Linear Models, but will not be over sophisticated.
2. This system should have graphic diagnostics (see graphics shelf), diagnostic statistics and basic regression statistics as far as ANOVA. Presentation of results would be limited.
3. In constructing the system some thought should be given to non-standard approaches, (Non-parametric, Ridge, Robust Regression), and to non-standard diagnostic techniques. It is doubtful though these would be included in the first release.

A survey of APL statisticians would be carried out to find their priorities and for donation of appropriate code.

4. The need for Stepwise Regression was discussed and it was accepted that it had a high priority though no best method arose in discussion. Again a survey of views was thought to be helpful on the issue.
5. Some guidance on the workspaces from the standards group would be useful including comments on full-screen editors. Some discussion on housekeeping suggested global variables needed controlling.
6. The audience was initially accepted to be the knowledgeable user with front ends later on for those with lower level of expertise, probably within a higher shelf.

Appendix D: Good APL Code/Practice

1. 2nd Generation APL was chosen since it reflects the future market.
2. Data Formats: There was a need to specify in-house standards. Given that the main target of the bottom shelf would be users of APL in industry and academia, clearly laid down documentation would suffice. Simple arrays should be used where possible. There should be an attempt to anticipate the data formats which would be appropriate for macros when a number of functions are glued together to provide ASL idiom.
3. Controlling the mess: The effects of 'mess' are controlled by discipline, which arises from management and design. Global or semi-global variables should be tolerated only if they do statistical jobs and should be documented in the workspace. Nested arrays will remove the need for some global variables.
4. Error Trapping: For bottom shelf functions, errors should result only from misuse (using parameters beyond those specified). Hence the library should cut back stack by one and signal the appropriate error message.
5. Naming convention: Functions should be named to imply their source, *AS_* or *ASΔ*.
6. Index origin: Always assumed to be 1 though there is no reason why a function should not have it otherwise if *IO* is local.
7. Direct definition: This should not be used.
8. APL variants: Quad or system functions should be avoided where possible. It should also be noted that there are different second generation interpretations of APL primitives.
9. Bolt-on-statistics: Where fast functions/utilities exists outside APL they can be used but APL equivalent should also be provided.
10. Data inputs/exports: There should be facilities to read rectangular databases from ASCII and LOTUS123 files into APL.
11. Documentation: Should be good and robust. A technical manager should be responsible for the initial draft. A professional technical writer might subsequently be employed. All functions need a standard documentation which covers: arguments, source algorithm, last edited date, obscure lines and user documentation to indicate the range of valid arguments. HELP facilities are regarded as part of the documentation and give examples to be used as part of the test-harness.
12. Distribution of the software: ASL will need a variety of high level library management and information tools to ensure that the software can be transferred in a user-friendly mode.

Some Questions of Programming Style for the APL Statistical Library

by David Appleton

Introduction

I should like to discuss some issues of programming style in the context of plans to produce an APL Statistical Library [1]. I shall consider for the sake of example a user who wishes to investigate some method of multivariate analysis and needs to produce simulated data matrices whose columns are standard Normal variates from a population with a known correlation matrix. Briefly, what he needs are independent Normal variates and a method of factorising the correlation matrix into the form AA^T . For my own purposes I usually use the three functions *UNIF*, *BOXMUL* and *CHOLESKY* as shown below:

▽ *Z*←*UNIF* *N*

[1] *Result* is independent $U(0,1)$ variates in a vector or higher dimensional array whose shape is given by the right-hand parameter.

[2] *Z*←(?*Np*2147483647)÷2147483647

▽

▽ *Z*←*BOXMUL* *N*; *M*

[1] *Result* is independent $N(0,1)$ variates in a vector or higher dimensional array whose shape is given by the right-hand parameter.

[2] *M*←[0.5××/, *N*

[3] *Z*←*Np*((2, *M*) ρ ⁻²×**UNIF* *M*)×0.5)×1 2×.002×**UNIF* *M*

▽

▽ *L*←*CHOLESKY* *M*; *P1*; *P2*; *J*; *N*

[1] *Result* is a lower triangular matrix *L* such that $L+.*\phi L$ is *M*, the symmetric positive-definite matrix given as the right-hand parameter.

[2] *L*←(*pM*) ρ 0 $\diamond$ *N*←1 $\uparrow$ *pZ* $\diamond$ *J*←1

[3] *NEWCOL*: *P1*←*J*+ ι *N*-*J* $\diamond$ *P2*← ι *J*-1

[4] *L*[*J*; *J*]←(*M*[*J*; *J*] - +/ *L*[*J*; *P2*] × *L*[*J*; *P2*]) × 0.5

[5] *L*[*P1*; *J*]←(*M*[*P1*; *J*] - +/ *L*[*P1*; *P2*] × ((*N*-*J*), (*J*-1)) ρ *L*[*J*; *P2*]) ÷ *L*[*J*; *J*]

[6] ←(*N*≥*J*+1)/*NEWCOL*

▽

If *CORM* is the $p \times p$ population correlation matrix, then the simulated data matrix (of shape $n \times p$) is given by:

ϕ (*CHOLESKY* *CORM*) +.* *BOXMUL* *p*, *n*

... or a function *DATAMAT* may be written, such as:

```

▽ Z-CORM DATAMAT N
[1] A Result is a simulated data matrix of  $N(0,1)$  variates from a population
whose correlation matrix is given by the left-hand parameter and the
number of whose rows is given by the right-hand parameter.
[2] Z ← (CHOLSKY CORM) + . * BOXMUL(1 ↑ pCORM), N
▽

```

The theory behind this can be obtained from Newman & Odell [2], though there is a misprint in the formula and they refer to the factorisation as Crout's method, which is incorrect [3].

The Problem

A user may find this entirely satisfactory, but he has perhaps unwittingly been forced to accept the following:

- (i) The Lewis, Goodman and Miller [4] random number generator

$$x_{n+1} = 16807 x_n \pmod{2^{31}-1}$$

when, for the sake of compatibility with other programs, he may prefer the NAG [5] generator

$$x_{n+1} = 1313 x_n \pmod{2^{59}}$$

- (ii) The Box-Müller [6] method of transforming to standard Normal deviates (with a slight peculiarity of which he may be unaware, and which we shall discuss later)
- (iii) Cholesky decomposition of the correlation matrix, when he may prefer, or indeed have available, the spectral decomposition, that is he may know the eigenvalues and eigenvectors of the population correlation matrix.

In other words, this programming style, where one function internally calls another, reduces the user's choice and may not be appropriate for library functions. For let us now suppose that the user wishes to adapt the APL random number generator, either by shuffling the sequence in some way, or by using the sum, modulo unity, of two of its uniform variates, or taking some other precaution against some possible imperfections in it. He may therefore wish to use *UNIF2*, and instead of Box-Müller he may wish to use *HASTINGS* which programs a Hastings [7] approximation to the inverse of the Normal distribution function, a procedure which preserves the ranking of the original random number generator:

▽ Z←UNIF2 N

[1] A Result is independent $U(0,1)$ variates in a vector or higher dimensional array whose shape is given by the right-hand parameter.

[2] Z←1|(UNIF N)+(UNIF N)

▽

▽ Z←HASTINGS N;X;H

[1] A Result is independent approximate $N(0,1)$ variates in a vector or higher dimensional array whose shape is given by the right-hand parameter.

[2] H←0.5<X←UNIF2 N

[3] Z←(2*H|X-H)*0.5

[4] X←(2.515517+Z*0.802853+Z*0.010328)+1+Z*1.432788+Z*0.189269+Z*0.001308

[5] Z←(1-2*H)*X-Z

▽

He is entitled to believe that, trusting his uniform variates, he should transform them as simply as possible to obtain his Normal variates. We now have a new version of *DATAMAT*: *DATAMAT2*. But this is only one of the combinations which may be chosen by the user, and each possibility results in another *DATAMAT* function.

▽ Z←CORM DATAMAT2 N

[1] A Result is a simulated data matrix of $N(0,1)$ variates from a population whose correlation matrix is given by the left-hand parameter and the number of whose rows is given by the right-hand parameter.

[2] Z←(CHOLESKY CORM)+.*HASTINGS(1↑pCORM),N

▽

The 'peculiarity' of *BOXMUL* referred to is that the uniform variates whose sine and cosine are calculated are treated in reverse order to those which are logarithmically transformed. This is an attempt to avoid the phenomenon described by Neave [8], but a full justification for it has never been carried out. Notice that the use of both *sine* and *cosine* is substantially more efficient than the use only of *cosine* [9], a procedure requiring the generation of twice as many uniform variates and the calculation of twice as many square roots.

Possible Solutions

Apart from providing all of the possible combinations, there appear to be two other solutions, neither of which is wholly satisfactory. Firstly, we can prevent functions from calling other functions and merely pass parameters. For example, *HASTINGS* would not act on the shape of the result required but on the rectangular variates; but this is a little less straightforward for the Box-Müller method, because if an odd number of Normal deviates is required an extra

uniform variate must be used, and for acceptance-rejection methods the idea is rather absurd, although if the rejection rate is known it is possible to program such methods efficiently in APL [10].

An alternative is to pass function names. This is more convenient in a second generation APL, and specimen functions are shown below:

∇ Z←UFN ΔBOXMUL N;M

[1] ∇ Result is independent $N(0,1)$ variates in a vector or higher dimensional array whose shape is given by the right-hand parameter; the $U(0,1)$ variates required are supplied by the function whose name is given as the left-hand parameter.

[2] M←⌈0.5××/N

[3] UFN←UFN, ' ', *M

[4] Z←Np((2,M)p(-2×*UFN)+0.5)× 1 2 0.002×*UFN

∇

∇ Z←UFN ΔHASTINGS N;X;H

[1] ∇ Result is independent approximate $N(0,1)$ variates in a vector or higher dimensional array whose shape is given by the right-hand parameter; the $U(0,1)$ variates required are supplied by the function whose name is given as the left-hand parameter.

[2] H←0.5<X←*UFN, ' ', *N

[3] Z←(-2×*|X-H)×0.5

[4] X←(2.515517+Z×0.802853+Z×0.010328)+1+Z×1.432788+Z×0.189269+Z×0.001308

[5] Z←(1-2×H)×X-Z

∇

∇ Z←FNS ΔDATAMAT NCORM;TXT1;TXT2

[1] ∇ Result is a simulated data matrix of correlated $N(0,1)$ variates; the right-hand parameter is a vector consisting of the number of rows required and the correlation matrix; the left-hand parameter is a vector of vectors containing the names respectively of the $U(0,1)$ generator, the method of transforming to $N(0,1)$ variates, and the method of factorising the correlation matrix.

[2] TXT1←(∇FNS[3]), ' ∇NCORM[2]'

[3] TXT2←'(∇FNS[1]) ', (∇FNS[2]), ' ', *(1↑pNCORM[2]), NCORM[1]

[4] Z←q(ΔTXT1)+.×(ΔTXT2)

∇

With such functions and a 5×5 correlation matrix, the call:

'UNIF' 'ΔBOXMUL' 'CHOLESKY' ΔDATAMAT 100 NCORM

... gives 100 rows of the required data matrix, using the basic random number generator, Box-Müller, and Cholesky; whereas:

```
'UNIF2' 'ΔHASTINGS' 'CHOLESKY' ΔDATAMAT 100 NCORM
```

... uses the alternative random number generator and method of transformation. However the user has been given his choice of methods at some cost, both to himself in terms of what he is required to know and do, and to the programmer because the simplicity of *DATAMAT* as we first encountered it has disappeared entirely.

More importantly, this procedure may be highly inefficient because many successive uses of *ΔDATAMAT* may require the same factorisation of the same correlation matrix, or the user may wish to retain the original uniform variates for successive runs with a view to investigating some form of variance reduction technique [11].

Discussion

The point is that we cannot cater for every conceivable, or even likely, use of such a fundamental system as a statistical library, but if we are to cater for as many as possible, as simply as possible, some difficult decisions have to be made about the style of functions we need.

I have little doubt that in many cases we shall have functions like *CHOLESKY* where it is fairly clear that a single parameter is operated on and a single result given - though that is to ignore the important question of whether another result indicating that the data are appropriate (e.g. that the matrix is symmetric) and that the calculations have been satisfactorily completed (e.g. that the matrix is positive definite) should also be provided.

However, I am equally sure that to give the potential users control over their analyses we must provide many different functions which carry out essentially the same operation. Of course the decision must be based on who the potential users of the system are. Some may not wish to become involved in details and will require simple-to-use prescriptive functions, but if the APL Statistical Library is to be a unique contribution it must not simply duplicate the NAG library in an APL environment; it must cater for the statistician who wishes to construct new methodologies, and it must therefore provide him with the wherewithal to do so.

Dr. D.R. Appleton,
Division of Medical Statistics, Medical School,
University of Newcastle upon Tyne, NE3 3BU,
England.

References

- [1] O'Hagan, A. (1990) *ASL after Gregynog*. Vector 6 (3) 63-65.
- [2] Newman, TG & Odell, PL (1971) *The generation of random variates*. Griffin, London, 38-39.
- [3] Ralston, A & Rabinowitz, P. (1965) *A first course in numerical analysis*. (2nd edition) McGraw-Hill, New York, 421-425.
- [4] Lewis, PA, Goodman, AS & Miller, JM (1969) *A pseudo-random number generator for the System/360*. IBM Systems Journal 8 136-146.
- [5] NAGFLIB:1443/0:Mk6:May77.
- [6] Box, GEP & Müller, ME (1958) *A note on the generation of random normal deviates*. Annals of Mathematical Statistics 29 610-611.
- [7] Abramowitz, M & Stegun, IA (eds) (1964) *Handbook of Mathematical Functions*. National Bureau of Standards, Washington, DC.
- [8] Neave, HR (1973) *On using the Box-Müller transformation with multiplicative congruential pseudo-random generators*. Applied Statistics 22 92-97.
- [9] Thomson, N. (1989) *APL programs for the mathematics classroom*. Springer-Verlag, New York, 106-107.
- [10] Appleton, DR. (1976) *Discussion on paper by Atkinson, AC & Pearce MC "The computer generation of beta, gamma and normal random variables"*. Journal of the Royal Statistical Society A 139 449-451.
- [11] Kleijnen, JPC. (1974) *Statistical techniques in simulation*. Dekker, New York, 105-239.

Designing an APL Typeface Using a Partial Implementation of POSTSCRIPT in APL

by Adrian Smith (Rowntree Mackintosh)

Abstract

As well as a unique character set, APL has in the past been distinguished by a unique typeface: the 2741 golfball. This consistency of typography is in danger of being lost as more and more APL typefaces appear on the market. These vary from quite acceptable (e.g. the Laserjet fonts from STSC) to absolutely awful (most of the Mac fonts I have seen). The typographic language of the future is clearly PostScript; a good PostScript version of APL's original typeface is clearly a must if we are to retain the visual identity of APL code.

In order to design a typeface, you need to show it on the screen, along with its bounding box and various guide-lines. In the case of APL you also need to check that the compound symbols register correctly when overstruck. Fortunately the syntax of PostScript is sufficiently APL-like that it is quite easy to write a reasonable subset of PostScript in APL. This can then be used to draw the characters on the screen (scaled as required) and then to write the completed definition to disk for processing by a genuine PostScript device.

The completed typeface is best described as an original design in the style of the 2741 golfball. It includes all the APL*PLUS/PC characters, all the APL2 characters, the PC line-drawing characters, and any others I have come across (e.g. the quad-arrow combinations from APL68000). It is scalable to any required size, and conforms to the font metrics of courier-oblique. The character encoding is APL*PLUS/PC by default, but can easily be changed to suit any requirements. It is available to any APL or publishing organisation which can make use of it.

Introduction:

Why is a Raven like a Writing Desk?

Just what is this thing called PostScript? Most people will have heard of it, but I suspect that few of us APLers know what it actually is. The common view is that it is some kind of printer-control protocol (a bit like the \015 that you send to your Epson to put it into compressed mode), but of course much more powerful

and even harder to understand. This view is only confirmed if you make a superficial examination of the kind of junk that the average word-processor generates when you send your document to a PostScript printer. It looks a bit like this:

```
1296 12448 P 0 12 F 26 15 F B (Introduction:) S E
1296 12768 P B (Why) S 75 J ( is) S 75 J ( a) S 75 J ( Raven) S 75 J (
like) S 75 J ( a) S 75 J ( Writing) S 75 J ( Desk?) S E
```

... you might just make out the subheading at the top of the preceding paragraph! This all serves to conceal the fact that PostScript is actually a very powerful, general-purpose programming language, which just happens to be rather good at handling text. When I first got interested enough to go out and buy the two standard works on the subject [1] and [2] I began to have a strange sense of déjà vu:

- see for instance page 77 of the Cookbook: "PostScript arrays are one-dimensional collections of objects. PostScript arrays are different from those in other languages in that their elements need not all be of the same type. That is, a single array may contain, for example, strings, integers, dictionaries, and other arrays." There is even a *forall* operator to process each element of an array without the need for an explicit loop!
- OK, so here we have a general-purpose programming language with nested arrays. I can cope with that ... what about its attitude to functions and variables? We find on page 28: "A variable is defined by placing the variable's name and value into the current dictionary. This is done with the *def* operator, as in */ppi 72 def*." In other words *PPI* ← 72, without any fuss or bother. Functions are defined in a very similar way: */inch {72 mul} def* is the equivalent of *INCH : 72 × w* in I-APL's direct definition.
- a couple of other parallels struck me immediately. The first was the use of arrays to manipulate text on the page. The reference manual (page 64 onward) spends some time explaining the use of transformation matrices in scaling and rotating text. The second is the refreshing lack of any operator precedence. Being stack-based, PostScript works backwards, but at least it is consistent about it, and an APLer can quickly get into the habit of thinking left to right rather than right to left!

I hope that gives you enough background to appreciate that the idea of implementing a reasonable subset of PostScript in APL*PLUS/PC is not only practical, but actually rather simple! Before I go on to explain how the workspace is put together, I feel I should add a little more background on why I felt the need to bother.

A Short History of APL Typography

Once upon a time, long ago and far away, a little old man worked long hours alone in his little old house, chiselling away at a strangely shaped piece of bright metal. As he cut away, he uttered incantations like "□ goes near the equator 'cos it's fat; | is nice and slim ... further North for him ". Little did he know that centuries after his death, his work would be rediscovered and used as the basis of a worldwide revolution in mathematical notation.

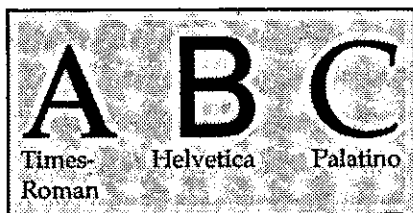
The 2741-golfball, for such it was that the little old man had accidentally created all those years ago, formed the basis for APL typography for at least the first ten years of the language's existence. It is a brilliant piece of design, fitting as it does a variety of symbols of differing widths into a standard 10-chars-per-inch Selectric typewriter. It permits endless combinations of the basic symbol set (overstrikes) to allow the notation to be extended almost without limit. It also looks really good on the page!

Since those early days of APL, we have been treated to an ever greater range of display and output devices, and (with a few honourable exceptions) APL typography has gone rapidly downhill. As editor of Vector, I have had to live with Xerox's attempt (APL-A10) at a font for their 3700 laser printer. Much of it is quite acceptable, but the 1 looks like some combination of a backwards Upstile overstruck with a downstile! [3]. I also get to see the originals of submitted papers, often on lovely HP-laserjet with the APL bits written in by hand. Alternatively it will be dot-matrix with STSC's T=3 graphics output to an Epson printer.

In short, we have almost lost our visual identity, and in consequence we are finding it harder and harder to underline the difference between APL (which I still believe is first and foremost a notation) and all the other fancy things you can do with computers these days. I decided that I would like to see two things happen:

- someone should recreate the original 2741 font (extended to include lower-case and the full ASCII set) in PostScript, and make it available to the APL community.
- the APL group on the ISO should take up APL typography as an important issue, and should recommend a standard APL typeface for use in all APL-related publications. It would also help the typeface designer if there were standard names for all the APL symbols!

My hope is that this will once again give APL a clear visual identity; 10 years' experience in the confectionery industry tells me that this can only be beneficial when it comes to marketing the product out there in the real world.

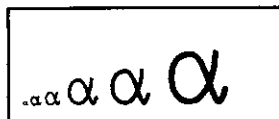


How to Build a Typeface

Just what goes into making an 'A'? There are really two ways of doing it: the first option is to set out a grid of zeros and ones, with the ones representing the dark spots, and the zeros the light spots. This lets you define any character at any size, as long as you have enough patience to type in all those zeros and ones! For the old 9-pin FX-80 you work with 8 rows of 11 columns, and the process is fairly painless. The results, however, are only just acceptable! The second option is to define your character as a series of vectors, which may enclose areas to be shaded in. This is what PostScript refers to as a scalable outline font, and is how all the standard typefaces are defined (See box).

Each method has its advantages and drawbacks. The main benefit of the binary image method is that you have exact control over every dot; on a decent printer such as the LQ-500 (a typical 24-pin device) the results can be really excellent [4]. The drawback is the inordinate length of time needed to create a complete font at this kind of resolution.

Of course you might create your dot-font by scanning in text printed on some other device. For some time, Ed Cherlin [5] has been using a PostScript image font which was scanned from an old 2741! This is again quite effective, but of course it faithfully reproduces all the errors in the original rendition! For example in his α , the $\circ$ and the n are ever so slightly offset.



The scalable outline (as its name suggests) can be scaled to any size, and written in any direction. However it does leave you open to rounding errors when written too small on a low-resolution device (see box). In spite of this minor disadvantage, I felt

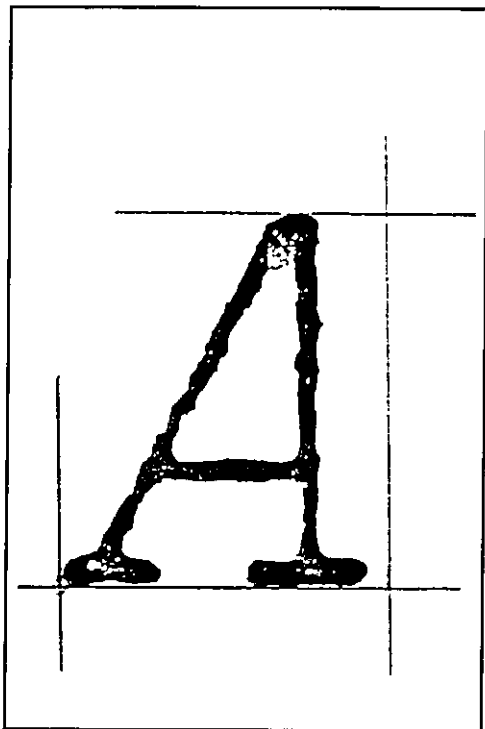
that the benefits of being able to set my APL any size I liked far outweighed the slight loss of control over small characters.

Fortunately, when I tracked down an old listing and put it under the low-power microscope, I realised that the 2741-golfball had simply been engraved with a fixed width tool, in other words the font could be built from a series of line

segments of fixed width (what PostScript calls a *stroked font*) with only a very few characters (such as ', ') requiring filled outlines. This is just like the *Courier-Oblique* font which all PostScript printers already provide, so I had somewhere to start from.

The next stage was to decide on a co-ordinate system, and to plot out the vectors needed for each character on this grid. Because I wanted to match 12-point Courier (one point = $1/72$ of an inch) I needed to get 6 chars to the inch vertically and 10 to the inch horizontally at this size. The most convenient grid turned out to be an x-range of 0-720 (with the widest character occupying 0-600), and a y-range of 0-1200 (with descenders down to about -100 and the top of the $\square$ at 700). What I now needed was a complete print of the original font 7cm high, preferably

on centimetre graph-paper! Several hours' work with the photocopier on 156% produced a set of samples like the one on the right, and all that was left was to measure up and plot out the resulting vectors.



Where APL came into it

PostScript has one very serious drawback: it expects to find a printer as its output device. This means that in order to test out your creations, you really have no alternative but to print them out. I very rapidly found that I needed to see the characters on the screen (preferably overlaid with some xy-axes) to check how they looked. This was particularly vital for characters which would need to be overstruck, such as $\square$ and $|$, or the lower-case characters *abc* which I had to make up as I went along.

This led me to wonder just how hard it would be to write something in APL which would take a simple PostScript fragment and draw it on the screen. As an

example, here is the PostScript for A, and the stages needed to generate the APL code to draw it on the screen (in APL*PLUS/PC):

```
/A
{ % PostScript code to draw APL A on 720 by 1200 grid
  % (assumes all lines are 40 units thick)
  20 20 moveto 150 0 rlineto
  370 20 moveto 188 0 rlineto
  70 20 moveto 420 680 lineto
  40 0 rlineto 470 20 lineto
  175 215 moveto 470 215 lineto
  stroke
} def
```

a as an APL character vector

```
20 20 MOVETO ◊ 150 0 RLINETO
370 20 MOVETO ◊ 188 0 RLINETO
70 20 MOVETO ◊ 420 680 LINETO
40 0 RLINETO ◊ 470 20 LINETO
175 215 MOVETO ◊ 470 215 LINETO
STROKE
```

a rebuilt as APL function

```
△△△△
20 20 MOVETO ◊
150 0 RLINETO ◊
370 20 MOVETO ◊
188 0 RLINETO ◊
70 20 MOVETO ◊
420 680 LINETO ◊
40 0 RLINETO ◊
470 20 LINETO ◊
175 215 MOVETO ◊
470 215 LINETO ◊
STROKE ◊
```

I worked with character vectors (called simply 'A', 'ALPHA' etc.), and (being lazy) I used ◊ to separate the commands, rather than taking the trouble to split the lines intelligently. Because you can't define an APL function with a left argument only, all my drawing routines were of the form ∇ ARG FOO DUM, with no explicit results. Hence the patch of ◊ to the right of all the functions before execution. The PostScript philosophy is to leave the paper blank until a *stroke* or *fill* operator is encountered, so I did the same and simply built a 3-column numeric matrix of x/y pairs and a boolean to indicate 'pen-up' or 'pen down'. *STROKE* then writes this array to the screen, and clears it down having done so.

```
 $\nabla$  NEWPATH DUM
[1] a Set up an empty path. Cols are [x,y,0/1]
[2] PATH← 0 3 p0
 $\nabla$ 
```

```

      ▽ X LINETO DUM
[1]  ⚡ Add drawn segment to path.
[2]  PATH←PATH; (SCALE×OFF MOVE 2ρX),1
      ▽

      ▽ STROKE DUM;SEG;LEN
[1]  ⚡ Draw line segments in PATH, and clear it.
[2]  Loop:LEN←(1+PATH[;3])⋮0
[3]  SEG←(LEN,2)⋮PATH
[4]  GSΔDRAW SEG
[5]  PATH←(LEN,0)⋮PATH ⋮ +(1+ρPATH)⋮Loop
      ▽

```

All I now required was a simple driver function to draw the grid and execute the transformed pseudo-PostScript as an APL function:

```

      ▽ RES←DO LTR;PATH;OFF;SCALE
[1]  ⚡ Test out Postscript Letter
[2]  GSΔINIT ΔGTYPE ⋮ GSΔWIN 1000 XYΔMAX GSΔQPS
[3]  OFF← 0 250 ⋮ SCALE←0.95 ⋮ NEWPATH ⋮
[4]  GSΔUSE 'DOT,CYAN'
[5]  GSΔDRAW BOX SCALE×OFF MOVE 0 250 600 700
[6]  GSΔDRAW SCALE×OFF MOVE 0 0 600 0
[7]  GSΔDRAW SCALE×OFF MOVE 0 530 600 530
[8]  GSΔDRAW SCALE×OFF MOVE 325 0 325 700
[9]  GSΔDRAW SCALE×(OFF+ 238 0)MOVE 0 0 175 700
[10] GSΔUSE 'SOLID,YELLOW'
[11] ΔEX LTR
[12] RES←ΔPFHOLD ⋮ +(RES=6)⋮Exit ⋮ GSΔCOPY
[13] Exit:GSΔTERM ⋮ RES←'Ready'
      ▽

      ▽ ΔEX MAT;ΔΔΔΔ
[1]  ⚡ Execute tidied up MAT
[2]  MAT[(MAT=⊂TCNL)/⋮ρMAT]←'⋮'
[3]  MAT←'⋮' ΔTAB MAT
[4]  ⋮⊂FX 'ΔΔΔΔ' CAT MAT,'⋮'
      ▽

```

To go the other way and generate the PostScript code is really just a matter of eliminating the diamonds, adding some {}, changing the comments to % signs, and making everything else lower-case:


```

∇ RES+PS CHAR;MAT;LDC;POS;STRK;NPTH
[1]  R Wrap up char in PostScript definition
[2]  R All linedrawing chars get SQUARE ends.
[3]  RES+' ' ∘ POS+ΔAV[;1];CHAR ∘ →(POS>1+ρΔAV)+0
[4]  LDC+CHAR∉[AV[179+;45]
[5]  CHAR+7+ΔAV[POS;] ∘ RES+'/',CHAR-' ' ∘ →(2*∅NC CHAR)+Ndef
[6]  CHAR+*CHAR
[7]  MAT+DTR ∅TCNL ΔTAB ΔNEATEN CHAR-' ∘'
[8]  RES+RES VCAT ' { '
[9]  R Prefix with line width and style for square chars ...
[10] RES+RES,LDC/'0 setlinecap 0 setlinejoin'
[11] RES+RES VCAT VPACK(((1+ρMAT),3)ρ' '),ΔLJUST ΔLOWER MAT
[12] RES+RES VCAT ' } def'
[13] RES+RES VCAT ' ' ∘ →0
[14] Ndef:RES+RES,' {} def'
[15] RES+RES VCAT ' '
∇

```

R Part of ΔAV, which records the character names and
 R encoding position

```

A 65 /A
B 66 /B
α 224 /ALPHA

```

```

∇ R+VPACK OBJ
[1]  R Pack matrix with CR on the end.
[2]  R+OBJ ∘ →(2>ρρOBJ)+0
[3]  R+~1+((ϕ∇\ϕOBJ≠' ').1)/,OBJ,∅TCNL
∇

```

```

∇ R+A VCAT B
[1]  R+A,((∅TCNL*~1+A)/∅TCNL),B
∇

```

... and the entire font can be built with the addition of a prefix, an encoding vector, and a suffix:

```

%% Font definition header text for APL
/newfont 12 dict def
newfont begin

/FontType 3 def          % Required elements of font
/FontMatrix [1 1200 div 0 0 1 1200 div 0 0] def
/FontBBox [0 -250 720 950] def
/Encoding 256 array def
0 1 255 {Encoding exch /.notdef put} for

% ..... only a small part of the encoding is shown
Encoding 33 /QUOTEDOT put
Encoding 65 /A put
Encoding 66 /B put
Encoding 90 /Z put
Encoding 133 /RIGHTARROW put
Encoding 159 /FROWN put
Encoding 166 /LAMP put
Encoding 224 /ALPHA put
% .....
Encoding 248 /JOT put
Encoding 249 /CIRCLE put
Encoding 250 /OR put
Encoding 251 /RHO put
Encoding 252 /DOWNSHOE put
Encoding 253 /HIGHBAR put
Encoding 254 /STILE put
%.....
% all the character definitions go in here ...
%.....
/CharProcs 216 dict def      % for chars
CharProcs begin
/.notdef {} def

/LEFTPAREN
{
    420 -120 moveto
    180 0 180 580 420 700 curveto
    stroke
} def

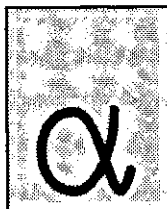
% and so on .....

/BuildChar                  % Stack: font char
{ 720 0                    % char width
    0 -250 720 950         % bounding box
    setcachedevice
    1 setlinecap 1 setlinejoin
    40 setlinewidth newpath
    exch begin
        Encoding exch get
        CharProcs exch get
    end
    exec
} def

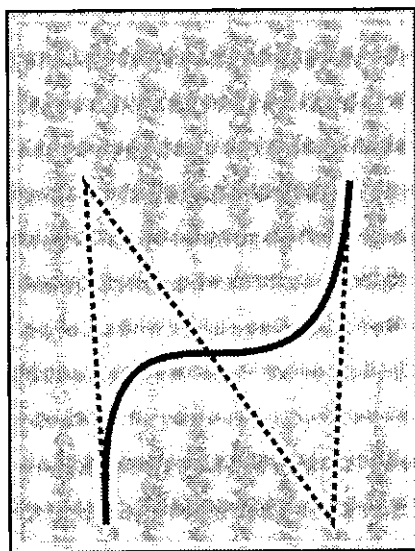
```

... at least it could if all the characters were as simple as Δ and $\square$. The power of PostScript is nicely illustrated by a more complex symbol such as α :

```
/ALPHA
{ % just 3 curves are needed to make an alpha!!
  504 448 moveto
  508 280 420 20 280 20 curveto
  56 20 56 440 280 440 curveto
  450 442 490 -240 590 130 curveto
  stroke
} def
```



The *curveto* operator adds what is known as a 'Bézier cubic section' [6] and [7] to the current path. This is a very economical way of defining complex curves, which is best illustrated by a simple example.



```
130 270 moveto
120 450 lineto
250 270 lineto
260 450 lineto
[3] 0 setdash 2 setlinewidth
stroke
130 270 moveto
120 450 250 270 260 450 curveto
[] 0 setdash 3 setlinewidth
stroke
showpage
```

It is defined by 4 x/y pairs, the current point being the first of the set. These points define three construction lines, the resulting curve starts off tangential to the first line, ends tangential to the last, and always remains within the quadrilateral made by the four points.

With just a little bit of skill, and a deal of playing with various curves, you soon learn how to assemble even quite complex shapes, the α being a fairly extreme example! Much more typical are things like () which can be done very easily indeed.

The APL code to add a Bézier curve to the current path is the final element in the chain of functions I needed to test out my PostScript characters on the screen.

```

▽ R+STEP BEZ MAT;T;ABC
[1]  ⍝ Bezier polynomial from {4,2} matrix of {X,Y} co-ords
[2]  ⍝ T is the parametric driver .... 0..1 in steps of STEP
[3]  T←(÷STEP)×0,1STEP
[4]  ⍝  $X(T) = AxT^3 + BxT^2 + CxT + X(0)$ 
[5]  ABC← 2 3 ρ1
[6]  ⍝ Cx,Cy
[7]  ABC[;3]←3×MAT[2;]-MAT[1;]
[8]  ⍝ Bx,By
[9]  ABC[;2]←(3×MAT[3;]-MAT[2;])-ABC[;3]
[10] ⍝ Ax,Ay
[11] ABC[;1]←MAT[4;]-MAT[1;]+ABC[;3]+ABC[;2]
[12] ⍝ X(T) and Y(T)
[13] R←((T*3)∘.×ABC[;1])+((T*2)∘.×ABC[;2])+(T∘.×ABC[;3])
[14] R←R+(ρR)ρMAT[1;]
▽

▽ X CURVETO DUM;SEG
[1]  ⍝ Add Bezier Curve to current path.
[2]  X←((⌈0.5×ρX),2)ρX
[3]  SEG←12⌈[⌈ΔCSEG×SCALE
[4]  X←CPT;SCALE×OFF MOVE X
[5]  PATH←PATH;(1 0 +SEG BEZ X),1
▽

```

Just for completeness, you sometimes need a simple circular arc (e.g. ° or o); the APL analogue of the PostScript *arc* operator simply adds a roundish polygon (ΔCSEG is a global to determine how well curves are drawn) to the current path:

```

▽ X ARC DUM;D2R;NSEG;SEGS
[1]  ⍝ Add a series of arc segments to current path.
[2]  ⍝ X gives [x,y,r,α,ω] where α and ω are in degrees..
[3]  D2R←(○2)÷360 ○ NSEG←12⌈[⌈ΔCSEG×SCALE
[4]  SEGS←X[4]+((-/X[5 4])÷NSEG)×0,1NSEG
[5]  SEGS←SCALE×(OFF+X[1 2])MOVE X[3]×q 2 1 ○.○SEGS×D2R
[6]  PATH←PATH;SEGS,1
▽

```

The addition of various other PostScript operators (such as *arcto*) is left as an exercise for the reader; I didn't need them!

Summary

PostScript and APL have enough in common that you can quite quickly assemble enough of a PostScript interpreter in APL to try out simple lines and curves. With this assistance available, it is possible to design a stroked PostScript APL font that follows the spirit of the original APL golfball, but can be scaled to any (reasonable) size and printed at any rotation.

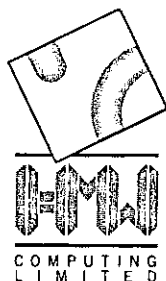
The entire font was printed in Vector 6.4, page 116: obviously I welcome all comments, and will do my best to take account of everyone's views before I make it more generally available. It can be used from within any standard wordprocessor (e.g. Microsoft Word) which supports PostScript printers. Simply select Courier-Oblique in the document where you want the APL to appear and type the appropriate character from □AV. (e.g. alt-149 for □). When you print the file, your word processor will first send a prologue file (postscrp.ini in the case of MS Word) to the printer. Patch this to substitute APL-2741 for Courier-Oblique; because the fonts are designed to exactly the same scale, everything will function just as normal, except that instead of sloping courier you will get *APL*.

Please note that although I have used STSC's □AV where possible, I have moved any characters which fell below 32, as these tend to do very nasty things to word-processors! Also the oddball symbols like ☐☐ just found holes where there was room. Of course you can very easily re-encode the entire font if you prefer APL2!

Adrian Smith
Operational Research
Rowntree-Mackintosh Confectionery Ltd.
YORK YO1 1XY
England

References

- [1] POSTSCRIPT Language Tutorial and Cookbook. Adobe Systems Inc. Addison Wesley, ISBN 0-201-10179-3
- [2] POSTSCRIPT Language Reference Manual. Adobe Systems Inc. Addison Wesley, ISBN 0-201-10174-2
- [3] Vector Vol.5 No.4 page 29.
- [4] Iain Hayward (1989), Technical Correspondence, Vector Vol.5 No.3 page 110.
- [5] APL Market News / APL News ... any recent edition.
- [6] POSTSCRIPT Language Reference Manual, page 140
- [7] Fundamentals of Interactive Computer Graphics. J.D. Foley and A. Van Dam. Addison Wesley 1982.

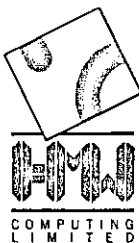


SOLUTIONS FOR THE CITY

We are a specialist firm of consultants with a prestigious list of clients including major City institutions.

For one of these clients we have recently developed a foreign exchange dealing system with the following features:- deal entry, deal enquiry, multi-currency position keeping, basket currency calculators, arbitrage calculators and ticket production.

If you are interested in foreign exchange dealing, treasury operations, portfolio management, commodity broking or similar areas we would be delighted to discuss how we could help. Call Stan Wilkinson, HMW COMPUTING LIMITED Hamilton House 1 Temple Avenue London EC4Y 0HA Telephone 01-353 4212



EXCITING POSITIONS AT THE LEADING EDGE

With top City institutions and leading Blue chip companies amongst our client list we have a lot to offer experienced APL programmers.

Call Stan Wilkinson to discuss your future.

HMW COMPUTING LIMITED
Hamilton House 1 Temple Avenue
London EC4Y 0HA Telephone 01-353 4212

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner	Adrian Smith	110
Technical Correspondence	Eugene McDonnell et al	111
APL Standards Update	David Ziemann	124
A Survey of $\square NC$	Joseph L.F. De Kerf	127

Hacker's Corner: Lots More VGA Colours

by Adrian Smith

I have always been slightly puzzled by those ads in the computer press which talk about the zillions of possible colours you can get out of XYZ Corporation's latest VGA. I always seemed to be stuck with 16 foreground and 16 background (assuming you kill the blink bit) which makes 256 in all.

The first breakthrough was when I noticed that `GINIT` can take a left argument specifying up to 16 VGA Palette Registers. Of course there were no clues as to which of the possible 64 colours was which (other than the hint that the normal range was found in the bottom 8 and the top 8 - 0 1 2 3 4 5 6 7 and 56 57 58 59 60 61 62 63). At the time, I was playing with a Hercules-oriented version of C&D's Starmap workspace, adding a bit of helpful colour. One obvious extension was to put the bright stars into their correct spectral types ... Betelgeuse should be red, and the sun should be orange. Armed with an old star atlas, I experimented with the palette settings, and quickly discovered that colour 38 is a rather excellent orange!

Of course this trick only works for graphics, and APL*PLUS helpfully resets everything when you exit! As a last resort, I read the instructions (in this case a very useful MicroSoft book all about VGA programming) to find the BIOS calls involved. The screen border and palette registers are trivially easy to set, and can be used to good effect to brighten up your screen designs. There are several excellent background colours (e.g. a nice deep green) available, as well as the above mentioned orange. As with all these settings, note that <Ctrl-Esc> resets everything to default in APL*PLUS/PC.

▽ VGAΔPALETTE VEC;RG;BX

```
[1]  A Set Vga palette register VEC[1] to VEC[2]
[2]  A AH = 10hex      ... BIOS service 10 (EGA/VGA only)
[3]  A AL = 0hex       ... subservice 0 (1 sets the border)
[4]  A BH = 0-63       ... required colour
[5]  A BL = 0-16       ... palette register to set (0 for bdr)
[6]  BX+(256×VEC[2])+VEC[1]
[7]  RG+(4096,BX)□INT 16
```

▽

```
VGAΔPALETTE 6 38      A Switch brown to orange
VGAΔBORDER 38         A Use an orange frame
43 15 55 62 38 52 4   A OBAFGKM ... star spectral types!
```


If you want an example of a really good use of this feature, try out a wonderful little program from Quarterdeck called Manifest (comes along with QEMM 5.0). To select an appropriate range of colours for your application, I suggest you set up a simple workspace which splashes colour around the screen, and experiment. Of course different types of monitor may give more or less acceptable results, and users can always wreck the whole thing by fiddling with the contrast button!

Of course 64 colours is more than 16, but we are still some way short of the 64,000 or so that the advertisers claim! The next stage is to fiddle with the Red/Green/Blue balance of any of the 64. Having created a colour which is to your liking, you then map it to one of the 16 palette registers as above.

▽ *VGAΔCOLOUR VEC;RG;BX;CX;DX*

```
[1]  A Set Vga colour register VEC[1] to VEC[2 3 4] (RGB)
[2]  A AH = 10hex      ... BIOS service 16 (EGA/VGA only)
[3]  A AL = 10hex      ... subservice 16
[4]  A BX = 0-63       ... colour register
[5]  A CH = 0-63       ... green intensity
[6]  A CL = 0-63       ... blue intensity
[7]  A DH = 0-63       ... red intensity
[8]  A Note that intensities go down by half for -8 from 63
[9]  A e.g. 63 .... 55 .... 47 .... 39
[10] A full... half... qtr ... eighth
[11] VEC+4+VEC
[12] BX+VEC[1]
[13] DX+256×VEC[2]
[14] CX+(256×VEC[3])+VEC[4]
[15] RG+(4112,BX,CX,DX)□INT 16
```

▽

VGAΔCOLOUR 38, 48 48 0 A Set up orange as Red + Green

The intensity of each colour is specified on a scale from 0..63, but the relationship is highly non-linear; a good rule of thumb is that by subtracting 8 you halve the brightness:

```
63 63 63 ..... bright white
55 55 55 ..... half intensity white
47 0 0 ..... quarter intensity red
42 24 42 ..... Laura Ashley 'Autumn Plum' etc.
```

When setting you a range of colours, you need to decide whether you belong to the Benetton school (lots of primaries), or the Laura Ashley school (subtle blends). Have fun.

TECHNICAL CORRESPONDENCE

The Dangers of APL RISC programming ...

From: Bob Bykerk

5 Feb 1990

The article by Gerard Langlet on APL Risc programming (Vector 6.2) created the impression that Gerard does not program in the real world. His argument is loosely strung together and not adequately supported by example. Translated, his argument reads something like this:

Here is a function which works on homogeneous data.
Here is my way of writing it without GOTOs, locals, or matrices.
Therefore, the use of GOTOs, locals and matrices is not good practice.
I have written thousands of these functions, so the above must be true.

The example is useful for homogeneous data, and would not require any data structure other than a vector anyway. It merely shows an alternative, shorter solution to the problem, and does not even demonstrate any tangible benefits, e.g. efficiency. Surely he could have used an example derived from one of his thousands of functions which would demonstrate how a problem dealing with heterogeneous data could be more easily solved by stripping out the structure and converting the data to a series of vectors?

I feel that **being pedantic** about eliminating GOTOs is a useless exercise, especially when structured code can easily be written (to flow from top to bottom, without spaghetti) using them. Sure, **trying to reduce** the number of GOTOs may reveal a more elegant and efficient solution, but the use of GOTOs for special casing, taking into account particular properties of the data, eliminating redundancies, or selecting only relevant parts of the data often reveal solutions that are many times faster than non-branching solutions. String searching as developed in the *Collected Whizzbangs* texts is a good example of this.

His idea of not using locals is fundamentally wrong. Not only does it mean that multi-line functions will almost invariably not be restartable, but they also

become unreadable, and consequently a nightmare to debug. Local variables are used to store intermediate results. These are most useful for debugging, as one can see the results of all major transformations up to any point in a function, enabling one to retrace one's steps. That way you don't burn your bridges.

Vectors are great for data without structure, unless you want the overhead of partitioning or enclosing. Matrices or higher dimensional arrays are ideal for mimicking structure inherent in business operations, and there are many primitives in APL which naturally and optimally manipulate this type of data. These data structures mirror the way that one would view the information, e.g. sales data in 3 dimensions - salesman $\times$ month $\times$ year would be seen as pages of sales data, one page per year. A complete analysis can be done on this data without breaking it into vectors, so why entertain the idea?

To conclude, I think that the ideas Gerard presented are on the whole quite worthless until he backs them up with substantial examples. These ideas, if presented to beginners in APL could severely warp the process of learning a healthy coding style.

Bob Bykerk
c/o Sydney APLUG
P.O. Box 1425
GPO Sydney 2001
Australia

VCAT in Sharp APL

From: E.E. McDonnell

9 April 1990

I was interested to see Norman Thomson's note "VCAT: A Response" in Vector 6, 3. He provides a "modern didactic view" of the problem which, as your editor points out, is slower than the "antique, pragmatic" utility which it offers to supplant. It is, of course, not at all uncommon for the initial implementation of a new feature to be slower than the facility it is intended to replace; when the implementors have enough experience (or often enough complaints from customers) they often find ways to make the new facility more competitive in speed.

I can report, however, on an alternative which is also modern, and while I can hazard no guess as to its didactic merits, I do know that it has the merits of being much shorter even than Mr Thomson's solution, handles the degenerate cases of

scalar and vector arguments with no extra fuss, and is faster than the antique it can replace.

The solution is written in Sharp APL. The view of the problem it takes is influenced by the design of the monadic format function and by the availability of the table, or matrix-making primitive. A boxed list of the arguments is formed using the link function: $l \triangleright r$; this two-element list is made into a two-by-one matrix, using the table function: $\tau l \triangleright r$; and this is formatted using monadic thorn, which has the property that it expands boxed items to match the largest in size: $\nabla \tau l \triangleright r$. That's all there is to it.

Here is the whole function:

```
[0] z+l vcat r
[1] z+ $\nabla \tau l \triangleright r$ 
```

Timings made on the Sharp mainframe system for arguments of varying sizes are as follows:

<i>pl</i> left		<i>pr</i> ight		$z+l \text{ vcat } r$	
				<i>eem</i>	<i>PB</i>
				<i>vcat</i>	<i>VCAT</i>
2	5	3	3	118	115
12	80		23	125	112
56	132		63	137	131
100	200	300	30	270	320

Yours faithfully

Eugene McDonnell
Reuters, Suite 200
425 Sherman Avenue
Palo Alto, CA 94306 USA

How to Kill More Cats ...

From: Norman Thomson

25 May 1990

Peter Branson's reply (Vector Vol.6 No.4) to my *VCAT* suggestions rightly takes me to task for using the pompous phrase 'a modern didactic approach'. In simple terms I mean that APL2 allows us to cut through much of the algorithmic obscurity of old-time APL, thereby making APL2 of great value as a specification language. I agree with both Adrian and Peter that my *VCAT* is horrendously inefficient with screenful-sized matrices, but that I submit is not the point. Consider rather that *VCAT* is only the tip of an iceberg, in the sense that if you are motivated to have *VCAT* in your toolkit, you are likely also to want *HCAT* (horizontal catenation). Then there is the further matter of justification, - do you want primary (left/top) or secondary (right/bottom)? So generalise *CAT* to take a vector of matrices as right argument *R* and 1 or 2 as left argument to denote horizontal or vertical:

```
[0] Z+L CAT R
[1] Z+>[L]>,/<[L]"R
```

```
      A B
WEE  RATHER
ONE  LARGER
      MATRIX
```

```
      (1 CAT A B)(2 CAT A B)
WEERATHER WEE
ONELARGER ONE
      MATRIX  RATHER
              LARGER
              MATRIX
```

Justification is then an exercise in using rotate as a dual transformation, and can be embodied in:

```
[0] Z+L TAC R
[1] Z+<[L]L CAT<[L]"R
```

```
      (1 TAC A B)(2 TAC A B)
      RATHER    WEE
WEELARGER      ONE
ONEMATRIX      RATHER
              LARGER
              MATRIX
```

Now degeneracy has to be considered. A vector requiring catenation may be regarded either as a row or a column vector, so there is another 1/2 option for the left argument L of the function MAT which converts its right argument R into a matrix.

```
[0]  Z←L MAT R
[1]  Z←[L]1/←1/R

      (1 MAT 'PIG')(2 MAT 'PIG')
P  PIG
I
G
```

(1/R) on the right ensures that MAT will also deal with a scalar, but in order to work as a null function for a non-degenerate matrix R it is necessary to add the first line:

```
[.1]  →(2=ρρR)/0
```

The eight significant choices, i.e. all combinations of horizontal/vertical, primary/secondary and row/column resolution have now been dealt with in a systematic way, leaving the path open for efficiency if the goal is other than a pure teaching one. I don't think Peter and I have any fundamental disagreements, and I certainly don't suggest the abandonment of old practices and idioms where these are of worth at the implementation stage. I do however advocate the use of APL2 at specification and design stages, and invite readers to ponder which the designer for efficiency has by viewing the whole of this particular problem so compactly.

Norman Thomson

Mail Point 188,
IBM UK Laboratories Ltd.,
Hursley Park
WINCHESTER
Hants SO21 2JN

Comment: I never thought that such an innocuous little function could generate such interest; maybe we should expose some more 'pragmatic antiques' to competition from advanced APLs? Ed(ex).

From: John Scholes

15 May 1990

I have received a letter from Mr De Kerf referring to my article in Vector 6.4 on defined operators. He points out that the *ON* operator is a version of the *POWER* operator, and includes a copy of his excellent article "A Note on the Power Operators (Loops are Harmful)" from the APL-CAM Journal Vol.11 No.1, 16 Jan 1989 pp 228-234. This provides a history of the operator with references, and a better coding of it using $\pm$.

Also, the enclosed letter (addressed to you) appeared for some reason at Dyadic HQ recently. I pass it on without comment

From: I. Phelfritt (née Leigh)

May 1990

I was thrilled to see your review of IBM's new APL2.5 in April's Vector 6.4, and felt moved to send you a further couple of examples. One consequence of the direct definition of operators which may not be immediately obvious is the ease with which you can define the *INVERSE* operator for monadic functions:

INV: $\omega\beta$

INV produces a derived function which reverses the effect of its operand. Formally: $A \leftrightarrow F \text{ } INV \text{ } F \text{ } A$ for any array *A* and any monadic function *F*. The following examples show the power of this operator ...

```

      ÷4
0.25
      ÷INV 0.25
4
      +\INV 1 3 6
1 2 3

```

Where the inverse is not unique, the system provides a 'least squares' solution:

```

      +\INV 6
6
      ?INV 4
7

```

and of course, *INV* may take a system function as operand, so:

```
FOO      □EX INV 1      a re-instates a previously expunged function
2        □DL INV 2      a has the same effect as □DL -2 in dyalog APL
```

We are only now beginning to appreciate the power of this new implementation, but I'm sure that such examples will soon swell our idiom libraries.

I. Phelfritt
c/o Dyadic Systems Ltd.

The APL*PLUS 'dead key' Problem ...

From: Cedric Heddle

12 June 1990

I hope that one of your readers may be able to tell me how to solve the 'dead key' problem on UK PS/2 keyboards when using STSC APL*PLUS/PC and the Unified keyboard. The key is immediately to the left of 'Z', has a key button number of 45, a scan code of 86 and in DOS it produces '/' when unshifted and '|' when shifted. Apparently this key does not exist on American keyboards, whose left-hand shift key is wider than those on the UK keyboard.

STSC say that they believe that the scan code for this key is 86 and that their □POKE 198 197 code should work with this key, although they do not have a UK keyboard with which to test this belief. In my experience, this key remains dead despite poking characters into the 86th row of the keyboard table. I have even poked into every one of the twenty or so empty rows in the 128-row keyboard table without having any effect on this key. Incidentally, the shifted '/' above '8' on the numeric pad, which also produces a '/' seems also to be unchangeable.

An assembler routine executed via)CMD confirms the BIOS scan code of 86. If any of your correspondents has a solution I am sure I shall not be the only grateful reader.

Yours sincerely,

Cedric Heddle,
APL Impetus Ltd.

The Falkoff Keyboard: a Franco-Belgian Response!

From: Alain Delmotte

Rome 29th of May 90

I read with interest the article of Adrian Smith on the Falkoff keyboard (VECTOR 6.4 p.92). As an inveterate French speaker and typist, I completely agree with the first precept of Falkoff: "If a symbol is shown on a key, then that is where it should go." This makes justice to other languages keyboard layouts.

This is what I did for the SharpAPL/PC and tested also with APL*PLUS. On a French or Belgian keyboard (AZERTY) figures are uppercase, some specific symbols appear (accented a,e,u and ç) and some common use symbols (@#\^[]) are obtained with Ctrl-Alt-key (key being the key on which these symbols are engraved in front).

I enclose an example of my keyboard on the Toshiba 1000. The left side shows the base and shift normal keys (the engraved normal keytop). The right side shows the Alt and Ctrl combinations (for APL*PLUS Alt-Shift in place of Ctrl).

Clavier SharpAPL pour Toshiba 1000

1 ! @	2 " #	3 \$ %	4 ^ &	5 ()	6 & * =	7 & * =	8 ! @ []	9 ~ { }	0 ~ { }	~ { }	~ { }	Backsp		
Tab	A a	Z c	X e	R p	T ~	Y †	U ‡	I v	O o	P •	~ ^	* J	> }	< [
Ctrl	Q ?	S r	D l	F f	G g	H h	J •	K x	L o	M i	~ u	~ t	Enter	
Shift	W w	X x	C n	V u	B b	N r	? ^	~ _	/ =	+ =	Shift	†		
Alt	Cap loc	Fn	Space					~ { }	Ins	Del	+	+		

\ was moved from the <> key to leave room for ≤ ≥. Mu not existing in □AV was replaced by €. There are sometimes problems with the <> key (scan code 43); when redefined it could move to the left of the &1 key or to the left of the W key depending on the keyboard.

It is also possible and very useful to define the numerical keypad in APL*PLUS (Numlock) as it is in SharpAPL: + gives a blank and - a high minus. Smith's arguments would be:

```
NKEY←74 78
NKEYDEFS←⊞2 6ρ'-' + ' '
```

Yours sincerely

Alain DELMOTTE
Avenue du Marathon, 6
B1348 Louvain-la-Neuve BELGIQUE

N.B. I think that Smith's $\rho AFKEYS$ should be 51 7 with APL*PLUS unified keyboard, the Shift-Alt being in position [6] with $\square IO \leftarrow 0$.

Surely There Must Be a Better Way ...

From: Adrian Smith

17 June 1990

Code fragment recently excavated and much pored over ...

```
RC←⊡(1+⊡(4800,5052)∈⊡ΔADMIN)⊃(('EEV') ('EV'))
```

We believe that this is meant to execute *EV* if $\Delta ADMIN = '5052'$, otherwise execute *EEV*. Suggestions for simplification welcome!

P.S. The following fragment from John Preston (Saladin) via Bob Poate (MicroAPL):

```
23817 □INT 33
```

This closes a network print file, which will otherwise sit on the print queue until you exit from APL to the DOS prompt. Thanks.

Speculations on Fermat Equations ...

From: Anne D Wilson

12 June 1990

Zdenek V Jizba's article on Fermat type equations led to me to some interesting thoughts on solutions to $x^2 + y^2 = z^2$ where x, y, z are integers and cannot be reduced to another equation by division; e.g. $6^2 + 8^2 = 10^2$ reduces to $3^2 + 4^2 = 5^2$.

The following discussion uses the concept of modulo algebra and numbers **prime to each other**. Modulo algebra deals only with the remainders, which can be generated using the APL primitive. If two numbers are prime to each other they have no common factor other than 1. Thus 4 and 9 are pteo, but 12 and 21 have a common factor of 3. The following recursive function returns the highest common factor (HCF) of 2 numbers. Where numbers are pteo the HCF equals 1.

```

      ∇ R←A HCF B;C
[1]  ⍺←R←B', (0<C+B|A)/' HCF C'
      ∇

```

It uses a chain of quotients and remainders to find the HCF. The intermediate results for 31 and 12 are:

by calculation	using the above function
HCF of 31 and 12 is 1, ie they are pteo.	
$31 = 2 \times 12 + 7$	31 HCF 12
$12 = 1 \times 7 + 5$	12 HCF 7
$7 = 1 \times 5 + 2$	7 HCF 5
$5 = 2 \times 2 + 1$	5 HCF 2
$2 = 2 \times 1$	2 HCF 1

- (1) In the equation

$$x^2 + y^2 = z^2 \quad x, y, z \text{ are pteo.}$$

Proof:- If x, y have a common factor k

$$k^2 a^2 + k^2 b^2 = k^2 (a^2 + b^2)$$

and k is a factor of z ;

therefore the equation can be reduced by division by k .

- (2) In the equation

$$x^2 + y^2 = z^2 \quad x \text{ and } z \text{ are odd and } y \text{ is even, or } y \text{ and } z \text{ are odd and } x \text{ is even.}$$

Proof:-

(i) If 2 of the variables are even they have a common factor of 2, and the equation can be reduced by (1).

(ii) Odd numbers are either $4m + 1$ or $4m + 3$

which when squared give $4n + 1$ or $4k + 9 = 4n + 1$

hence the square of any odd number is $4n + 1$

Even numbers are $2m$ and the square of an even number is $4n$.

Considering all possible combinations:

$x = 4m+1$	$4m+3$	$2n$	
$x^2 = 4r+1$	$4r+1$	$4r$	
$y = 4n+1$	$4s+2$	$4s+2$	$4s+1$
$y^2 = 4r+1$	X	X	z odd
$y = 4n+3$	$4s+2$	$4s+2$	$4s+1$
$y^2 = 4r+1$	X	X	z odd
$y = 2n$	$4s+1$	$4s+1$	$4s$
$y^2 = 4r$	z odd	z odd	X

Table showing
derivation of z^2
where $x^2 + y^2 = z^2$

As x, y, z are not all even, and the square of an odd number is $4s+1$ the only valid combinations are:

- x even, y and z odd
or y even, x and z odd

With no loss of generality we can take the case where x is odd and consider the equation in the form: $x^2 = z^2 - y^2$

- (3) If p, q are odd numbers pteo $(p^2 + q^2) \bmod 4 = 2$

Proof:-

p and q are odd therefore $(p^2 + q^2) \bmod 4 = 1 + 1 = 2$

- (4) If p, q are odd numbers pteo $(p^2 - q^2) \bmod 8 = 0$

Proof:-

p and q are odd therefore

$$p = 2m + 1$$

$$q = 2n + 1 = 2(m - k) + 1$$

$$p^2 - q^2 = (p + q)(p - q) = 4(2m - k - 1)k$$

Either k is odd when $(2m - k + 1)$ is even or k is even when $(2m - k + 1)$ is odd and therefore another factor of 2 can be extracted.

$$\text{Hence } (p^2 - q^2) \bmod 8 = 0$$

- (5) If a, b are odd numbers pteo

$$\text{and } z = 0.5(a + b)$$

$$y = 0.5(a - b)$$

then if there exists x such that $z^2 - y^2 = x^2$ a and b are squares.

Proof:-

$$z^2 - y^2 = (0.5(a + b))^2 - (0.5(a - b))^2 = ab$$

Hence $ab = x^2$ and a, b pteo otherwise z and y have a common factor; therefore a and b are perfect squares.

Indeed any pair of odd numbers p, q pteo will generate a solution to the equation. Thus it is only necessary to check that the parameters p, q are pteo to produce triplets of Pythagorean numbers.

$$\begin{aligned}x &= pq \\ y &= 0.5(p^2 - q^2) \\ z &= 0.5(p^2 + q^2)\end{aligned}$$

(6) We can deduce some characteristics of these triplets:-

(i) $z \bmod 4 = 1$ i.e. z has the form $(4m + 1)$

Proof:-

$$z = 0.5(p^2 + q^2); \text{ } p, q \text{ are odd therefore } z \bmod 4 = 0.5(1 + 1) = 1$$

(ii) $y \bmod 4 = 0$ i.e. y is a multiple of 4

Proof:-

$$y = 0.5(p^2 - q^2) = 0.5(8m) = 4m \text{ therefore } y \bmod 4 = 0$$

(iii) $z - y = q^2$ which is a perfect square.

$$(iv) \quad z - x = 0.5(p^2 + q^2) - pq = 0.5(p - q)^2$$

and as p and q are both odd it equals $2 \times$ a perfect square.

Some example generated from p and q are ...

p	q	$x=p \cdot q$	$y=0.5(p^2 - q^2)$	$z=0.5(p^2 + q^2)$
3	1	3	4	5
11	5	55	48	73
101	67	6767	2856	7345
101	99	9999	200	10001
998	6	1497	124496	124505
1933	499	964567	1743744	1992745

Although the above discussion depends on the choice of x as an odd number similar calculations can be performed when x is an even number. In this case the parameters r and s are defined and deduced as:

$$\begin{aligned}y &= (r^2 - s^2) \\ z &= (r^2 + s^2) \\ x &= 2rs = \sqrt{2}r \cdot \sqrt{2}s\end{aligned}$$

However this merely generates the same triplets, but with x and y interchanged.

For the triplets generated by Jizba's function $R \leftarrow K \quad PFERMAT \quad W$

$$\begin{aligned}\text{where} \quad & K = z - x \text{ and } W = y \\ \text{or} \quad & K = z - y \text{ and } W = x\end{aligned}$$

K must be either (the square of an odd number) or $(2 \times \text{square})$ see 6(iii) and 6(iv). This explains why $K = 3$ and $K = 4$ do not yield triplets. $K = 8, 9, 18, 25, \dots$ would yield triplets.

Anne D Wilson
Rowntree, York

14th ISO APL Working Group meeting 30 April 90 - 4 May 90 Waterloo, Ontario, Canada

report by Dave Ziemann

Background

This is a report of the proceedings of the last ISO meeting for the standardisation of APL, which I attended on behalf of the BSI APL Standards Panel. This was made possible by funding provided by the British APL Association, without which British representation would not have been possible due the withdrawal of funding for ISO meetings by the DTI.

I took to the meeting five sets of BSI comments, and a report on symbol conflicts.

The meeting was relatively small, with eight representatives from Belgium, Canada, UK and the USA. A number of regular attendees, the French group in particular, were not able to come due to funding difficulties.

The main technical work of the meeting involved reviewing newly received documents, and working through the 1st Working Draft of the Extended APL Standard, page by page.

The New Document

The changes to the existing standard are so pervasive that a supplement was not possible, and so a new document was created, based on IS 8485, the existing standard.

The 1st Working Draft was prepared by Eugene McDonnell, the editor, based on the existing standard and twenty-six accepted proposals from the following authors; Eugene McDonnell, Roland Pesch, Everett Allen, Bruce Frost, Greg Jaxon, Joseph de Kerf, Karl Soop, Brent Hawks and Phil Benkard.

The original standard document was produced by Alex Morrow using a version of IBM's Script language. This has now been converted to T_EX by Eugene McDonnell with help from a T_EX expert, thereby making the document more easily portable.

The topics in the document have not changed substantially since the Princeton meeting; they include, in no particular order: without, GCD and LCM, duplicate/commute operator, table, first-axis join, mixed arrays, use of overbar/underbar in names, replicate, character grade, unique, shared variable extensions, alpha and omega as names, ambivalent defined functions, event handling, transfer form, n-wise reduction, complex arithmetic, left and right, function rank operator, defined operators, *)OPS*, *)NMS*, component file cover functions and a revised Workspace Interchange Standard.

Changes made at Waterloo

One significant change is in the area of name-class. Joseph de Kerf had earlier presented a review of name classifications in about forty dialects of APL. We had already agreed to use 4 for defined operators, and now Joseph was suggesting that 5 and 6 be standardised for system variables and system functions. In addition, -1 should be returned for arguments to *⊔NC* which are not valid names, rather than returning 4 or signalling an unhelpful *DOMAIN ERROR*.

All three changes were passed, with no votes against.

Although the new document did contain grade-up and grade-down extended for character arrays, they had not been defined for numeric matrices and higher rank numeric arrays. A proposal to plug this hole was accepted.

A number of extensive changes were required in order to fit defined operators into the document. Defined operators follow the APL2 philosophy, which means that they are not "true" operators, but are more accurately described as defined derived functions. Defined operators and the new function rank operator will accept defined function operands in the Extended Standard, even though it is not currently possible to generalise this to the existing primitive operators in the language.

The Annex which describes the Workspace Interchange Standard has been upgraded to WSIS2. Although the Extended Standard itself does not address nested or boxed arrays, WSIS2 describes a convention for dealing with them. The Annex is still termed "Descriptive", which means that a conforming implementation is not required to provide the facility.

The Technical Report

A "Technical Report" is an ISO designation applied to a standard which fails to meet the necessary standardisation criteria. WG3 is using this mechanism to collect all the proposals for which consensus could not be achieved.

The topics to be included in the Technical Report have not changed since the last meeting. They are: reduction, nested arrays and fills, enclose / disclose / box / open, formatting for nested arrays, pick, partition, depth, link, enlist, strand notation, match, generalised primitive operators, compose, dual, first, each, national language characters, function rank in standard metalanguage, function assignment.

The document will contain brief descriptions of why each facility had to be included in the Technical Report, and could not go into the new standard. The Technical Report will be useful for implementors who require guidance, and will provide a strong basis for future standardisation efforts.

A Warning

Standards committees have a habit of changing their minds. A decision made at one meeting can be reversed at the next; this can happen because the meeting composition varies according to the country in which it is held. Also, people do occasionally change their minds.

For this reason none of the information given in this report should be considered final. Anyone who implements an APL on the basis of pre-publication material takes a big risk. Some implementors fell foul of this in the first APL standard, where an early draft required the `)RESET` system command, which was later changed to the more expressive `)SIC` (SI Clear).

Name Classifications in APL: a Survey

by Joseph L.F. De Kerf

Abstract

Originally the user of APL could only identify the types of the objects in his workspace with the system commands)VARS,)FNS, and - if groups are supported - the system command)GRPS. In addition, in most implementations the listing displayed cannot be trapped in the form of an explicit result. To overcome these deficiencies, the system functions Name Class $\square NC$ and Name List $\square NL$ have been incorporated. A survey is given of the related name class implementations for about forty APL dialects. Remarks concerning the standardization of this name classification are stated.

Name Classifications in APL

APL does not need explicit data typing. Variables are introduced by the user through assignment and their type and structure may be changed dynamically. User-defined functions and - if supported - user-defined operators are introduced either with the editor or in some implementations through direct definition. If supported, variables and user-defined functions/operators may be collected to groups. As such, the user is building up a set of reserved words - the names of the objects in his workspace. Eventually, the user may discard members of this set by erasing or expunging.

Originally the user could only identify the types of the objects in his workspace with the inquiry commands)VARS for a variables listing,)FNS for a defined function listing, and eventually)GRPS for a group listing. More recently, the system command)OPS has been added for - if supported - the defined operator listing and the command)NMS for the listing of the total set of object names. Each of these commands displays the names of the objects in alphabetical order, eventually restricted to an alphabetical subset as specified by the parameter(s) of the command. In most implementations however, the listing displayed cannot be trapped in the form of an explicit result.

To overcome the deficiencies of this approach, the system functions Name Class $\square NC$ and Name List $\square NL$ were incorporated. $\square NC$ returns as explicit result a vector of name class integers giving the local usage of the identifier in the

corresponding row of the right argument matrix. $\square NL$ returns as explicit result an array containing the name list of the local identifiers specified by the right argument scalar/vector of name classes. In its dyadic form, the name list is restricted to those identifiers whose initial character is the same as one of the characters occurring in the left argument character scalar/vector. Whether the names are listed in alphabetical order or not, depends on the implementation. In most implementations the names are alphabetically ordered, but in some implementations the names are listed in fortuitous order or in the order as they first appeared in the workspace.

Note: For those implementations which support the system command $\rangle NMS$, the names listed by this command are followed by a dot and their name class. The name classes are the same as those produced by the system function $\square NC$.

Name Class Implementations

Descriptions of name class implementations for about forty dialects are given at the end of this paper. For purposes of comparison the list starts with the standardizations proposed A.D. Falkoff and D.L. Orth at APL79 (1979) [1] and by the ISO Draft International Standard DIS 8485 (1986) [2].

The descriptions give the name classes returned by $\square NC$. Any deviations of those used by $\square NL$ and additional information if significant are reported in the form of a note. In most implementations $\square NC$ returns name class 0 for names available without active referent, name class 1 for line labels, name class 2 for user-defined variables, and name class 3 for user-defined functions.

In many implementations $\square NC$ returns name class 4 for names not available: invalid names, distinguished names, and - if supported - group names. In some implementations however, name class 4 is reserved for groups only and in all implementations which support user-defined operators name class 4 is reserved for such operators. Invalid names in those implementations return an individual name class such as -1 or cause an error report such as "domain error" or "name error". Names of system variables and system functions in those implementations return an individual name class such as 5 and 6 respectively or are treated as common objects by $\square NC$ but are not listed by $\square NL$. In VAX APL Versions 2 and 3, a distinct name class is associated to niladic and non-niladic system functions. Maybe this is done because niladic system functions are in fact system variables, shared between the workspace and the processor, but which are reset by the processor before they are used.

$\square NC$ returns the name classes for the local usage of the identifiers and $\square NL$ lists the local identifiers of the type(s) selected. In some implementations additional facilities are provided for identifying global usage or/and for listing the global identifiers:

- In Xerox APL (Atkins), classification codes returned by $\square NC$ and used by $\square NL$ are different. Name classes 2/3 list the active variables and functions, while name classes 5/6 list the global variables and functions. 3 $\square WM$ 'type' corresponds to $\square NL$, 6 $\square WM$ 'namelist' corresponds to $\square NC$, but 7 $\square WM$ 'namelist' identifies the global use instead of the local use of the names listed.
- In APL 2.0 and NOS/VE APL (CDC), the system variable $\square ENV$ controls whether $\square NC$ and $\square NL$ refer to the global or the current environment. When 0, the global environment is used. When 1, the current environment is used. The default is 1. When the state indicator is empty, the global and current environment are the same and $\square ENV$ has no effect.
- In OS4000 APL (GEC), $\square NL$ may list global objects: name classes 2/3/4 are used to list local variables/functions/groups, while name classes 5/6/7 are used to list global variables/functions/groups.
- In 2900 APL (ICL), $\square NC$ may be used dyadically. If the left argument is 0, global classifications are reported. If 1, current classifications are reported. The default is 1. If the left argument of $\square NL$ is 0 instead of a character scalar/vector global objects are listed. If 1, current objects are listed. The default value is 1.
- In SHARP APL (IPSA), 1 $\square WS$ versus $\square NL$ and 5 $\square WS$ versus $\square NC$ provide similar information but classification codes are different. In addition, 5 $\square WS$ reports the name class at each level of the state indicator, the global definitions included.
- In APL*PLUS (STSC), $\square IDLIST$ versus $\square NL$ and $\square IDLOC$ versus $\square NC$ provide similar capabilities, but classification codes are different. In addition, $\square IDLOC$ reports the name class at each level of the state indicator, the global definitions included.

Note: Some implementations reserve separate name classes for non-standard objects, such as intrinsic functions in Xerox APL (Atkins), special functions in NOS/VE APL (CDC), APLGOL functions in APL/3000 (HP), and special data types in MCM/APL (MCM). More details may be found in the descriptions given below.

Name Class Standardizations

In the standard proposed by A.D. Falkoff and D.L. Orth at APL79 (1979) [1], unavailable names return name class 4. In ISO DIS 8485 (1986) [2] however, such

unavailable names cause a "domain error". This permits the implementor to add name classes as consistent extensions.

In the meantime, ISO-IEC/JTC1/SC22/WG3 is preparing a new standard for Extended APL. As this new standard will probably support user-defined operators, it is obvious to associate name class 4 with those user-defined operators. At the same time it would be opportune to introduce extra name classes for system variables and system functions, for instance name classes 5 and 6 respectively. This would give the user the opportunity to list the system variables and system functions available in his processor. the complete set of name classes would become:

- 0 for identifiers available without current referent
- 1 for labels
- 2 for variables or shared variables
- 3 for user-defined functions
- 4 for user-defined operators
- 5 for system variables
- 6 for system functions

Finally, a mechanism should be standardized to give the user the possibility to choose whether $\square NC$ and $\square NL$ refer to global or local objects. The most handsome method seems to be to make available a system variable which controls this choice, such as is done in APL 2.0 and NOS/VE APL (CDC) with the implementation of the system variable $\square ENV$.

References

- [1] A.D. Falkoff and D.L. Orth; *Development of an APL Standard*; APL79 Conference Proceedings, Rochester, New York, 30 May - 1 June 1979; Edited by A.L. Anger; APL Quote Quad, Vol. 9, No.4, June 1979, pp.409-453.
- [2] *Draft International Standard DIS 8485 - Programming Language APL; Document ISO/TC97/SC22/WG3/N55*; Edited by L.A. Morrow; AFNOR, Secretariat ISO /TC97/SC22/WG3-APL, Paris, France, May 1986.

F & O APL (A.D. Falkoff and D.L.Orth - 1979)

- 0 name available without active referent
- 1 label
- 2 variable or name offered for sharing
- 3 defined function
- 4 other (distinguished name, group name, not a name)

ISO APL (ISO-IEC/JTC1/SC22/WG3 -DIS 8485)

- 0 identifier available without current referent
- 1 label
- 2 variable or shared variable
- 3 defined function

Note: Invalid names and distinguished names cause a "domain error".
Groups are not supported.

APLSV and VSAPL (International Business Machines IBM)

- 0 name available for any use
- 1 label
- 2 variable or shared variable
- 3 function
- 4 other (group name, distinguished name, not a valid name)

5100/10/20 APL and APL/PC (International Business Machines IBM)

- 0 name available for any use
- 1 label
- 2 variable or shared variable
- 3 function
- 4 not available (distinguished name or not a valid name)

Note: Groups are not supported.

APL2 and APL2/PC (International Business Machines IBM)

- ~1 invalid name or unused distinguished name
- 0 unused but valid name
- 1 label
- 2 variable or shared variable
- 3 function
- 4 operator

Note: Distinguished names are treated as common objects by $\square NC$.
Distinguished names however are not listed by $\square NL$. Groups are not supported.

Xerox APL (Atkins Computing Services Limited)

- 0 no current referent
- 1 logical variable
- 2 character variable
- 3 integer variable or label
- 4 real variable
- 5 index sequence
- 6 list
- 7 internal processor use only
- 8 user-defined function, niladic, no result
- 9 user-defined function, niladic, with result
- 10 user-defined function, monadic, no result
- 11 user-defined function, monadic, with result
- 12 user-defined function, dyadic, no result
- 13 user-defined function, dyadic, with result
- 14 intrinsic function, dyadic
- 15 intrinsic function, monadic
- 16 intrinsic function, niladic
- 17 group

Note: Intrinsic functions are analogous to locked functions in that they cannot be displayed. $\square NC$ and $\square NL$ classification codes are different. Type specification codes are:

- 1 labels
- 2 active variables
- 3 active functions
- 4 groups
- 5 global variables
- 6 global functions

$\square NL$ in its dyadic form is not supported. 3 $\square WM$ 'type' corresponds to $\square NL$. 6 $\square WM$ 'namelist' corresponds to $\square NC$. 7 $\square WM$ 'namelist' identifies the global use instead of the local use of the names listed. $\square WM$ is implemented as a duplicate of the intrinsic function ΔWM .

MAINSTREAM-APL (Boeing Computer Services BCS)

- 0 name not defined (not in use)
- 1 label
- 2 variable

- 3 function
- 4 name not available (e.g. a group)

APL/700 (Burroughs - Unisys)

- 0 no associated meaning
- 1 label
- 2 variable
- 3 function
- 4 other (group)

APLB (Burroughs - Unisys)

- 0 undefined
- 1 label
- 2 variable
- 3 function
- 4 operator

Note: Distinguished names are treated as common objects by $\square NC$.
Distinguished names however are not listed by $\square NL$. Invalid
names in the argument of the $\square NC$ cause a "name error". Groups
are not supported.

APL-64 (Cii Honeywell Bull)

- 0 available for any use
- 1 label
- 2 variable
- 3 function
- 4 not available (group name, distinguished name or not a valid name)

APL 2.0 (Control Data Corporation CDC)

- 0 available (not in use)
- 1 locked variable (label or group)
- 2 unlocked variable
- 3 function
- 4 other (e.g. a defined distinguished name)

Note: User-defined variables may be locked and as such they cannot be reset using specification. Invalid names in the argument cause a "domain error". $\square NL$ 4 provides a list of distinguished names. $\square ENV$ controls whether $\square NC$ and $\square NL$ refer to the global or to the current environment. When 0, the global environment is used. When 1, the current environment is used. The default value is 1. When the state indicator is empty, the global and the current environment are the same and δENV has no effect.

NOS/VE APL (Control Data Corporation CDC)

- 0 no object associated or not a distinguished name
- 1 locked variable (label or group)
- 2 unlocked variable
- 3 user-defined function
- 4 distinguished name or special function

Note: User-defined variables may be locked and as such they cannot be reset using specification. A special function is a function supplied by the system to provide capability or efficiency not available by other means. Special functions behave the same as user-defined functions. Invalid names in the argument cause a "domain error". $\square NL$ 4 provides a list of distinguished names and special functions. $\square ENV$ controls whether $\square NC$ and $\square NL$ refer to the global or to the current environment. When 0, the global environment is used. When 1, the current environment is used. The default is 1. When the state indicator is empty, the global and the current environment are the same and $\square ENV$ has no effect.

AOS/VS APL (Data General Corporation DG)

- 0 available for use
- 1 label
- 2 user variable
- 3 defined function
- 4 other

Note: Groups are not supported.

APL-11 (Digital Equipment Corporation DEC - DECUS)

- 0 name available for any use
- 1 label
- 2 variable
- 3 function
- 4 not a valid name or group name

APL-Basic and APLSF (Digital Equipment Corporation DEC)

- 0 name available for any use
- 1 label
- 2 variable
- 3 function
- 4 not available (not a valid name or group name)

VAX APL Version 1 (Digital Equipment Corporation DEC)

- 0 name not in use
- 1 label
- 2 variable
- 3 function
- 4 group
- 5 system variable
- 6 system function

Note: Invalid names in the argument cause a "domain error".

VAX APL Version 2 (Digital Equipment Corporation DEC)

- 5 niladic system function
- 4 group
- 3 monadic, dyadic, or ambivalent system function
- 2 system variable
- 1 ill-formed identifier
- 0 name not in use
- 1 label
- 2 variable
- 3 user-defined function

VAX APL Version 3 (Digital Equipment Corporation DEC)

- 20 derived function
- 5 niladic system function
- 4 group
- 3 monadic, dyadic, or ambivalent system function
- 2 system variable
- 1 ill-formed identifier
- 0 name not in use
- 1 label
- 2 variable
- 3 user-defined function
- 4 user-defined operator

Dyalog APL Version 1 (Dyadic Systems Limited)

- 1 invalid name
- 0 unused name
- 1 label
- 2 variable
- 3 function

Note: Distinguished names are classified as invalid names. The name class of an external variable is 2 as soon as it is associated by $\square XT$. Groups are not supported.

Dyalog APL Version 5 (Dyadic Systems Limited)

- 1 invalid name
- 0 unused name
- 1 label
- 2 variable
- 3 function
- 4 operator

Note: Distinguished names are classified as invalid names. The name class of an external variable is 2 as soon as it is associated by $\square XT$. Groups are not supported.

OS4000 APL (GEC Computers Limited)

- 0 undefined or expunged
- 1 label
- 2 variable
- 3 function
- 4 group

Note: $\square NL$ may list global objects (code 5 for global variables, 6 for global functions, and 7 for global groups).

Harris APL (Harris Computer Systems)

- 0 unused name
- 1 label
- 2 variable
- 3 function
- 4 improperly formed name, distinguished name, or group name

APL/3000 (Hewlett-Packard HP)

- 0 name available for any use
- 1 label
- 2 variable
- 3 APL function
- 4 APLGOL function
- 5 not valid (distinguished name or incorrectly formed name)

Note: The user has the opportunity of defining functions in either APL or APLGOL. A workspace may contain any mixture of APL and APLGOL functions, who can be used in any combination. Groups are not supported.

I-APL (I-APL Limited)

- 0 available for use
- 1 visible line label
- 2 visible variable name
- 3 visible function name

Note: Groups are not supported.

2900 APL (International Computers Limited ICL)

- 0 name undefined
- 1 label or locked variable
- 2 variable
- 3 user-defined function
- 4 unacceptable name (including distinguished names)

Note: User-defined variables and user-defined functions may be locked ($\square LOCK$). $\square NC$ may be used dyadically. If the left argument is 0, global classifications are reported. If 1, current classifications are reported. The default is 1. If the left argument of $\square NL$ is 0, global objects are listed. If 1, current objects are listed. The default is 1. Groups are not supported.

MCM/APL (MCM Computers Limited)

- 0 no value
- 1 label
- 2 variable
- 3 function
- 4 invalid name
- 5 special type data

Note: Special Type Data are somewhat analogous to locked functions, except that they can only be created by MCM. Groups are not supported.

APL.68000 (The Computer Company - MicroAPL Limited)

- 0 name not in use
- 1 line label
- 2 variable
- 3 function
- 4 invalid name
- 5 group

APL/LW and APL/LWSF (MIPS Software Development)

- 0 name available for use
- 1 label
- 2 variable
- 3 function
- 4 not available (invalid name, distinguished name, or name of a group)

APL*MYRIADE (OOM Incorporated)

- 0 name available for use
- 1 label
- 2 variable
- 3 function
- 4 not available (invalid name, distinguished name, or name of a group)

PortaAPL (Portable Software)

- 0 undefined
- 1 label
- 2 variable
- 3 function
- 4 group

APL/PRIME (Prime Computer)

- 0 name not in the symbol table (including illegal names)
- 1 label
- 2 variable
- 3 function
- 4 group
- 5 name in the symbol table but currently having no value

APL/16 (SEMS)

- 0 undefined object
- 1 system function
- 2 label or variable
- 3 defined function
- 4 invalid identifier

Note: Groups are not supported.

SHARP APL (I.P. Sharp Associates IPSA)

- 0 well formed name with no visible use
- 1 visible use as a label
- 2 visible use as the name of a variable
- 3 visible use as the name of a function
- 4 other (group names, distinguished names, invalid names)

Note: $\square NC$ and 5 $\square WS$ provide similar information. $\square NC$ reports only visible use, while 5 $\square WS$ reports the class at each level of the state indicator, the global definition included. $\square NC$ and 5 $\square WS$ return different classification codes. Classification codes returned by 5 $\square WS$ are:

- 1 not localized at this level
- 0 localized with no assigned value at this level or globally undefined
- 1 function
- 2 variable
- 4 group
- 8 label

$\square NL$ and 1 $\square WS$ both return the names of the visible objects, but the classification codes used are different: $\square NL$ uses an encoding consistent with $\square NC$ and 1 $\square WS$ uses an encoding consistent with 5 $\square WS$. The right argument of 1 $\square WS$ must be the sum of the codes of the classes of the objects whose names are to be reported. When the argument is negative or greater than 15, its 16 residue is used.

VIZ::APL (Sigma Software Unit - Inner Product Limited)

- 0 name free for use
- 1 label
- 2 variable
- 3 function
- 4 invalid name
- 5 system variable
- 6 system function

Note: Groups are not supported.

APL-NET (SLIGOS - BARTS)

- 0 name available for any use
- 1 label
- 2 variable
- 3 function
- 4 ill-formed

APL 1100 (Sperry Univac - Unisys)

- 0 name available for use
- 1 label
- 2 variable
- 3 user-defined function
- 4 not available for use (group name, distinguished name, or invalid)

APL*PLUS (Scientific Time Sharing Corporation - STSC)

- 0 not defined
- 1 label
- 2 variable
- 3 defined or primitive function
- 4 other (group name, distinguished name, not a valid name)

Note: $\square NC$ and $\square IDLOC$ provide similar capabilities, $\square NC$ accepts ill-formed identifiers, while $\square IDLOC$ produces a "domain error".
 $\square NC$ merely returns the locally active classification, while $\square IDLOC$ returns the classification at each level of the state indicator, the global definition included. $\square NC$ and $\square IDLOC$ return different classification codes. Classification codes returned by $\square IDLOC$ are:

- 1 not localized at this level
- 0 localized with no assigned value at this level or globally undefined
- 1 user-defined function or system function
- 2 user-defined variable or system variable
- 4 group
- 8 label

`⌈NL` and `⌈IDLIST` both return the names of the visible objects but the classification codes used are different: `⌈NL` uses an encoding consistent with `⌈NC`, while `⌈IDLIST` uses an encoding consistent with `⌈IDLOC`. The right argument of `⌈IDLIST` must be the sum of the codes of the classes of the objects whose names are to be reported. When the argument is negative or greater than 15, its 16 residue is used. `⌈IDLIST` and `⌈IDLOC` correspond to the primitive functions 1 ΔWS and 5 ΔWS respectively, the difference being that `⌈IDLIST` with its dyadic form allows the result to be restricted to names with specified first letters, while 1 ΔWS does not.

APL 90 (Ecole des Mines de Saint-Etienne - SYNC)

- 1 invalid name
- 0 unused but valid name
- 1 label
- 2 variable
- 3 defined function
- 4 defined operator

Note: Distinguished names are treated as common objects by `⌈NC`.
Distinguished names however are not listed by `⌈NL`. Groups are not supported.

TIS-APL (Telecompute Integrated Systems TIS)

- 0 name available for use
- 1 line label
- 2 variable
- 3 function
- 4 illegal name

Note: `⌈NL` in its dyadic form is not supported. Groups are not supported.

Waterloo microAPL (Waterloo Computing Systems Limited)

- 0 name available for usage
- 1 label
- 2 variable
- 3 defined function
- 4 not a name
- 5 system variable
- 6 system function

Note: Groups are not supported.

WATCOM APL (WATCOM Products Incorporated)

- 0 name available for usage
- 1 label
- 2 variable
- 3 defined function
- 4 not a name
- 5 system variable
- 6 system function

Note: Groups are not supported.

APL/Z80 and APL/V80 (Vanguard Systems Corporation)

- 1 label
- 2 variable
- 3 function

Note: $\square NC$ is not supported. $\square NL$ in its dyadic form is not supported.
Groups are not supported.

Paper prepared for the APL Workgroup ISO-IEC/JTC1/SC22/WG3 (August 1990). Copyright 1990: The Belgian APL-CAM Users Society BACUS.

Joseph L.F. De Kerf
Agfa-Gevaert NV/SA - 5900
Research Laboratories
Septestraat 27
B-2510 Mortsel
Belgium

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please tick box): 90/91

UK private membership	£10	☐
Overseas private membership	£18	☐
Airmail supplement (not needed for Europe)	£8	☐
Corporate membership	£85	☐
Corporate membership overseas	£140	☐
Sustaining membership	£360	☐
Non-voting student membership (UK only)	£5	☐

For student applicants:

Name of course: _____
 Supervisor: _____
 Signature of supervisor: _____

If you would like to pay by credit card ...

Card no (Access/Visa): _____

Signature (and Expiry date) _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Access or Visa number. Please send the completed form to:

BAA Administration,
 Alison Chatterton, 9 Oak Grove,
 HERTFORD SG13 8AT
 England

Index to Advertisers

APL People	8
Cocking and Drury	6
Dyadic Systems Ltd	48
HMW Computing	108
MicroAPL	2,58
APL Booklist (Renaissance Data Systems)	32

All queries regarding advertising in VECTOR should be made to Alison Chatterton, at the address on the inside back cover.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Jonathan Barman,
Hill Top House,
East Garston,
NEWBURY, Berks RG16 7HD
Tel: 048839-575

(not after 10.00pm please!)

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members news) should be sent to Vector Production, c/o Adrian Smith, Brook House, Gilling East, YORK Tel: 04393-385 (6.00pm - midnight).

Product Guide updates should continue to go to Alison Chatterton, as should requests for advertising space.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1990/91 Committee

Chairman:	Peter Donnelly 0420-87024	Dyadic Systems Ltd., Park House, The High Street, ALTON, Hants GU34 1EN
Secretary:	Anthony Camacho 0727-60130	2 Blenheim Rd., St Albans, Herts AL1 4NR.
Treasurer:	John Sullivan 081 462-3628	24 Mounthurst Road, Hayes, Kent BR2 7QN
Journal Editor:	Jonathan Barman, 048839-575	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Activities:	Dr Peter Branson 0737-242950	Oaklands Cottage, Wray Common, Reigate, Surrey RH2 0LE
Education:	Dr Alan Sykes 0792-295296	European Business Management School Swansea University, Singleton Park, SWANSEA SA2 8PP
Publicity:	Jonathan Martin 081 562-5697	(S499) British Airways, PO Box 10, Heathrow Airport, HOUNSLOW, Middlesex TW6 2JA
Technical:	David Eastwood 071-922 8866	MicroAPL Ltd South Bank Technopark, 90 London Road, LONDON SE1 6LN
Recruitment:	Jill Moss 0225-462602	APL People Ltd, The Old Malthouse Clarence St., BATH, Avon BA1 5NS
Projects:	John Searle 081 948-6737	13A Mount Ararat Road, Richmond, Surrey TW10 6PQ
Administration:	Alison Chatterton 0992-552489	9, Oak Grove, HERTFORD, SG13 8AT

Journal Working Group

Editor:	Jonathan Barman	048839-575
Secretary:	Anthony Camacho	0727-60130
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (04393-385)
Support Team:	David Ziemann (071-436 9481), John Searle (081 948-6737), Ray Cannon (0252-874697), Sylvia Camacho, Briget Barman, Gill Smith	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
The Old Malthouse
Clarence St. BATH, BA1 5NS
Tel:0225-462602

Cocking & Drury Ltd
180 Tottenham Court Rd
LONDON, W1P 9LE
Tel:071-436 9481

HMW Computing Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel:071-353 4212
Fax:071-353 3325

MetaTechnics Systems Ltd
356 West Barnes Lane
Molesey Park, Surrey
Tel:081-949-6238
Fax:081-336-1062

I.P. Sharp Associates
1-4 Singer St.
LONDON EC2A 4BQ
Tel:071-867 1166
Fax:071-867 9792

Intelligent Programs Ltd
9 Gun Wharf, 130 Wapping High St,
LONDON, E1 9NH
Tel:071-265 1120

Dyadic Systems Ltd
Park House, The High Street
ALTON, Hants
Tel:0420-87024

APL Impetus Ltd
Rusper, Sandy Lane
Ivy Hatch, SEVENOAKS
Kent TN15 0PD
Tel:0732-885126

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel:071-922 8866

Peter Cyriax Systems
213 Goldhurst Terrace
LONDON NW6 3ER
Tel:071-824-7013
Mobile:0860-377963



The British Computer Society, 13 Mansfield Street, London W1M 0BD.